Journal of Artificial Intelligence Research 20 (2003) 61-124 Submitted 09/02; published 12/03

PDDL2.1 : An Extension to PDDL for Expressing Temporal Planning Domains

Maria Fox MARIA.FOX@CIS.STRATH.AC.UK
Derek Long DEREK.LONG@CIS.STRATH.AC.UK
Department of Computer and Information Sciences
University of Strathclyde, Glasgow, UK

Abstract

In recent years research in the planning community has moved increasingly towards application of planners to realistic problems involving both time and many types of resources. For example, interest in planning demonstrated by the space research community has inspired work in observation scheduling, planetary rover exploration and spacecraft control domains. Other temporal and resource-intensive domains including logistics planning, plant control and manufacturing have also helped to focus the community on the modelling and reasoning issues that must be confronted to make planning technology meet the challenges of application.

The International Planning Competitions have acted as an important motivating force behind the progress that has been made in planning since 1998. The third competition (held in 2002) set the planning community the challenge of handling time and numeric resources. This necessitated the development of a modelling language capable of expressing temporal and numeric properties of planning domains. In this paper we describe the language, PDDL2.1, that was used in the competition. We describe the syntax of the language, its formal semantics and the validation of concurrent plans. We observe that PDDL2.1 has considerable modelling power — exceeding the capabilities of current planning technology — and presents a number of important challenges to the research community.

1. Introduction

In 1998 Drew McDermott released a Planning Domain Description Language, PDDL (McDermott, 2000; McDermott & the AIPS-98 Planning Competition Committee, 1998), which has since become a community standard for the representation and exchange of planning domain models. Despite some dissatisfaction in the community with some of the features of PDDL the language has enabled considerable progress to be made in planning research because of the ease with which systems sharing the standard can be compared and the enormous increase in availability of shared planning resources. The introduction of PDDL has facilitated the scientific development of planning.

Since 1998 there has been a decisive movement in the research community towards application of planning technology to realistic problems. The propositional puzzle domains of old are no longer considered adequate for demonstrating the utility of a planning system — modern planners must be able to reason about time and numeric quantities. Although several members of the community have been working on applications of planning to real domains of this nature for some time (Laborie & Ghallab, 1995; Ghallab & Laruelle, 1994; Muscettola, 1994; Drabble & Tate, 1994; Wilkins, 1988) there has always been a gap

人工智能研究杂志 20 (2003) 61-124 提交于 09/02；发表于 12/03

PDDL2.1 : PDDL 的扩展，用于表达时间规划领域

Maria Fox MARIA.FOX@CIS.STRATH.AC.UK
Derek Long DEREK.LONG@CIS.STRATH.AC.UK
计算机与信息科学系，斯特拉斯克莱德大学，格拉斯哥，英国

摘要

近年来，规划领域的研究越来越倾向于将规划器应用于涉及时间和多种资源类型的实际问题。例如，空间研究社区对规划的兴趣激发了在观测调度、行星漫游车探索和航天器控制领域的相关工作。其他时间敏感和资源密集型领域，包括物流规划、工厂控制和制造业，也帮助社区关注了必须面对的建模和推理问题，以使规划技术满足应用挑战。自1998年以来，国际规划竞赛一直是规划领域取得进展的重要推动力。第三届竞赛（于2002年举行）向规划社区提出了处理时间和数值资源挑战的任务。这需要开发一种能够表达规划领域时间和数值属性的建模语言。在本文中，我们描述了在竞赛中使用的语言，PDDL2.1。我们描述了该语言的语法、形式语义以及并发计划的验证。我们观察到PDDL2.1 具有相当大的建模能力——超过了当前规划技术的功能——并向研究社区提出了一系列重要挑战。

1. 简介

1998年，Drew McDermott发布了一种规划领域描述语言， PDDL (Mc- Dermott, 2000; McDermott & the AIPS-98 Planning Competition Committee, 1998)，该语言已成为规划领域模型表示和交换的社区标准。尽管社区对 PDDL 语言的一些特性存在不满，但由于共享标准的系统可以轻松比较，以及共享规划资源的可用性大幅增加，该语言使得规划研究取得了相当大的进展。 PDDL的引入促进了规划的科学发展。

自1998年以来，研究界在将规划技术应用于现实问题方面取得了决定性进展。旧的命题谜题领域不再被认为是展示规划系统实用性的充分条件——现代规划器必须能够推理时间数和数值。尽管该领域的几个成员已经为将规划应用于此类现实领域工作了一段时间（Laborie & Ghallab, 1995；Ghallab & Laruelle, 1994；Muscettola, 1994；Drabble & Tate, 1994；Wilkins, 1988），但始终存在一个差距

between the modelling requirements of such domains and what can be expressed in PDDL. Application-driven planners come equipped with their own modelling conventions and black arts and, as a consequence, it is difficult to reproduce their results and to make empirical comparisons with other approaches, both of which are essential for scientific progress to be made.

The PDDL language provides the foundation on which an expressive standard can be constructed, enabling the domain models of the applications-driven community to be shared and motivating the development of the planning field towards realistic application. The third International Planning Competition, which took place in 2002, had the objective of closing the gap between planning research and application. As organisers of the third competition the authors therefore took the first step in defining an expressive language capable of modelling a certain class of temporal and resource-intensive planning domains. This had to be done both with an eye to the future and with awareness of the current capabilities of planners (it had to be possible for the language to be used by members of the community, or there would be no competitors). In this paper we describe the resulting language, PDDL2.1, in terms of its syntax, semantics and modelling capabilities.

PDDL2.1 has been designed to be backward compatible with the fragment of PDDL that has been in common usage since 1998. This compatibility supports the development of resources which help to establish a scientific foundation for the field of AI planning. Furthermore, McDermott's original PDDL provides a clean and well-understood basis for development and embodies a number of design principles that we considered it important to retain. PDDL2.1 extends PDDL in principled ways to achieve the additional expressive power following, as far as possible, McDermott's maxim "physics, not advice" (McDermott, 2000). We take this maxim to mean that a language should focus on expressing the physical properties of the world, not advice to the planner about how to search the associated solution spaces. Of course, any model of physical systems makes simplifying assumptions and abstracts behaviours at some level, so no model can be claimed to be purely physics and free of decisions that could influence the use of the model. We do not attempt to make strong judgements about what constitutes advice but try to implement the maxim by keeping the language as simple as possible. We make the following two guarantees of backward compatibility:

1. All existing PDDL domains (in common usage) are valid PDDL2.1 domains. This is important to enable existing libraries of benchmark problems to remain valid.
2. Valid PDDL plans are valid PDDL2.1 plans.

An important contribution made in the development of PDDL2.1 is a means by which domain designers can provide alternative objective functions that can be used to judge the value of a plan. The use of numbers in a domain provides a platform for measuring consumption of critical resources and other parameters. An example of a metric that can be modelled is that energy consumption must be minimized. This is very important for many practical applications of planning in which plan quality might be dependent on a number of interacting domain-dependent factors.

The organisation of the paper is as follows. In Section 2 we introduce non-specialist readers to the PDDL domain description language used in the planning research community.

在这样领域的建模需求与 PDDL中可以表达的内容之间。驱动应用的规划器配备了它们自己的建模约定和黑魔法，因此，很难重现它们的结果，并与其他方法进行实证比较，这两者都是科学进步所必需的。

该 PDDL 语言为构建表达性标准提供了基础，使应用程序驱动社区中的领域模型能够共享，并推动规划领域朝着实际应用方向发展。2002年举办的第三届国际规划竞赛旨在弥合规划研究与实际应用之间的差距。作为第三届竞赛的组织者，作者们首先在定义一种能够建模特定类时间资源和资源密集型规划领域的表现力语言方面迈出了第一步。这项工作既要着眼于未来，又要意识到规划者当前的能力（该语言必须能够被社区成员使用，否则将没有竞争对手）。在本文中，我们根据其语法、语义和建模能力描述了由此产生的语言，PDDL2.1。

PDDL2.1 已设计为向后兼容自1998年以来广泛使用的 PDDL片段。这种兼容性支持开发有助于为AI规划领域建立科学基础的资源。此外，McDermott最初的 PDDL 为开发提供了一个清晰且易于理解的基礎，并体现了我们认为重要保留的若干设计原则。 PDDL2.1 以原则性的方式扩展了 PDDL ，以实现额外的表现力，尽可能遵循McDermott的格言“物理，而非建议”（McDermott，2000）。我们理解这一格言是指语言应专注于表达世界的物理属性，而不是向规划者提供关于如何搜索相关解空间的建议。当然，任何物理系统的模型都在某种程度上做出了简化假设和抽象行为，因此不能声称任何模型是纯粹的物理且完全不受可能影响模型使用的决策的影响。我们试图对构成建议的内容做出强判断，但通过尽可能保持语言简单来实现这一格言。我们做出以下两项向后兼容性保证：

1. 所有现有的 PDDL 域名（在常用中）都是有效的 PDDL2.1 域名。这对于使现有的基准问题库保持有效非常重要。
2. 有效的 PDDL 计划是有效的 PDDL2.1 计划。

在 PDDL2.1 开发中的一个重要贡献是，它为领域设计者提供了一种提供替代目标函数的方法，这些函数可用于判断计划的价值。领域中使用数字提供了一个平台，用于测量关键资源的消耗和其他参数。一个可以建模的指标示例是，必须最小化能源消耗。这对于许多实际应用规划非常重要，在这些应用中，计划质量可能取决于多个相互作用的领域相关因素。

本文的组织结构如下。在第二节中，我们向非专业读者介绍了规划研究社区中使用的 PDDL 领域描述语言。

This background is given in order to provide the foundations for the numeric and durative extensions made in developing PDDL2.1. The paper then focusses on the specific extensions introduced: numeric expressions and durative actions. In Section 3 we start by explaining the syntax of numeric expressions and their use in action descriptions. We then explain, in Section 4, how *metrics* can be provided as part of the problem description so that the quality of a plan involving numeric change can be evaluated in terms appropriate to the problem domain. We present the syntax in which metrics are expressed and give examples.

In Section 5 the paper introduces the notion of *durative* action as a way of modelling the temporal properties of a planning domain. Both discretised and continuous durative actions are considered. The syntax is described and examples of modelling power and limitations are presented in both cases. Having given examples of the syntactic representation of durative actions we present a formal semantics for both discretised and continuous actions and for plans. Sections 6, 7, 8 and 9 provide the details. The semantics gives us a way of tackling the problem of confirming plan validity — something that becomes an important issue in the face of concurrent activity. In Section 10 we describe the process by which plans were validated in the competition and discuss the complexity of the validation question for PDDL2.1. Finally, Section 11 describes some related work in the temporal reasoning community, in order to put the contributions made by PDDL2.1 into a wider context. A full BNF description of PDDL2.1 can be found in the appendix.

PDDL2.1 was developed for use in the third International Planning Competition in which competing planners demonstrated that many discretized temporal and metric models can now be efficiently handled by both domain-independent planners and those using hand-tailored control rules. For ease of reference in the competition we identified the features of PDDL2.1 with a series of *levels* of increasing expressive power. Thus, the STRIPS fragment of PDDL2.1 was referred to as level 1, the numeric extensions comprised level 2, the addition of discretised durative actions resulted in level 3, continuous durative actions resulted in level 4 and a final level, level 5, comprised all of the extensions of PDDL2.1 and additional components to support the modelling of spontaneous events and physical processes. Level 5 is not discussed in this paper but details can be found in earlier work by Fox and Long (2002). The competition focussed on the use of levels 1, 2 and 3 and did not use levels 4 or 5 because the planning technology was not at that stage sufficiently advanced to handle the additional complexities. Despite the fact that level 4 was not used in the competition we devote some discussion to it in this paper. We feel that level 4 presents some important immediate challenges for the planning community that affect the extent to which planning can be applied to real problems.

The purpose of this paper is to provide an overview of the new features introduced in PDDL2.1, discuss the rationale for our language choices and explain some of the issues that have arisen in trying to extend PDDL. Although we have provided the BNF for PDDL2.1 as an appendix, this paper is not intended to be either a language manual or a tutorial on the use of the language. For examples of the use of the language and other relevant materials, readers should consult archived resources currently held at `http://www.dur.ac.uk/d.p.long/competition.html`.

提供这种背景是为了为在开发 PDDL2.1过程中所做的数值和时序扩展奠定基础。本文随后重点关注引入的具体扩展：数值表达式和时序动作。在第三节中，我们首先解释数值表达式的语法及其在动作描述中的使用。然后在第四节中，我们解释如何将度量作为问题描述的一部分提供，以便在适当的问题域范围内评估涉及数值变更的计划的质量。我们展示了度量表达式的语法并给出了示例。

在5节中，论文介绍了持续行动的概念，将其作为建模规划域时间属性的一种方式。考虑了离散化和连续的持续行动。语法被描述，并且在两种情况下都展示了建模能力和局限性的示例。在给出了持续行动的句法表示示例后，我们为离散化和连续行动以及计划提出了形式语义。第6、7、8和9节提供了详细信息。语义为我们提供了一种解决计划有效性确认问题的方式——在并发活动面前，这成为一个重要问题。在10节中，我们描述了在比赛中计划验证的过程，并讨论了 PDDL2.1验证问题的复杂性。最后，第11节描述了时间推理社区中的一些相关工作，以便将 PDDL2.1 的贡献置于更广泛的背景下。 PDDL2.1 的完整 BNF 描述可以在附录中找到。

PDDL2.1 是为第三届国际规划竞赛开发的，在竞赛中，参赛的规划器展示了许多离散时间度量模型现在都可以被领域无关的规划器和使用定制控制规则的规划器高效处理。为了在竞赛中方便参考，我们用一系列表达能力不断增强的级别来识别PDDL2.1 的特征。因此，PDDL2.1 的 STRIPS 片段被称为级别1，数值扩展组成级别2，添加离散持续动作导致级别3，持续持续动作导致级别4，最后一个级别，级别5，包含 PDDL2.1 的所有扩展以及支持自发事件和物理过程建模的附加组件。本文未讨论级别5，但可以在Fox和Long（2002）的早期工作中找到详细信息。竞赛专注于使用级别1、2和3，而没有使用级别4或5，因为当时的规划技术尚未足够先进以处理额外的复杂性。尽管在竞赛中未使用级别4，但本文仍对其进行了部分讨论。我们认为级别4为规划界带来了一些重要的即时挑战，这些挑战影响了规划在现实问题中的应用程度。

本文的目的是概述 PDDL2.1引入的新功能，讨论我们语言选择的原因，并解释在尝试扩展 PDDL 时出现的一些问题。尽管我们已经将 BNF作为附录提供给 PDDL2.1 ，但本文不打算成为语言手册或语言使用的教程。有关语言使用的示例和其他相关资料，读者应查阅目前保存在 http://www.dur.ac.uk/d.p.long/competition.html 的存档资源。

2. PDDL Background

PDDL is an action-centred language, inspired by the well-known STRIPS formulations of planning problems. At its core is a simple standardisation of the syntax for expressing this familiar semantics of actions, using pre- and post-conditions to describe the applicability and effects of actions. The syntax is inspired by Lisp, so much of the structure of a domain description is a Lisp-like list of parenthesised expressions. An early design decision in the language was to separate the descriptions of parameterised actions that characterise domain behaviours from the description of specific objects, initial conditions and goals that characterise a problem instance. Thus, a planning problem is created by the pairing of a domain description with a problem description. The same domain description can be paired with many different problem descriptions to yield different planning problems in the same domain. The parameterisation of actions depends on the use of variables that stand for terms of the problem instance — they are instantiated to objects from a specific problem instance when an action is grounded for application. The pre- and post-conditions of actions are expressed as logical propositions constructed from predicates and argument terms (objects from a problem instance) and logical connectives.

Although the core of PDDL is a STRIPS formalism, the language extends beyond that. The extended expressive power includes the ability to express a type structure for the objects in a domain, typing the parameters that appear in actions and constraining the types of arguments to predicates, actions with negative preconditions and conditional effects and the use of quantification in expressing both pre- and post-conditions. These extensions are essentially those proposed as ADL (Pednault, 1989).

Although the original definition of the PDDL syntax was not accompanied by a formal semantics, the language was really a proposal for a standard syntax for a commonly accepted semantics and there was little scope for disagreement about the meaning of the language constructs. Two parts of the original language proposal for which this claim fails are an attempt to offer a standard syntax for describing hierarchical domain descriptions, suitable for HTN planners and the subset of the language concerned with expressing numeric-valued fluents. The former was an ambitious project to construct a syntax in which the entire structure of domains using hierarchical action decompositions could be expressed. In contrast to STRIPS-based planning, the differences between planners using hierarchical decomposition appear to be deeper, with domain descriptions often containing structures that go beyond the description of domain behaviours (for example, SHOP (Nau, Cao, Lotem, & Muñoz-Avila, 1999) often uses mechanisms that represent goal agendas and other solution-oriented structures in a domain encoding). This diversity undermined the efforts at standardisation in hierarchical domain descriptions and this part of the language has never been successfully explored.

The syntax proposed for expressing numeric-valued fluents was not tested in the first use of the language (in the 1998 competition) and, indeed, it underwent revision in the early development of the language. The second competition in 2000 also avoided use of numeric-valued fluents, so a general agreement about the syntax and semantics of the numeric-expressivity of the language remained unnecessary. McDermott's original PDDL provides support for numbers by allowing numeric quantities to be assigned and updated. The syntax of numeric-valued fluents changed between the PDDL manuals 1.1 and 1.2 (McDermott &

2. PDDL背景

PDDL 是一种以行为为中心的语言，灵感来自著名的 STRIPS 规划问题公式。其核心是对表达行为熟悉语义的语法进行简单标准化，使用前置条件和后置条件来描述行为的应用和效果。语法受Lisp启发，因此领域描述的大部分结构是括号括起来的Lisp样式的表达式列表。语言的一个早期设计决策是将参数化行为（这些行为特征化了领域行为）的描述与特定对象、初始条件和目标（这些特征化了问题实例）的描述分开。因此，规划问题是通过领域描述与问题描述的配对创建的。相同的领域描述可以与许多不同的问题描述配对，以在同一领域内生成不同的规划问题。行为的参数化依赖于代表问题实例术语的变量的使用——当行为被实例化为特定问题实例时，这些变量会被实例化为对象。行为的前置条件和后置条件被表达为从谓词和参数项（问题实例中的对象）以及逻辑连接词构建的逻辑命题。

尽管 PDDL 的核心是一个 STRIPS 形式化系统，但该语言已经超出了这个范围。扩展的表达能力包括能够为领域中的对象表达类型结构、为动作中出现的参数进行类型检查、约束谓词、具有负预条件或条件效果的动作的参数类型，以及使用量词来表示前置条件和后置条件。这些扩展基本上是作为 ADL (Pednault, 1989) 提出的。

尽管 PDDL 的原始语法定义没有附带一个形式化的语义，但该语言实际上是一个针对被广泛接受的语义的标准语法的提议，并且对于语言结构的含义几乎没有争议的空间。这个主张在原始语言提议的两个部分中失败了：一个是为描述分层领域描述提供标准语法的尝试，适合 HTN 规划器，以及与表达数值型流相关的语言子集。前者是一个雄心勃勃的项目，旨在构建一个语法，在其中可以使用分层动作分解表达领域结构的整个结构。与基于STRIPS的规划相比，使用分层分解的规划器之间的差异似乎更深，领域描述通常包含超出领域行为描述的结构（例如， SHOP(Nau, Cao, Lotem, & Mu˜noz- Avila, 1999)经常使用表示目标议程和其他面向解决方案结构的机制来表示领域编码）。这种多样性削弱了分层领域描述标准化工作的努力，并且这部分语言从未被成功探索过。

用于表达数值型流动态的语法在语言（1998年竞赛）的首次使用中未经测试，并且确实在语言的早期开发中进行了修订。2000年的第二次竞赛也避免了使用数值型流动态，因此关于语言数值表达能力的语法和语义的普遍共识仍然没有必要。McDermott的原始 PDDL 通过允许数值量被赋值和更新来支持数字。数值型流动态的语法在 PDDL 1.1和1.2版本的手册之间发生了变化（McDermott &

```
(define (domain jug-pouring)
  (:requirements :typing :fluents)
  (:types jug)
  (:functors
        (amount ?j -jug)
        (capacity ?j -jug)
          - (fluent number))
  (:action empty
        :parameters (?jug1 ?jug2 - jug)
        :precondition (fluent-test
                (>= (- (capacity ?jug2) (amount ?jug2))
                        (amount ?jug1)))
        :effect (and (change (amount ?jug1) 0)
                     (change (amount ?jug2)
                           (+ (amount ?jug1) (amount ?jug2)))))
)
```

Figure 1: Pouring water between jugs as described in the AI Magazine article (McDermott, 2000).

the AIPS-98 Planning Competition Committee, 1998) and the later AI Magazine article on PDDL (McDermott, 2000). McDermott presented a version of numeric fluents used in PDDL in the article in AI Magazine (2000) which could be taken as a definitive statement of the syntax. An example using numeric fluents, presented by McDermott (2000), is shown in Figure 1. This action models an action from the well-known jugs-and-water problem, allowing the water in one jug to be emptied into a second jug provided that the space in the second jug is large enough to hold the water in the first. The effect is a discrete update of the values of the current contents of the jugs by an assignment (denoted here by the `change` token).

Even without the numeric extensions, PDDL is an expressive language, capable of capturing a wide variety of interesting and challenging behaviours. Figure 2 illustrates how PDDL can be used to capture a domain in which a vehicle can move between locations, consuming fuel as it does so.

It can be seen in the example that PDDL includes a syntactic representation of the level of expressivity required in particular domain descriptions through the use of requirements flags. This gives the opportunity for a planning system to gracefully reject attempts to plan with domains that make use of more advanced features of the language than the planner can handle. Syntax checking tools can be used to confirm that the requirements flags are correctly set for a domain and that the types and other features of the language are correctly employed. An example of a problem description to accompany the vehicle domain is shown in Figure 3. The example illustrates that the description of an initial state requires an exhaustive listing of all the (atomic) propositions that hold. Symmetric or transitive relations must be modelled by exhaustive and explicit listing of the propositions that hold. The use of domain axioms to simplify the description of domains that use such relationships has been considered, but remains an untested part of PDDL and therefore


```
(define (domain jug-pouring)
  (:requirements :typing :fluents)
  (:types jug)
  (:functors
        (amount ?j -jug)
        (capacity ?j -jug)
          - (fluent number))
  (:action empty
        :parameters (?jug1 ?jug2 - jug)
        :precondition (fluent-test
                (>= (- (capacity ?jug2) (amount ?jug2))
                        (amount ?jug1)))
        :effect (and (change (amount ?jug1) 0)
                     (change (amount ?jug2)
                           (+ (amount ?jug1) (amount ?jug2)))))
)
```


图1：根据AI杂志文章（McDermott，2000年）所述，在两个壶之间倒水。

AIPS-98规划竞赛委员会，1998年）以及后来的关于 PDDL 的AI杂志文章（McDermott，2000年）。McDermott在2000年AI杂志文章中展示了一个数值流动态的版本，该版本可以被视为对语法的权威声明。McDermott（2000年）展示的一个使用数值流动态的示例显示在图1中。该动作模拟了众所周知的壶和水问题中的一个动作，允许一个壶中的水被倒入第二个壶中，前提是第二个壶的空间足够大，可以容纳第一个壶中的水。其效果是通过赋值（在此处用变化标记表示）对当前壶中内容的值进行离散更新。

即使没有数字扩展， PDDL 也是一种表达性语言，能够捕捉各种有趣和具有挑战性的行为。图2说明了如何使用PDDL 来捕获一个领域，在这个领域中，车辆可以在不同位置之间移动，并在移动过程中消耗燃料。

从示例中可以看出， PDDL 通过使用需求标志包含了对特定领域描述中所需表达性级别的句法表示。这为规划系统提供了机会，使其能够优雅地拒绝尝试使用比规划器能够处理的更高级语言功能的领域进行规划。可以使用句法检查工具来确认需求标志是否正确设置，以及语言类型和其他功能是否正确使用。与车辆领域一起使用的问题描述示例显示在图3中。该示例说明，初始状态的描述需要详尽列出所有（原子）命题。对称或传递关系必须通过对所有成立的命题进行详尽和显式的列出进行建模。已经考虑使用领域公理来简化使用此类关系的领域的描述，但这仍然是 PDDL 中未经测试的部分，因此

```
(define (domain vehicle)
 (:requirements :strips :typing)
 (:types vehicle location fuel-level)
 (:predicates (at ?v - vehicle ?p - location)
              (fuel ?v - vehicle ?f - fuel-level)
              (accessible ?v - vehicle ?p1 ?p2 - location)
              (next ?f1 ?f2 - fuel-level))

 (:action drive
   :parameters (?v - vehicle ?from ?to - location
                ?fbefore ?fafter - fuel-level)
   :precondition (and (at ?v ?from)
                      (accessible ?v ?from ?to)
                      (fuel ?v ?fbefore)
                      (next ?fbefore ?fafter))
   :effect (and (not (at ?v ?from))
                (at ?v ?to)
                (not (fuel ?v ?fbefore))
                (fuel ?v ?fafter))
 )
)
```

Figure 2: A domain description in PDDL.

an unstable part of the syntax. PDDL domains are not case-sensitive, which is somewhat anachronistic in the light of standard practice in modern programming languages.

In the following sections we review the extensions made to PDDL in its development into PDDL2.1, the version of the language used in the third International Planning Competition.

3. Numeric Expressions, Conditions and Effects

One of the first decisions we made in the development of PDDL2.1 was to propose a definitive syntax for the expression of numeric fluents. We based our syntax on the version described in the AI Magazine article (McDermott, 2000), with some minor revisions (discussed below). Numeric expressions are constructed, using arithmetic operators, from primitive numeric expressions, which are values associated with tuples of domain objects by domain functions. Using our proposed syntax for expressing numeric assignments and updates we can express the jug-pouring operator originally described in the PDDL1.2 manual and in the AI Magazine article (see Figure 1), in PDDL2.1, as presented in Figure 4. In this example the functions `capacity` and `amount` associate the jug objects with numeric values corresponding to their capacity and current contents respectively. As can be seen in the example, we have used a prefix syntax for all arithmetic operators, including comparison predicates, in order to simplify parsing. Conditions on numeric expressions are always comparisons between pairs of numeric expressions. Effects can make use of a selection of assignment operations in order to update the values of primitive numeric expressions. These include direct assignment and relative assignments (such as `increase` and `decrease`). Numbers are not distinguished in their possible roles, so values can represent, for example, quantities of resources, accumulating utility, indices or counters.


```
(define (domain vehicle)
 (:requirements :strips :typing)
 (:types vehicle location fuel-level)
 (:predicates (at ?v - vehicle ?p - location)
              (fuel ?v - vehicle ?f - fuel-level)
              (accessible ?v - vehicle ?p1 ?p2 - location)
              (next ?f1 ?f2 - fuel-level))

 (:action drive
   :parameters (?v - vehicle ?from ?to - location
                ?fbefore ?fafter - fuel-level)
   :precondition (and (at ?v ?from)
                      (accessible ?v ?from ?to)
                      (fuel ?v ?fbefore)
                      (next ?fbefore ?fafter))
   :effect (and (not (at ?v ?from))
                (at ?v ?to)
                (not (fuel ?v ?fbefore))
                (fuel ?v ?fafter))
 )
)
```

图2： PDDL中的一个领域描述。

句法的不稳定部分。 PDDL 领域不区分大小写，这在现代编程语言的标准实践中显得有些过时。

在以下几节中，我们回顾了在 PDDL 发展到PDDL2.1（第三国际规划竞赛中使用的语言版本）的过程中所做的扩展。

3. 数字表达式、条件和效果

在 PDDL2.1 的开发过程中，我们做出的第一个决定之一是提出一个用于表达数字流语的确定语法。我们的语法基于AI杂志文章中描述的版本（McDermott，2000），并进行了一些小的修订（将在下文中讨论）。数字表达式使用算术运算符从原始数字表达式构建，原始数字表达式是领域函数与领域对象元组相关联的值。使用我们提出的用于表达数字赋值和更新的语法，我们可以表达 PDDL1.2 手册和AI杂志文章中最初描述的倒水操作符（见图1），在 PDDL2.1中，如图4所示。在这个例子中，函数capacity和amount将壶对象与分别对应其容量和当前内容的数字值关联起来。从示例中可以看出，我们为所有算术运算符（包括比较谓词）使用前缀语法，以简化解析。数字表达式上的条件总是成对数字表达式之间的比较。效果可以利用一系列赋值操作来更新原始数字表达式的值。这包括直接赋值和相对赋值（例如增加和减少）。数字在其可能的角色中不被区分，因此值可以表示，例如，资源的数量、累积效用、索引或计数器。

```
(define (problem vehicle-example)
 (:domain vehicle)
 (:objects
      truck car - vehicle
      full half empty - fuel-level
      Paris Berlin Rome Madrid - location)
 (:init
      (at truck Rome)
      (at car Paris)
      (fuel truck half)
      (fuel car full)
      (next full half)
      (next half empty)
      (accessible car Paris Berlin)
      (accessible car Berlin Rome)
      (accessible car Rome Madrid)
      (acessible truck Rome Paris)
      (accessible truck Rome Berlin)
      (accessible truck Berlin Paris)
 )
 (:goal (and (at truck Paris)
             (at car Rome))
 )
)
```

Figure 3: A problem instance associated with the vehicle domain.

```
(define (domain jug-pouring)
   (:requirements :typing :fluents)
   (:types jug)
   (:functions
        (amount ?j - jug)
        (capacity ?j - jug))

   (:action pour
       :parameters (?jug1 ?jug2 - jug)
       :precondition (>= (- (capacity ?jug2) (amount ?jug2)) (amount ?jug1))
       :effect (and (assign (amount ?jug1) 0)
                    (increase (amount ?jug2) (amount ?jug1)))
)
```

Figure 4: Pouring water between jugs, PDDL2.1 style.


```
(define (problem 车辆示例)
 (:domain 车辆)
 (:objects
      卡车 汽车 - 车辆
      满 半 空 - 燃油量
      巴黎 柏林 罗马 马德里 - 位置)
 (:init
      (at truck Rome)
      (at car Paris)
      (fuel truck half)
      (fuel car full)
      (next full half)
      (next half empty)
      (accessible car Paris Berlin)
      (accessible car Berlin Rome)
      (accessible car Rome Madrid)
      (acessible truck Rome Paris)
      (accessible truck Rome Berlin)
      (可访问卡车柏林巴黎)
 )
 (:goal (and (at卡车巴黎)
             (at小汽车罗马))
 )
)
```

图3：一个与车辆领域相关的问题实例。

```
(define (domain倒水)
    (:requirements :typing :fluents)
    (:types 桶)
    (:functions
         (amount ?j - 桶)
         (capacity ?j - 桶))

    (:action 倒
        :parameters (?jug1 ?jug2 - 桶)
        :前提条件 (>=  (- (容量 ?jug2) (数量 ?jug2)) (数量 ?jug1))
        :效果 (and (赋值 (数量 ?jug1) 0))
                   (increase (amount ?jug2) (amount ?jug1)))
)
```

图4：在壶之间倒水，PDDL2.1 风格。

The differences between the PDDL2.1 syntax and the AI Magazine syntax are in the declaration of the functions and in the use of `assign` instead of `change`. We decided to only allow numeric-valued functions, making the declaration of function return types superfluous. We therefore simplified the language by requiring only the declaration of the function names and argument types, as is required for predicates. We felt that `change` was ambiguous when used alongside the operations `increase` and `decrease` and that `assign` would be clearer.

Numeric expressions are not allowed to appear as terms in the language (that is, as arguments to predicates or values of action parameters). There are two justifications for this decision — a philosophical one and a pragmatic one. Philosophically we take the view that there are only a finite number of objects in the world. Numbers do not exist as unique and independent objects in the world, but only as values of attributes of objects. Our models are object-oriented in the sense that all actions can be seen as methods that apply to the objects given as their parameters. The object-oriented view does not directly inform the syntax of our representations, but is reflected in the way in which numbers are manipulated only through their relationships with the objects that are identified and named in the initial state. Pragmatically, many current planning approaches rely on being able to instantiate action schemas prior to planning, and this is only feasible if there is a finite number of action instances. The branching of the planner's search space, at choice points corresponding to action selection, is therefore always over finite ranges. The use of numeric fluent variables conflicts with this because they could occur as arguments to any predicate and would not define finite ranges.

Our decision not to allow numbers to be used as arguments to actions rules out some actions that might seem intuitively reasonable. For example, an action to *fly* at a certain altitude might be expected to take the altitude as a number-valued argument. This is only possible in PDDL2.1 if the range of numbers that can be used is finite. From a practical point of view we think that this is unlikely to be an arduous constraint and that the benefits of keeping the logical state space finite compensates for any modelling awkwardness that results.

Functions in PDDL2.1 are restricted to be of type $Object^n \rightarrow \mathbb{R}$, for the (finite) collection of objects in a planning instance, *Object* and finite function arity n. Later extensions of PDDL might introduce functions of type $Object^n \rightarrow Object$, allowing *Object* to be extended by the application of functions to other objects. The advantage of this would be to allow objects to be referred to by their relationships to known objects. (For example, `(onTopOf ?x)` could be used to refer to the object currently on top of an object instantiating `?x`). Unfortunately, such functions present various semantic problems. In particular, the interpretation of quantified preconditions becomes significantly harder, since the collection of objects is no longer necessarily finite, so extensional interpretations are not possible. A further difficulty is the identity problem — as objects are manipulated by actions, the functional expressions that refer to them are also affected, but implicitly. For example, as objects are moved, `(onTopOf A)` can change without any action manipulating it explicitly. Managing the way in which functional terms map to specific objects in the domain (which might or might not have specific names of their own) appears to introduce considerable complication into the semantics. We believe that it is important to avoid extending PDDL with elements that are still poorly understood.

PDDL2.1 语法与AI Magazine语法的差异在于函数的声明和使用assign代替change。我们决定只允许数值型函数，这使得函数返回类型的声明变得多余。因此，我们通过仅要求声明函数名称和参数类型来简化语言，这与谓词的要求相同。我们觉得change在与increase和decrease操作一起使用时很模糊，而assign会更清晰。

数值表达式不允许作为语言中的项出现（也就是说，不允许作为谓词的参数或动作参数的值）。这个决定有两个理由——一个哲学上的和一个实用上的。从哲学上看，我们认为世界上只有有限数量的对象。数字不是作为独特且独立的对象存在于世界上，而只是作为对象属性值的值。我们的模型是面向对象的，因为所有动作都可以看作是应用于其参数对象的方法。面向对象的观点没有直接影响我们表示的语法，但体现在数字仅通过它们与初始状态中识别和命名的对象的关系来操作的方式中。从实用上看，许多当前的规划方法依赖于在规划之前能够实例化动作模式，而这只有在存在有限数量的动作实例时才可行。因此，规划器的搜索空间在对应于动作选择的决策点上的分支总是覆盖有限的范围。数值流变量（numeric fluent variables）的使用与此冲突，因为它们可以作为任何谓词的参数出现，并且不会定义有限的范围。

我们决定不允许数字作为动作的参数，这排除了某些看似直观合理的动作。例如，一个在特定高度飞行的动作可能会被期望将高度作为数值参数。这在 PDDL2.1 中只有当可使用的数字范围是有限时才可能。从实际角度来看，我们认为这不太可能是一个艰巨的限制，并且保持逻辑状态空间有限的优点可以补偿任何由此产生的建模不便。

PDDL2.1 中的函数被限制为类型$Object^n \rightarrow \mathbb{R}$，对于规划实例中（有限的）对象集合，Object和有限函数arity n。后续对 PDDL 的扩展可能会引入类型为 $Object^n \rightarrow Object$ 的函数，允许通过将函数应用于其他对象来扩展Object。这样做的好处是允许通过对象与已知对象的关系来引用对象。（例如，(onTopOf ?x)可用于引用当前位于实例化？x的对象上）。不幸的是，此类函数会带来各种语义问题。特别是，量化前提的解释变得显著困难，因为对象集合不再一定是有限的，因此无法进行外延解释。另一个困难是身份问题——由于对象通过动作进行操作，引用它们的函数表达式也会受到影响，但这是隐式的。例如，随着对象的移动，(onTopOf A)可能会改变，而没有任何动作明确地对其进行操作。管理函数项映射到域中特定对象的方式（这些对象可能有或没有自己的特定名称）似乎给语义带来了相当大的复杂性。我们认为，避免用仍不完全理解的因素扩展 PDDL非常重要。

4. Plan Metrics

The adoption of a stable numeric extension to the PDDL core allowed us to introduce a further extension into PDDL2.1, namely a new (optional) field within the specification of problems: a plan metric. Plan metrics specify, for the benefit of the planner, the basis on which a plan will be evaluated for a particular problem. The same initial and goal states might yield entirely different optimal plans given different plan metrics. Of course, a planner might not choose to use the metric to guide its development of a solution but just to evaluate a solution *post hoc*. This approach might lead to sub-optimal, and possibly even poor quality plans, but it is a pragmatic approach to the handling of metrics which was quite widely used in the competition. This issue is discussed further in the companion paper analysing the results of the 3rd IPC in this issue (Long & Fox, 2003b).

The value `total-time` can be used to refer to the temporal span of the entire plan. Other values must all be built from primitive numeric expressions defined within a domain and manipulated by the actions of the domain. As a consequence, plan metrics can only express non-temporal metrics in PDDL2.1 domains using numeric expressions. Any arithmetic expression can be used in the specification of a metric — there is no requirement that the expression be linear. It is the domain designer's responsibility to ensure that plan metrics are well-defined (for example, do not involve divisions by zero). An example of use of a plan metric is shown in Figure 5.

The implications of having introduced this extension are far-reaching and have already helped to demonstrate some important new challenges for planning systems — particularly fully-automated systems. An enriched descriptive power for the evaluation of plans is a crucial extension for the practical use of planners, since it is almost never the case that real plans are evaluated solely by the number of actions they contain.

Metrics are described in the problem description, allowing a modeller to easily explore the effect of different metrics in the construction of solutions to problems for the same domain. In order to define a metric in terms of a specific quantity it is necessary to *instrument* that quantity in the domain description. For example, if the metric is defined in terms of overall fuel use a *fuel-use* quantity can be initialised to zero in the initial state and then updated every time fuel is consumed. In the domain shown in Figure 5 it is possible to minimise a linear combination of fuel used by each of the vehicles such as:

```
(:metric minimize (+ (* 2 (fuel-used car)) (fuel-used truck)))
```

However it is not possible to minimise distance covered since distance is not instrumented. It would be straightforward to instrument it if desired, simply by adding the appropriate initial value and incrementing effects to the domain description. Since actions cause quantities to change, instrumenting a value requires modification of the domain description itself, not just a problem file.

The use of plan metrics is subtle and can have dramatic impact on the plans being sought. Perhaps the simplest case is where all actions increase a metric that must be minimised, or decrease one that must be maximised. This is the case in the example shown in Figure 5, where any use of the drive action can only worsen the value of the plan metric (whether we use the metric shown in the figure or the maximising metric described in the last paragraph). This situation might appear to be relatively straightforward: a planner

4. 计划指标

PDDL 核心的稳定数值扩展使我们能够在 PDDL2.1中引入进一步的扩展，即在问题规范中引入一个新的（可选）字段：计划指标。计划指标为规划器指定，用于在特定问题中评估计划的基础。相同初始和目标状态可能会根据不同的计划指标产生完全不同的最优计划。当然，规划器可能选择不使用指标来指导其解决方案的开发，而只是在事后评估解决方案。这种方法可能导致次优甚至质量较差的计划，但它是一种处理指标的实用方法，这在比赛中被广泛使用。这个问题在分析本刊第3届IPC结果的伴生论文中进一步讨论（Long & Fox, 2003b）。

值 total-time 可用于指代整个计划的时序跨度。其他值必须全部基于领域内定义的原始数值表达式，并由领域的操作进行操作。因此，计划指标只能在 PDDL2.1 领域中使用数值表达式来表示非时序指标。任何算术表达式都可以用于指标的规范——表达式不需要是线性的。确保计划指标定义良好（例如，不涉及除以零）是领域设计者的责任。计划指标的一个使用示例显示在图 5 中。

引入此扩展的影响是深远的，并且已经帮助证明了规划系统的一些重要新挑战——特别是全自动化系统。对于规划器的实际使用而言，对计划评估的丰富描述能力是一个关键的扩展，因为几乎从未是仅通过计划包含的操作数量来评估真实计划的情况。

指标在问题描述中进行了描述，允许建模者轻松探索相同领域问题解决方案中不同指标的影响。为了以特定数量来定义一个指标，需要在领域描述中对该数量进行测量。例如，如果指标是以总燃料使用来定义的，可以在初始状态中将燃料使用量初始化为零，然后在每次消耗燃料时进行更新。在图5所示的领域中，可以最小化每辆车的燃料使用的线性组合，例如：

(:metric minimize (+ (* 2 (fuel-used car)) (fuel-used truck)))

然而无法最小化行驶距离，因为距离没有被测量。如果需要，可以很容易地对其进行测量，只需在领域描述中添加适当的初始值和增量效果即可。由于动作会导致数量发生变化，测量一个值需要修改领域描述本身，而不仅仅是问题文件。

计划指标的使用微妙且可能对所寻求的计划产生巨大影响。也许最简单的情况是所有操作都会增加一个必须最小化的指标，或减少一个必须最大化的指标。这在图5中所示的示例中就是这样，任何使用驱动操作都只能降低计划指标的价值（无论是我们使用图中的指标还是上一段描述的最大化指标）。这种情况似乎相对简单：一个规划器

```
(define (domain metricVehicle)
 (:requirements :strips :typing :fluents)
 (:types vehicle location)
 (:predicates (at ?v - vehicle ?p - location)
              (accessible ?v - vehicle ?p1 ?p2 - location))
 (:functions  (fuel-level ?v - vehicle)
              (fuel-used ?v - vehicle)
              (fuel-required ?p1 ?p2 - location)
              (total-fuel-used))

 (:action drive
   :parameters (?v - vehicle ?from ?to - location)
   :precondition (and (at ?v ?from)
                      (accessible ?v ?from ?to)
                      (>= (fuel-level ?v) (fuel-required ?from ?to)))
   :effect (and (not (at ?v ?from))
                (at ?v ?to)
                (decrease (fuel-level ?v) (fuel-required ?from ?to))
                (increase (total-fuel-used) (fuel-required ?from ?to))
                (increase (fuel-used ?v) (fuel-required ?from ?to)))
 )
)

(define (problem metricVehicle-example)
 (:domain metricVehicle)
 (:objects
      truck car - vehicle
      Paris Berlin Rome Madrid - location)
 (:init
      (at truck Rome)
      (at car Paris)
      (= (fuel-level truck) 100)
      (= (fuel-level car) 100)
      (accessible car Paris Berlin)
      (accessible car Berlin Rome)
      (accessible car Rome Madrid)
      (accessible truck Rome Paris)
      (accessible truck Rome Berlin)
      (accessible truck Berlin Paris)
      (= (fuel-required Paris Berlin) 40)
      (= (fuel-required Berlin Rome) 30)
      (= (fuel-required Rome Madrid) 50)
      (= (fuel-required Rome Paris) 35)
      (= (fuel-required Rome Berlin) 40)
      (= (fuel-required Berlin Paris) 40)
      (= (total-fuel-used) 0)
      (= (fuel-used car) 0)
      (= (fuel-used truck) 0)
 )
 (:goal (and (at truck Paris)
             (at car Rome))
 )
 (:metric minimize (total-fuel-used))
)
```

Figure 5: An example of a domain and problem instance describing a plan metric.


```
(define (domain metricVehicle)
 (:requirements :strips :typing :fluents)
 (:types vehicle location)
 (:predicates (at ?v - vehicle ?p - location)
              (accessible ?v - vehicle ?p1 ?p2 - location))
 (:functions  (fuel-level ?v - vehicle)
              (fuel-used ?v - vehicle)
              (fuel-required ?p1 ?p2 - location)
              (total-fuel-used))

 (:action drive
   :parameters (?v - vehicle ?from ?to - location)
   :precondition (and (at ?v ?from)
                      (accessible ?v ?from ?to)
                      (>= (fuel-level ?v) (fuel-required ?from ?to)))
   :effect (and (not (at ?v ?from))
                (at ?v ?to)
                (decrease (fuel-level ?v) (fuel-required ?from ?to))
                (increase (total-fuel-used) (fuel-required ?from ?to))
                (increase (fuel-used ?v) (fuel-required ?from ?to)))
 )
)

(define (problem metricVehicle-example)
 (:domain metricVehicle)
 (:objects
      truck car - vehicle
      Paris Berlin Rome Madrid - location)
 (:init
      (at truck Rome)
      (at car Paris)
      (= (fuel-level truck) 100)
      (= (fuel-level car) 100)
      (accessible car Paris Berlin)
      (accessible car Berlin Rome)
      (accessible car Rome Madrid)
      (accessible truck Rome Paris)
      (accessible truck Rome Berlin)
      (accessible truck Berlin Paris)
      (= (fuel-required Paris Berlin) 40)
      (= (fuel-required Berlin Rome) 30)
      (= (fuel-required Rome Madrid) 50)
      (= (fuel-required Rome Paris) 35)
      (= (fuel-required Rome Berlin) 40)
      (= (fuel-required Berlin Paris) 40)
      (= (total-fuel-used) 0)
      (= (fuel-used car) 0)
      (= (fuel-used truck) 0)
 )
 (:goal (and (at truck Paris)
             (at car Rome))
 )
 (:metric minimize (total-fuel-used))
)
```

图5：一个描述计划度量的域和问题实例的示例。

must attempt to use as few actions to solve a problem as possible. In fact, even this case is a little more complex than it appears — there can be rival plans in which one uses more actions but has lower overall cost than the other. A more complex case arises when some actions improve the quality metric while others degrade it. For example, if we use the maximising metric but also add a refuel action to the domain then driving will degrade plan quality (by reducing the fuel level of a vehicle) but refuelling will improve plan quality (by increasing the fuel level of a vehicle). In this case, a planner can attempt to use actions to improve the plan quality without those actions actually contributing to achieving the goals. For example, refuelling might not be necessary to get the vehicles to their destinations, but adding refuelling actions would improve the quality of a solution. This process could involve trading off finite and irreplaceable resources for the increased value of the plan. This would be the case if, for example, refuelling a vehicle took fuel from a finite reservoir. Alternatively a domain could allow plans of arbitrarily high value to be constructed by using more and more actions. This would occur in the metric vehicles domain using the maximising vehicle's fuel level metric if refuelling were not constrained, since the domain does not impose a limit on the fuel capacities of the vehicles.

The case in which plans are constrained by finite availability of resources, is an important and interesting form of the planning problem, but the case in which plans of arbitrarily high utility can be constructed, is obviously an ill-defined problem, since an optimal plan does not exist. It is non-trivial to determine whether a planning problem provided with a metric is ill-defined. In fact, as Helmert shows (Helmert, 2002), the introduction of numeric expressions, even in the constrained way that we have adopted in PDDL2.1, makes the planning problem undecidable. The problem of finding a collection of actions which does not consume irreplaceable resources and has an overall beneficial impact on a plan metric is at least as hard as the planning problem. Therefore it is clear that determining whether a planning problem is even well-defined is undecidable. This does not make it worthless to consider planning with metrics, of course, but it demonstrates that the modelling problem, as well as the planning problem, becomes even more complex when metrics are introduced.

One strategy available to planners working with problems subject to plan metrics is to ignore the metric and simply produce a plan to satisfy the logical goals that a problem specifies. In this case, the plan quality will simply be the value, according to the metric, of the plan that happens to be constructed. This strategy is unsophisticated and it is obviously better for a planner to construct a plan guided by the specified metric. How best to use a metric to expedite the search process in a fully-automated planner is still a research issue.

5. Durative Actions

Most recent work on temporal planning (Smith & Weld, 1999; Bacchus & Kabanza, 2000; Do & Kambhampati, 2001) has been based on various forms of durative action. In order to facilitate participation in the competition we therefore developed two forms of durative action allowing the specification only of restricted forms of timed conditions and effects in their description. Although constrained in certain ways, these durative actions are, nevertheless, more expressive than many of the proposals previously explored, particularly in the way that they allow concurrency to be exploited. The two forms are *discretised* durative actions and *continuous* durative actions.

必须尽可能少地使用动作来解决一个问题。事实上，即使在这种情况下也比看起来复杂一些——可能存在其他计划，其中一个使用更多动作，但总成本低于另一个。当一些动作提高质量指标，而另一些动作降低它时，会出现更复杂的情况。例如，如果我们使用最大化指标，但在领域中添加了一个加油动作，那么驾驶会降低计划质量（通过降低车辆的燃料水平），但加油会提高计划质量（通过增加车辆的燃料水平）。在这种情况下，规划器可以尝试使用动作来提高计划质量，而这些动作实际上并没有帮助实现目标。例如，加油可能不是让车辆到达目的地的必要条件，但添加加油动作会提高解决方案的质量。这个过程可能涉及用计划价值的增加来权衡有限的、不可替代的资源。如果例如，给车辆加油需要从有限的储油库中消耗燃料，这种情况就会发生。或者，领域可以允许通过使用越来越多的动作来构建任意高价值的计划。这会在使用最大化车辆燃料水平指标的度量车辆领域中发生，如果加油不受约束，因为领域没有对车辆的燃料容量施加限制。

当计划受限于有限资源可用性时的情况，是规划问题中一个重要且有趣的形式，但可以构建任意高效的计划的情况显然是一个定义不明确的问题，因为不存在最优计划。确定一个带有度量的规划问题是否定义不明确并不简单。事实上，正如 Helmert 所示（Helmert, 2002），即使采用我们在 PDDL2.1中采用的约束方式引入数值表达式，也会使规划问题不可解。寻找一个不消耗不可替代资源且对计划度量有整体积极影响的动作集合，至少和规划问题一样困难。因此很明显，确定一个规划问题是否定义良好本身是不可解的。当然，这并不使考虑带度量的规划变得毫无价值，但它表明，当引入度量时，建模问题以及规划问题都变得更加复杂。

一种可供规划者使用的策略是忽略指标，仅生成一个满足问题指定逻辑目标的计划。在这种情况下，计划质量将简单地是所构建计划根据指标的价值。这种策略很原始，显然，规划者根据指定的指标来构建计划会更好。如何最好地使用指标来加速全自动规划器的搜索过程仍然是一个研究问题。

5. 持续动作

最新的时序规划工作（Smith & Weld, 1999；Bacchus & Kabanza, 2000；Do & Kambhampati, 2001）基于各种形式的持续动作。为了便于参与比赛，我们因此开发了两种形式的持续动作，允许在其描述中仅指定受限形式的定时条件和效果。尽管在某些方面受到限制，但这些持续动作仍然比先前探索的许多提案更具表达能力，特别是在它们允许并发利用的方式上。这两种形式是离散持续动作和连续持续动作。

```
(:durative-action load-truck
   :parameters (?t - truck)
               (?l - location)
               (?o - cargo)
               (?c - crane)
   :duration (= ?duration 5)
   :condition (and (at start (at ?t ?l))
                   (at start (at ?o ?l))
                   (at start (empty ?c)
                   (over all (at ?t ?l))
                   (at end (holding ?c ?o))
   :effect (and (at end (in ?o ?t))
                (at start (holding ?c ?o))
                (at start (not (at ?o ?l)))
                (at end (not (holding ?c ?o)))
)
```

Figure 6: A durative action for loading a truck. We assume no capacity constraints.

Both forms rely on a basic durative action structure consisting of the logical changes caused by application of the action. We always consider logical change to be instantaneous, therefore the continuous aspects of a continuous durative action refer only to how numeric values change over the interval of the action. Figure 6 depicts a basic durative action, *load-truck*, in which there is no numeric change.

The modelling of temporal relationships in a discretised durative action is done by means of *temporally annotated* conditions and effects. All conditions and effects of durative actions must be temporally annotated. The annotation of a condition makes explicit whether the associated proposition must hold at the *start* of the interval (the point at which the action is applied), the *end* of the interval (the point at which the final effects of the action are asserted) or over the interval from the start to the end (invariant over the duration of the action). The annotation of an effect makes explicit whether the effect is immediate (it happens at the start of the interval) or delayed (it happens at the end of the interval). No other time points are accessible, so all discrete activity takes place at the identified start and end points of the actions in the plan.

Invariant conditions in a durative action are required to hold over an interval that is open at both ends (starting and ending at the end points of the action). These are expressed using the *over all* construct seen in Figures 6 and 8. If one wants to specify that a fact p holds in the closed interval over the duration of a durative action, then three conditions are required: *(at start p), (over all p)* and *(at end p)*.

We considered adopting the convention that *over all* constraints should apply to the start and end points as well as the open interval inside the durative action, but decided against this because it would then be impossible to express conditions that are actually only required to hold over this open interval. Examples of actions in which conditions are invariant only over the open interval include the action of loading a truck. The truck must remain at the loading location throughout the loading interval, but it can start to move away simultaneously with the loading being completed. The reason is that the start of the


```
(:durative-action load-truck
   :parameters (?t - truck)
               (?l - location)
               (?o - cargo)
               (?c - crane)
   :duration (= ?duration 5)
   :condition (and (at start (at ?t ?l))
                   (at start (at ?o ?l))
                   (at start (empty ?c)
                   (over all (at ?t ?l))
                   (at end (holding ?c ?o))
   :effect (and (at end (in ?o ?t))
                (at start (holding ?c ?o))
                (at start (not (at ?o ?l)))
                (at end (not (holding ?c ?o)))
)
```


图6：一个用于装载货车的持续动作。我们假设没有容量限制。

两种形式都依赖于一个基本的持续性行动结构，该结构由应用行动所引起的逻辑变化组成。我们始终认为逻辑变化是瞬时的，因此持续性行动中的连续方面仅指数值如何在行动的区间内变化。图6描绘了一个基本的持续性行动，load-truck，其中没有数值变化。

在离散化的持续性行动中对时间关系进行建模是通过时间标注的条件和效果来完成的。所有持续性行动的条件和效果都必须进行时间标注。条件的标注明确说明相关命题必须在区间的开始（行动应用时点）、结束（行动最终效果断言时点）或从开始到结束的区间内（在整个行动持续期间保持不变）成立。效果的标注明确说明效果是即时的（它在区间的开始发生）还是延迟的（它在区间的结束发生）。没有其他时间点是可访问的，因此所有离散活动都在计划中行动的已识别开始和结束点发生。

持续动作中的不变条件必须在一个两端都开放的区间内保持（从动作的端点开始和结束）。这些条件使用图6和图8中看到的overall结构来表示。如果想要指定一个事实 p在持续动作的闭区间内成立，那么需要三个条件：(at start p)，(overall p)和(at end p)。

我们考虑采用一种惯例，即所有约束都应适用于起始点和终点，以及持续性行为内部的开放区间，但决定不采用这种惯例，因为那样将无法表达实际上只需要在开放区间内成立的条件。仅开放区间内条件不变的持续性行为的例子包括装载货车的行为。货车必须在装载区间内始终保持在装载位置，但可以与装载完成的同时开始移动。原因是驾驶行为与装载结束是非互斥的，因此任何计划中驾驶行为在装载完成瞬间开始都有合理的解释。影响不变条件（例如货车位置）的行为只有在不变条件不被约束为在终点本身成立时，才能与持续性行为的终点同时执行。这突显了（所有）与（所有和终点）之间的重要区别。如果条件既作为终点前置条件又作为不变条件被要求，其含义是任何影响不变条件的行为都必须在要求该不变条件的行动结束后开始。例如，如果我们使（货车位置）既成为装载操作符的终点前置条件又成为不变条件，其后果是货车必须在装载完成的瞬间之后才能驶离。

drive action is non-mutex with the end of the load so there is a reasonable interpretation of any plan in which driving starts at the instant that loading is completed. Actions that affect an invariant condition (such as the location of the truck) can be executed simultaneously with the end point of a durative action only if the invariant is not constrained to hold true at the end point itself. This highlights an important difference between (*over all*) and (*over all* and *at end*). If a condition is required as an end precondition as well as an invariant condition the meaning is that any action that affects the invariant must start after the end of the action requiring that invariant. For example, if we make *(at truck location)* an end precondition of the load operator as well as an invariant, the consequence is that the truck cannot drive away until after the instant at which the load has completed.

Note that, in our definition of the *load-truck* action in Figure 6, we have chosen to make the condition *(holding ?c ?o)* be a start effect and an end precondition but not an invariant condition. This means that the crane could temporarily cease to hold the cargo at some time during the interval, as long as it is holding the cargo in time to deposit it at the end of the loading interval. This makes the action quite flexible, enabling the exploitation of concurrent uses of the crane where applicable.

The *load-truck* example shows how logical change can be wrapped up into durative actions that encapsulate much of the detail involved in achieving an effect by a sequence of connected activities. Naturally it would be useful to be able to combine such actions concurrently within a plan. In the next section we consider the extent to which concurrency is allowed and the ways in which concurrent plans are interpreted.

5.1 The Interpretation of Concurrent plans

When time is introduced into the modelling of a domain it is possible for concurrent activity to occur in a plan. Prior to the introduction of time into PDDL all plans were interpreted as sequential — even Graphplan-concurrent plans were sequenced before being validated — so concurrency was never an issue. In PDDL2.1 plan validity can depend on exploiting concurrency correctly. Actions can overlap and co-occur, giving rise to questions over the interpretation of synchronous behaviour. We discuss the problems arising in precise synchronization in Section 10. We now explain under what constraints actions can occur concurrently within a plan involving durative actions and numeric conditions and effects.

The key difference, between durative actions in PDDL2.1 and those used by planners prior to the competition, is that we distinguish between the conditions and effects at the start and end points of the durative interval and the invariant conditions that might be specified to hold over the interval. That is, actions can have pre- and postconditions that are local to the two end-points of the action, and a planner can choose to exploit a durative action for effects it has at its start or at its end. Conditions that are invariant are distinguished from pre-conditions, enabling the exploitation of a higher degree of concurrency than is possible if preconditions are not distinguished from invariants, as in TGP (Smith & Weld, 1999), TPSys (Garrido, Onaindía, & Barber, 2001) and TP4 (Haslum & Geffner, 2001). We discuss the consequences of these design decisions, together with several examples of durative actions, in the following sections.

It is important to observe that our view of time is point-based rather than interval-based. That is, we see a period of activity in terms of intervals of state separated by time points

影响不变条件（例如货车位置）的行为只有在不变条件不被约束为在终点本身成立时，才能与持续性行为的终点同时执行。这突显了（所有）与（所有和终点）之间的重要区别。如果条件既作为终点前置条件又作为不变条件被要求，其含义是任何影响不变条件的行为都必须在要求该不变条件的行动结束后开始。例如，如果我们使（货车位置）既成为装载操作符的终点前置条件又成为不变条件，其后果是货车必须在装载完成的瞬间之后才能驶离。

请注意，在我们的图6中定义的 load-truck 动作中，我们选择将条件（holding ?c ?o）作为起始效果和结束前置条件，而不是不变条件。这意味着起重机可以在某个时间点暂时停止抓取货物，只要它在装载间隔结束时仍能抓取货物即可。这使得该动作非常灵活，能够利用起重机在适用情况下的并发使用。

load-truck 示例展示了如何将逻辑变化封装到持续动作中，这些动作封装了通过一系列连接活动实现效果所涉及的大部分细节。当然，能够在计划中并发组合此类动作会很有用。在下一节中，我们考虑并发允许的程度以及并发计划如何被解释。

5.1 并发计划的解释

当时间被引入到领域建模中时，计划中可能会发生并发活动。在将时间引入 PDDL 之前，所有计划都被解释为顺序的——即使 Graphplan 并发计划在验证之前也被排序——因此并发从未成为一个问题。在 PDDL2.1 计划的有效性可能取决于正确利用并发。动作可以重叠和同时发生，从而产生关于同步行为解释的问题。我们在第10节讨论精确同步中产生的问题。现在我们解释在涉及持续动作和数值条件及效果的计划中，动作可以同时发生的约束条件。

在 PDDL2.1 中的持续性动作与比赛前规划者使用的动作之间的关键区别在于，我们区分了持续性区间起点和终点处的条件与效果，以及可能指定为在整个区间内保持不变的条件。也就是说，动作可以具有在动作的两个端点处局部化的前置和后置条件，而规划者可以选择利用持续性动作来利用其在起点或终点处的效果。不变条件与前置条件区分开来，这使得能够利用比前置条件与不变条件不区分时更高的并发度，正如在 TGP (Smith & Weld, 1999)， TPSys (Garrido, Onaindía, & Barber, 2001)和 TP4 (Haslum & Geffner, 2001)中所示。我们在以下几节中讨论了这些设计决策的后果，并提供了几个持续性动作的例子。

需要注意的是，我们看待时间的观点是基于时间点的，而不是基于时间间隔的。也就是说，我们将一个活动时期看作是由时间点分隔的状态间隔

at which state-changing activities occur. All logical state change occurs instantaneously, at the start or end point of a durative action. Propositions are true over half-open intervals that are closed on the left and open on the right. Activities might change logical state or they might update the values of numeric variables. In the discretised view of time we allow for only a finite number of activities (which we call *happenings*) between any two time points, although time itself is considered continuous and actions can be scheduled to begin at any time point.

For a plan to be considered valid, no logical condition can be both asserted and negated at the same instant. We impose the further constraint that no logical condition can both be required to hold and be asserted at the same instant. Although this might seem overly strong we claim that a plan cannot be guaranteed to be valid if the instant at which a proposition is required is exactly the instant at which it is asserted. We require that, for an action with precondition P to start at time t, there must be a half open interval immediately preceding t in which P holds. This is mathematically inconsistent with P being asserted at the instant at which it is required. We are conservative in our view of the validity of simultaneous update of and access to a state proposition. For example, if we have two instantaneous actions, A and B, where A has precondition P and effects (not P) and Q, while B has precondition $P \vee Q$ and effect R, we consider that an attempt to apply A and B simultaneously in a state in which P holds is ill-defined. The reason is that, although A switches the state from one in which P holds into one in which Q holds so one might suppose the precondition of B to be secure, A is an abstraction of a model in which the values of P and Q are changing and, we argue, any reliance on their values at this point of change is unstable. We adopt a rule we call *no moving targets*, by which we mean that no two actions can simultaneously make use of a value if one of the two is accessing the value to update it — the value is a moving target for the other action to access. This rule creates a behaviour for propositions in a planning state that is very much like the behaviour of variables in shared memory protected by a *mutex lock* (such as those in POSIX threads), with a difference between read and write access to the variable.

Validity also requires that no numeric value be accessed and updated simultaneously at the start or end point of a durative action. In the case of discretised durative actions, all numeric change is modelled in terms of step functions so numeric values can be accessed, or updated, during the interval of another durative action acting on that value (we provide examples in the following section) provided that any updates are consistent with all invariant properties dependent on the value. In the case of continuous durative actions, values can be simultaneously accessed and updated during the continuous process of change occurring in the interval of an action. In both the discretised and continuous cases we allow multiple simultaneous updates provided the update operations are commutative.

In order to implement the mutual exclusion relation we require *non-zero*-separation between mutually exclusive action end points. In our view, when end points are non-conflicting they can be treated as though it is possible to execute them simultaneously even though precise synchronicity cannot be achieved in the world. However, when end points are mutually exclusive the planner should buffer the co-occurrence of these points by explicitly separating them. In this way we ensure that the concurrency in the plan is at least plausible in the world.

来衡量的，状态变化的活动发生在这些时间点。所有逻辑状态变化都是瞬时发生的，发生在持续性行动的起点或终点。命题在左闭右开的半开区间内为真。活动可能会改变逻辑状态，或者更新数值变量的值。在离散化的时间观中，我们允许在任意两个时间点之间只有有限数量的活动（我们称之为事件），尽管时间本身被认为是连续的，并且行动可以安排在任何时间点开始。

要使一个计划被认为是有效的，没有任何逻辑条件可以在同一时刻既被断言又被否定。我们进一步施加约束，即没有任何逻辑条件可以在同一时刻既被要求成立又被断言。尽管这可能看起来过于严格，但我们声称，如果一个命题被要求成立的时间正好是它被断言的时间，那么这个计划不能被保证是有效的。我们要求，对于具有前置条件 P的动作在时间 t 开始，必须在 t 之前的一个左开区间内满足 P。这在数学上与 P 在它被要求成立的时间被断言相矛盾。我们对同时更新和访问状态命题的有效性持保守观点。例如，如果我们有两个瞬时动作， A 和 B，其中 A 有前置条件 P 和效果（不是 P）和 Q，而 B 有前置条件 $P \vee Q$ 和效果 R，我们认为在一个满足 P 的状态中同时应用 A 和B 是定义不明确的。原因是，尽管A 将状态从一个满足 P 的状态切换到一个满足 Q 的状态，因此人们可能会认为 B 的前置条件是安全的，但 A 是一个模型抽象，在该模型中 P和 Q 的值正在变化，我们认为在任何依赖它们在这个变化点的值都是不稳定的。我们采用一条称为“不移动的目标”的规则，意思是如果两个动作中有一个是访问值以更新它，那么没有两个动作可以同时使用该值——该值是另一个动作访问的移动目标。这条规则为规划状态中的命题创建了一种行为，非常类似于受互斥锁（如POSIX线程中的那些）保护的共享内存中的变量的行为，区别在于对变量的读访问和写访问。

有效性还要求在持续动作的起始点或终点处，不能同时访问和更新数值。对于离散化的持续动作，所有数值变化都通过阶跃函数建模，因此数值可以在另一个作用于该值的持续动作的区间内被访问或更新（我们在下一节提供示例），前提是任何更新都必须与所有依赖于该值的不变属性保持一致。对于连续的持续动作，值可以在动作区间内变化的连续过程中同时被访问和更新。在离散化和连续的情况下，我们允许多个同时更新，前提是更新操作是可交换的。

为了实现互斥关系，我们需要在互斥动作的终点之间保持非零间隔。在我们看来，当终点不冲突时，它们可以被视为可以同时执行，即使在这个世界中无法实现精确同步。然而，当终点互斥时，规划器应该通过明确地将它们分开来缓冲这些点的共存。通过这种方式，我们确保计划中的并发性至少在这个世界中是合理的。

```
(:durative-action heat-water
   :parameters (?p - pan)
   :duration (= ?duration (/ (- 100 (temperature ?p)) (heat-rate)))
   :condition (and (at start (full ?p))
                   (at start (onHeatSource ?p))
                   (at start (byPan))
                   (over all (full ?p))
                   (over all (onHeatSource ?p))
                   (over all (heating ?p))
                   (at end (byPan)))
   :effect (and    (at start (heating ?p))
                   (at end (not (heating ?p)))
                   (at end (assign (temperature ?p) 100)))
)
```

Figure 7: A simple durative action for boiling a pan of water.

Planners can exploit considerable concurrency in a domain by ensuring only that conflicting start and end points of actions are separated by a non-zero amount. A detailed specification of the mutual exclusion relation of PDDL2.1 is given in Section 8. We further discuss the implications of non-zero separation in Section 10.

5.2 Numeric Change within Discretised Durative Actions

This section explains how continuous change can sometimes be modelled in PDDL2.1 using durative actions with discrete effects. This is achieved by using step functions to describe instantaneous changes at the beginnings or ends of the durations of actions. Appendix A details the language constructs involved.

An example of a durative action, illustrating the use of numeric update operations, is shown in Figure 7. In this example showing a water heating action, the conditions *(full ?p)* and *(onHeatSource ?p)* must hold at the start of the interval as well as during the interval. To model this we enter these conditions as both *at start* and *over all* constraints. The action achieves as its start effect that the water is heating, and this condition is maintained invariant over the whole interval of the action. This is an example of an operator that achieves its own invariant condition, and draws attention to the fact that *over all* conditions hold over an interval that is open on the left (as well as on the right).

It should be noted that the actions in Figures 7 and 8 use fixed duration specifications. In the case of the water-boiling example this means that it is impossible to adjust the length of time over which the pan is heated and this has an impact on the context in which the action can be used. In particular, when an *assign* construct is used to update a numeric value, it is not possible for concurrent activity to affect the same value or else the model will be flawed. Because the water heating example uses an *assign* construct no concurrent activity should affect the temperature of the water. It is the responsibility of the modeller to ensure that the temperature is neither accessed nor updated during the interval over which the action is executing.

(:durative-action heat-water
   :parameters (?p - pan)
   :duration (= ?duration (/ (- 100 (temperature ?p)) (heat-rate)))
   :condition (and (at start (full ?p))
                   (at start (onHeatSource ?p))
                   (at start (byPan))
                   (over all (full ?p))
                   (over all (onHeatSource ?p))
                   (over all (heating ?p))
                   (at end (byPan)))
   :effect (and   (at start (heating ?p))
                   (at end (not (heating ?p)))
                   (at end (assign (temperature ?p) 100)))
)

图7：一个简单的持续动作，用于煮沸一锅水。

规划器可以通过确保动作的冲突起点和终点之间保持非零间隔来利用一个领域的相当大的并发性。PDDL2.1 的互斥关系的详细规范在第8节中给出。我们在第10节进一步讨论了非零间隔的影响。

5.2 离散持续动作中的数值变化

本节解释了如何在 PDDL2.1 使用具有离散效果的持续动作来建模连续变化。这是通过使用阶跃函数来描述动作持续时间开始或结束时的瞬时变化来实现的。附录A详细介绍了涉及的语言结构。

一个持续动作的示例，说明了数值更新操作的使用，如图7所示。在这个展示水加热动作的示例中，条件（full ?p）和（onHeatSource ?p）必须在区间的开始以及整个区间内都成立。为了建模这一点，我们将这些条件作为在开始时和在整个约束中的条件输入。该动作以其开始效果实现水正在加热，并且这个条件在整个动作的整个区间内保持不变。这是一个实现自身不变条件的算子的示例，并引起了人们对这样一个事实的注意：在区间上所有的条件都保持不变，该区间在左侧是开放的（以及在右侧）。

需要注意的是，图7和图8中的操作使用了固定时长规范。以水沸腾为例，这意味着无法调整锅加热的时间长度，这会对操作可使用的上下文产生影响。特别是，当使用赋值结构来更新数值时，并发活动不可能影响相同的值，否则模型将存在缺陷。由于水加热示例使用了赋值结构，因此没有并发活动应影响水的温度。建模人员有责任确保在操作执行的时间间隔内，温度既不被访问也不被更新。

```
(:durative-action navigate
:parameters (?x - rover ?y - waypoint ?z - waypoint)
:duration (= ?duration (travel-time ?y ?z))
:condition (and (at start (available ?x))
                (at start (at ?x ?y))
                (at start (>= (energy ?x)
                              (* (travel-time ?y ?z) (use-rate ?x))))
                (over all (visible ?y ?z))
                (over all (can_traverse ?x ?y ?z)))
:effect (and (at start (decrease (energy ?x)
                              (* (travel-time ?y ?z) (use-rate ?x))))
             (at start (not (at ?x ?y)))
             (at end (at ?x ?z))))

(:durative-action recharge
:parameters (?x - rover ?w - waypoint)
:duration (= ?duration (recharge-period ?x))
:condition (and (at start (at ?x ?w))
                (at start (in-sun ?w))
                (at start (<= (energy ?x) (capacity ?x)))
                (over all (at ?x ?w)))
:effect (at end (increase (energy ?x) (* ?duration (recharge-rate ?x)))))
```

Figure 8: Discretised durative actions for a rover to move between locations and to recharge.

We decided to leave it to the modeller to ensure correct behaviour of the *assign* construct because we did not want to forbid the modelling of truly discontinuous updates. For example, a durative action that models the deposit of a cheque in a bank account might have a duration of three days, with a discontinuous update to the account balance at the end of that interval — it would be inappropriate to prevent actions from accessing the balance during the three day period. In general, modelling continuous change with discrete effects is open to various pitfalls. This is the price that is paid for the convenience of not having to specify the details of the continuous processes.

The use of discretised durative actions in combination with numeric (step-function) updates requires care in modelling. In particular, it relies on the notion of *conservative resource updating*. The updating of resource levels is conservative if the consumption of a resource is modelled as if it happens at the start of a durative action, even though it actually happens continuously over the duration of the action, and production of a resource is modelled as if it happens at the end of the durative action even though, again, it might actually be produced continuously over the interval.

As an example of a discretised durative action, Figure 8 shows how the action of a rover navigating between two points is modelled. The local precondition of the start of the period is that the rover be at the start location. Local effects include that the rover consumes an appropriate amount of energy and that it is at the destination. The first of these is conservative and therefore immediate, while the second is a logical effect that occurs at the end point. This organisation ensures that no parallel activities will consume energy that has already been committed to the navigation activity. Similarly, the recharge action


```
(:durative-action navigate
:parameters (?x - rover ?y - waypoint ?z - waypoint)
:duration (= ?duration (travel-time ?y ?z))
:condition (and (at start (available ?x))
                (at start (at ?x ?y))
                (at start (>= (energy ?x)
                              (* (travel-time ?y ?z) (use-rate ?x))))
                (over all (visible ?y ?z))
                (over all (can_traverse ?x ?y ?z)))
:effect (and (at start (decrease (energy ?x)
                              (* (travel-time ?y ?z) (use-rate ?x))))
             (at start (not (at ?x ?y)))
             (at end (at ?x ?z))))

(:durative-action recharge
:parameters (?x - rover ?w - waypoint)
:duration (= ?duration (recharge-period ?x))
:condition (and (at start (at ?x ?w))
                (at start (in-sun ?w))
                (at start (<= (energy ?x) (capacity ?x)))
                (over all (at ?x ?w)))
:effect (at end (increase (energy ?x) (* ?duration (recharge-rate ?x)))))
```


图8：为漫游车在位置之间移动和充电而离散化的持续动作。

我们决定将分配结构的正确行为留给模型来确保，因为我们不想禁止对真正不连续更新的建模。例如，一个模拟在银行账户中存入支票的持续动作可能有三天的时间，并在该时间间隔结束时对账户余额进行不连续更新——在三天期间阻止动作访问余额是不恰当的。总的来说，用离散效果模拟连续变化容易陷入各种陷阱。这是不指定连续过程细节所带来的便利代价。

在离散化的持续动作与数值（阶跃函数）更新相结合时，建模需要小心。特别是，它依赖于保守资源更新的概念。如果资源消耗被建模为在持续动作的开始时发生，即使它实际上在整个动作期间持续发生，并且资源的产生被建模为在持续动作结束时发生，即使它实际上可能在整个区间内持续产生，那么资源水平的更新就是保守的。

作为离散化持续动作的一个例子，图8展示了如何对漫游车在两点之间导航的动作进行建模。该时段的局部前置条件是漫游车位于起始位置。局部效应包括漫游车消耗适量的能量以及它到达目的地。其中第一个是保守的，因此是立即发生的，而第二个是一个逻辑效应，发生在终点。这种组织方式确保没有任何并行活动会消耗已经承诺给导航活动的能量。类似地，充电动作

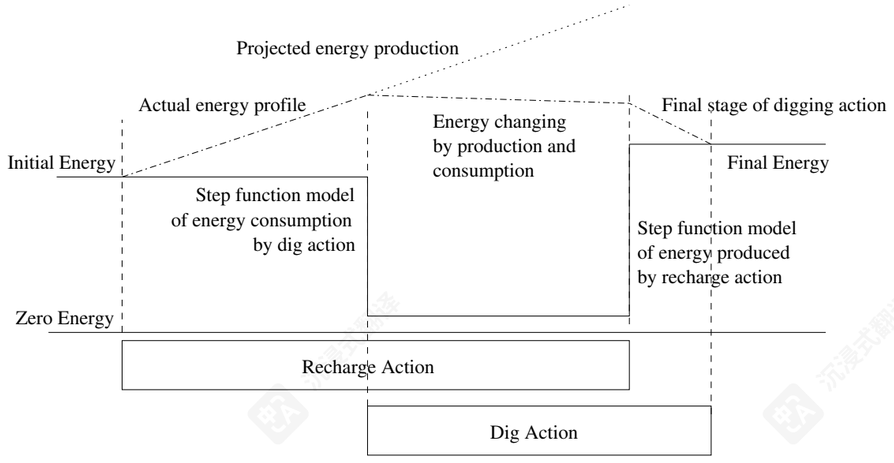

Figure 9: Using discrete actions to model the production and consumption of a resource. In reality, the recharge activity produces energy continuously and the concurrent dig activity continuously consumes it. The conservative model using step functions requires that the energy consumed by digging must be available at the start of that action, despite not having yet updated the model to show the additional energy accumulated because of the part of the recharge action so far executed. The final energy level is consistent with having used a continuous model.

only makes new charge available at the conclusion of the action, so that charge gained cannot be exploited until after the recharging is complete. The use of conservative updates ensures that a model does not support invalid concurrency.

Figure 9 illustrates how a recharging and a digging action (that consumes energy) would interact under a conservative energy consumption model. This model would allow concurrent actions to consume energy provided they did not consume more energy than was left under the conservative assumption that the dig action consumed all of its demands at the start and the recharge action produced nothing until the end. Note that the example assumes energy constraints but no capacity constraint.

The use of conservative updates is subtle. If there were a capacity constraint on the energy level of the rover then one would need to consider two separate resources: the energy itself and the space available for storage of energy. The dig action would consume energy at the start and only produce space at the end, while the recharge action would consume space at the start and produce charge at the end. Using this combination it would be possible to ensure that plans did not consume either resource before it was available.

Durative actions can have conditional effects. The antecedents and consequents of a conditional effect are temporally annotated so that it is possible to specify that the condition be checked at start or at end, and that the effect be asserted at either of these points. The

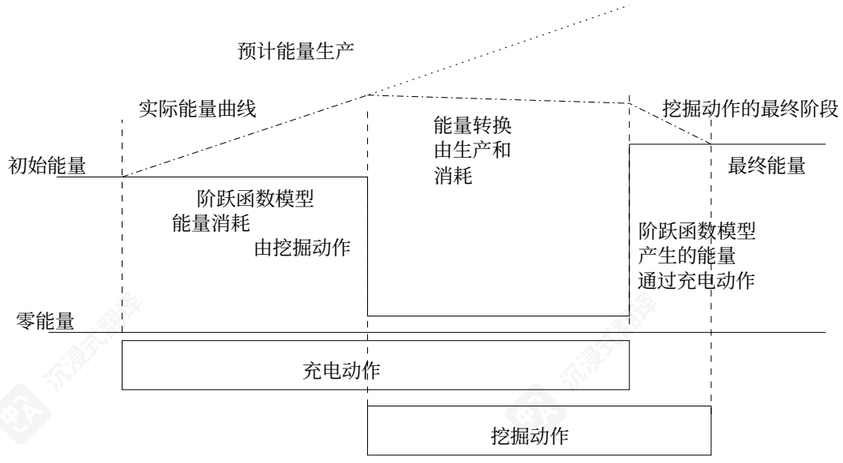

图9：使用离散动作来模拟资源的生产和消耗。现实中，充电活动持续产生能量，而并发挖掘活动持续消耗它。使用阶跃函数的保守模型要求挖掘消耗的能量必须在动作开始时可用，尽管模型尚未更新以显示由于充电动作部分执行而累积的额外能量。最终能量水平与使用连续模型一致。

只在动作结束时才提供新的充电量，因此获得的充电量只有在充电完成后才能利用。保守更新的使用确保模型不支持无效并发。

图9说明了在保守能耗模型下，充电和挖掘动作（消耗能量）将如何交互。该模型允许并发动作消耗能量，前提是它们消耗的能量不超过保守假设下挖掘动作在开始时消耗了所有需求，而充电动作直到结束都没有产生能量的情况。请注意，该示例假设存在能量约束但没有容量约束。

保守更新的使用是微妙之处。如果对探测器的能量水平存在容量约束，那么就需要考虑两个独立的资源：能量本身和用于存储能量的空间。挖掘动作会在开始时消耗能量，只在结束时产生空间，而充电动作会在开始时消耗空间，在结束时产生电荷。使用这种组合，可以确保计划在资源可用之前不会消耗任何资源。

持续动作可以有条件效果。条件效果的前件和后件都进行了时间标注，以便指定在开始或结束时检查条件，以及在这些点之一断言效果。The

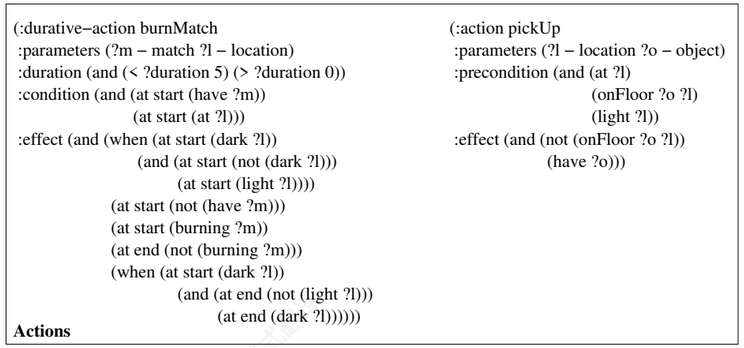

Figure 10: An example of a problem with a durative action useful for its start effects. The burning match produces the light necessary to pick up the coin.

semantics makes clear that a well-formed durative action with conditional effects cannot require the condition to be checked after the effect has been asserted. Conditional effects arise in all PDDL2.1 variants. We discuss how their occurrence in discretized durative actions is interpreted in Section 8.1.

PDDL2.1 allows the specification of duration inequalities enabling actions to be described in which external factors can be involved in determining their temporal extent. In the match-burning example shown in Figure 10 it can be seen that the effect at the start point is the only one of interest, so a planner would exploit this action for its start rather than its end effect. The duration inequality specifies that the match will burn for no longer than a specified upper bound. The model shows that the match can be put out early if the planner considers it appropriate. We discuss the use of duration inequalities further in Section 5.3.

图 10：一个具有持续性动作且其开始效果有用的问题示例。燃烧的火柴产生了拾起硬币所需的火光。

语义学明确表明，一个具有条件效果的格式良好的持续动作不能要求在效果断言后检查条件。条件效果在所有 PDDL2.1 变体中都出现。我们在第 8.1 节讨论了它们在离散持续动作中发生时的解释。

PDDL2.1 允许指定持续时间不等式，从而能够描述在哪些外部因素可以参与确定其时间范围的动作。在图10中显示的匹配燃烧示例中可以看出，起始点处的效果是唯一感兴趣的，因此规划器会利用该动作的起始效果而不是结束效果。持续时间不等式指定了火柴将在不超过指定上限的时间内燃烧。该模型显示，如果规划器认为合适，火柴可以提前熄灭。我们在第5.3节进一步讨论持续时间不等式的使用。

5.3 Durative Actions with Continuous Effects

The objective of discrete durative actions is to abstract out continuous change and concentrate on the end points of the period over which change takes place. The syntax allows precise specification of the discrete changes at the end points of durative actions. However, when a plan needs to manage continuously changing values, as well as discretely changing ones, the durative action language and semantics need to be more powerful. General durative actions can have continuous as well as discrete effects. These increase, or decrease, some numeric variable according to a specified rate of change over time for that variable. When determining how to achieve a goal a planner must be able to access the values of these continuous quantities at arbitrary points on the time-line of the plan. We use `#t` to refer to the continuously changing time from the start of a durative action during its execution. For example, to express the fact that the fuel level of a plane, ?p, decreases continuously, as a function of the consumption rate of ?p, we write:

```
(decrease (fuel-level ?p) (* #t (consumption-rate ?p)))
```

This is distinctly different from:

```
(at end (decrease  (fuel-level ?p)
                   (* (flight-time ?a ?b) (consumption-rate ?p))))
```

because the latter is a single update happening at the end point of the flight action, whilst the former allows the correct calculation of the fuel level of the plane at any point in that interval. The former is a continuous effect, whilst the latter is a discrete one. Continuous effects are not temporally annotated because they can be evaluated at any time during the interval of the action. `#t` is local to each durative action, so that each durative action has access to a purely local "clock". Another way to interpret the expression representing continuous change is as a differential equation:

$$\frac{d}{dt}\texttt{(fuel-level ?p)} = \texttt{(consumption-rate ?p)}$$

We chose to use the `#t` symbol instead of a differential equation because it is possible for two concurrent actions to be simultaneously modifying the same quantity. In that case, the use of differential equations would actually form an inconsistent pair of simultaneous equations, rather than having the intended effect of a combined contribution to the changing value of the quantity. Although all of the expressions describing continuous change take the form of a product of `#t` and some quantity, it is possible to express complex change using them with interdependent concurrent effects. For example, acceleration arises by simply increasing distance using a quantity describing velocity, while at the same time increasing velocity using a quantity describing acceleration. When dependencies between the changing terms include mutual dependencies between terms then the differential equations that arise can lead to continuous change dictated by exponential, logarithmic and exponential functions.

A plan containing continuous durative actions can assign to, consult, and continuously modify the same numeric variables concurrently (see Example 1).

In Figures 12 and 14 the discrete and continuous actions for heating a pan of water are presented (this simple model ignores heat loss). The discrete action presented in Figure 12 modifies the version presented in Figure 7 by the use of a duration *inequality* constraint.

5.3 持续动作与连续效果

离散持续动作的目标是抽象出连续变化并集中关注变化发生的期间端点。语法允许精确指定持续动作端点处的离散变化。然而，当计划需要管理连续变化值以及离散变化值时，持续动作语言和语义需要更强大。通用持续动作可以具有连续和离散效果。这些效果根据指定的时间变化率增加或减少某个数值变量。在确定如何实现目标时，规划器必须能够在计划的时序上的任意点上访问这些连续量的值。我们使用 #t 来指代持续动作执行期间从起始点开始的连续变化时间。例如，为了表达飞机燃料水平 p 根据 p 的消耗率连续减少的事实，我们写：

(decrease (fuel-level ?p) (* #t (consumption-rate ?p)))

这与明显不同：

```
(at end (decrease  (fuel-level ?p)
                   (* (flight-time ?a ?b) (consumption-rate ?p))))
```

因为后者是在飞行动作的终点发生的一次更新，而前者允许在该时间间隔内的任何时间点正确计算飞机的油量。前者是连续效果，而后者是离散的。连续效果不会进行时间标注，因为它们可以在动作的时间间隔内的任何时间进行评估。#t 是每个持续动作的局部变量，因此每个持续动作都可以访问一个纯粹的局部“时钟”。另一种解释表示连续变化的表达式的另一种方式是微分方程：

$$\frac{d}{dt}\texttt{(fuel-level ?p)} = \texttt{(consumption-rate ?p)}$$

我们选择使用 #t 符号而不是微分方程，因为两个并发动作有可能同时修改同一个量。在这种情况下，使用微分方程实际上会形成一对不一致的联立方程，而不是产生预期效果，即对变化量的综合贡献。尽管所有描述连续变化的表达式都采用 #t 与某个量的乘积形式，但它们可以用来表达复杂的连续变化，其中并发效果相互依赖。例如，加速度通过使用描述速度的量来简单地增加距离，同时在增加使用描述加速度的量来增加速度。当变化项之间的依赖关系包括项之间的相互依赖关系时，产生的微分方程会导致由指数、对数和指数函数决定的连续变化。

一个包含持续进行动作的计划可以同时分配、查询和持续修改相同的数值变量（参见示例1）。

在图12和图14中，展示了加热一锅水的离散和连续动作（该简单模型忽略了热量损失）。图12中展示的离散动作通过使用持续时间不等式约束修改了图7中展示的版本。

Example 1 *In the flying and refuelling example shown in Figure 11 it can be seen that the invariant condition, that the fuel-level be greater than (or equal to) zero during the flight, has to be maintained whilst the fuel is continuously decreasing. This could be expressed with discrete durative actions by abstracting out the continuous decrease and making the final value available at the end point of the flight. However, if a refuel operation happens during the flight time (in mid-air) then the fuel level after the flight will need to be calculated by taking into account both the continuous rate of consumption and the refuel operation. A discrete action could not calculate the fuel-level correctly because it would only have access to the distance between the source and destination of the flight, together with the rate of consumption, to determine the final fuel level. In order to calculate the fuel level correctly it is necessary to determine the time at which the refuel takes place, and to use the remaining flight-time to calculate the fuel consumed. Discrete durative actions do not give access to time points other than their own start and end points.*
Discrete durative actions can be used to express the desired combinations of flying and refuelling by providing additional durative actions, such as fly-and-refuel, that encapsulate all of the interactions just described and end up calculating the fuel level correctly. However, this approach requires more of the domain designer than it does of the planner — the domain designer must anticipate every useful combination of behaviours and ensure that appropriate encapsulations are provided.
In contrast with the discrete form, the continuous action, in which the fuel consumption effect is given in terms of `#t`*, is powerful enough to express the fact that the mid-flight refuelling of the plane affects the final fuel level in a way consistent with maintaining the invariant of the fly action.*

```
(:durative-action fly
     :parameters (?p - airplane ?a ?b - airport)
     :duration (= ?duration (flight-time ?a ?b))
     :condition (and (at start (at ?p ?a))
                     (over all (inflight ?p))
                     (over all (>= (fuel-level ?p) 0)))
     :effect (and (at start (not (at ?p ?a)))
                  (at start (inflight ?p))
                  (at end (not (inflight ?p)))
                  (at end (at ?p ?b))
                  (decrease (fuel-level ?p)
                            (* #t (fuel-consumption-rate ?p)))))

(:action midair-refuel
     :parameters (?p)
     :precondition (inflight ?p)
     :effect (assign (fuel-level ?p) (fuel-capacity ?p)))
```

Figure 11: A continuous durative action for flying.

在图11所示飞行和加油示例中可以看出
该不变条件，即燃料量应大于（或等于）零
在飞行过程中，必须保持，同时燃料不断减少。 *This*
可以用离散的持续动作来表示，通过抽象出连续
减少并在飞行终点提供最终值。然而，
如果在飞行时间（空中）发生加油操作，则燃油量
飞行后需要计算，同时考虑连续
飞行源点和目的地的距离，以及消耗率，以确定最终油量。
飞行源点和目的地的距离，以及消耗率，以确定最终油量。
飞行源点和目的地的距离，以及消耗率，以确定最终油量。
飞行源点和目的地的距离，以及消耗率，以确定最终油量。 飞行源点和目的地的距离，以及消耗率，以确定最终油量。
有必要确定加油发生的时间，并使用
剩余飞行时间以计算消耗的燃料。 离散持续动作
仅授予访问其自身起始点和结束点之外的时间点的权限。
离散持续动作可用于表达所需的飞行组合
并通过提供额外的持续性动作（如飞油）来补充燃料。
封装所有上述交互并最终计算燃料水平
正确。然而，这种方法需要领域设计师付出更多努力
而不是规划者——领域设计师必须预见所有有用的行为组合
并确保提供适当的封装。
与离散形式相比，连续动作，其中燃料消耗效应以 #t 的形式给出，足够强大以表达事实
在 #t 下给出的效应足以表达事实
空中加油会影响飞机的最终油量，其方式与
保持飞行动作的不变性。

```
(:durative-action fly
     :parameters (?p - airplane ?a ?b - airport)
     :duration (= ?duration (flight-time ?a ?b))
     :condition (and (at start (at ?p ?a))
                     (over all (inflight ?p))
                     (over all (>= (fuel-level ?p) 0)))
     :effect (and (at start (not (at ?p ?a)))
                  (at start (inflight ?p))
                  (at end (not (inflight ?p)))
                  (at end (at ?p ?b))
                  (decrease (fuel-level ?p)
                            (* #t (fuel-consumption-rate ?p)))))

(:action midair-refuel
     :parameters (?p)
     :precondition (inflight ?p)
     :effect (assign (fuel-level ?p) (fuel-capacity ?p)))
```

图11：一个连续的持续动作，用于飞行。

```
(:durative-action heat-water
   :parameters (?p - pan)
   :duration (at end (<= ?duration (/ (- 100 (temperature ?p))
                                                    (heat-rate))))
   :condition (and (at start (full ?p))
                   (at start (onHeatSource ?p))
                   (at start (byPan))
                   (over all (full ?p))
                   (over all (onHeatSource ?p))
                   (over all (heating ?p))
                   (at end (byPan)))
   :effect (and    (at start (heating ?p))
                   (at end (not (heating ?p)))
                   (at end (increase (temperature ?p)
                                     (* ?duration (heat-rate)))))
)
```

Figure 12: A discrete durative action for heating a pan of water, using a variable duration.

Duration inequalities add significant expressive power over duration equalities. Duration constraints that express inequalities are associated with an additional requirements flag because of the extended expressiveness over fixed-duration discrete durative actions.

In both actions, the logical post-condition of the start of the period is that the pan is heating. The conditions that the pan be heating, full and on the heat source are invariant, although the presence of the agent (by the pan) is only a local precondition of the two end-points and is not invariant. In the first action the duration is modelled by expressing the following duration inequality constraint:

```
(at end (<= ?duration (/ (- 100 (temperature ?p)) (heat-rate))))
```

and the effect at the end-point of the discrete durative action is that the temperature of the pan is increased by `(* ?duration (heat-rate))` (where `heat-rate` is a domain constant). In the continuous action of Figure 14 the duration constraint is unnecessary since the invariant

```
(over all (<= (temperature ?p) 100))
```

is added to ensure that the pan never exceeds boiling.

The durative action in Figure 12 models the heating pan in the face of possible concurrent activities affecting the temperature. The duration inequality allows the planner to adapt the duration to take account of other temperature-affecting activity in a way that is not possible when the duration is specified using an equality constraint. The duration constraint ensures that the temperature never exceeds boiling by checking, as a precondition for the updating activity, that the computed temperature increase can be executed without exceeding the boiling point. If this temperature increase would exceed boiling the plan is invalid. The temperature at the end of the interval of execution is computed from the current temperature and the heating rate, together with the duration over which the heating action has been active (see further discussion in Example 2).


```
(:durative-action heat-water
   :parameters (?p - pan)
   :duration (at end (<= ?duration (/ (- 100 (temperature ?p))
                                                    (heat-rate))))
   :condition (and (at start (full ?p))
                   (at start (onHeatSource ?p))
                   (at start (byPan))
                   (over all (full ?p))
                   (over all (onHeatSource ?p))
                   (over all (heating ?p))
                   (at end (byPan)))
   :effect (and    (at start (heating ?p))
                   (at end (not (heating ?p)))
                   (at end (increase (temperature ?p)
                                     (* ?duration (heat-rate)))))
)
```

图12：对一锅水进行加热的离散持续动作，使用可变持续时间。

持续时间不等式比持续时间等式具有显著的表示能力。表示不等式的持续时间约束与一个附加的要求标志相关联，因为它们在固定持续时间的离散持续动作上具有扩展的表示能力。

在这两种动作中，周期的起始的逻辑后置条件是锅正在加热。锅正在加热、装满并且在热源上的条件是不变的，尽管代理（通过锅）的存在只是两个端点的局部前置条件，并且不是不变的。在第一种动作中，持续时间通过表达以下持续时间不等式约束来建模：

```
(at end (<= ?duration (/ (- 100 (temperature ?p)) (heat-rate))))
```

并且在离散持续动作的端点处的效果是锅的温度增加了(* ?duration (heat-rate))（其中heat-rate是一个领域常量）。在图14的连续动作中，持续时间约束是不必要的，因为不变性

```
(over all (<= (temperature ?p) 100))
```

被添加以确保锅永远不会超过沸腾。

图12中的持续动作模拟了在可能存在并发活动影响温度的情况下加热锅。持续时间不等式允许规划器根据其他影响温度的活动调整持续时间，而使用等式约束指定持续时间时则无法做到这一点。持续时间约束确保温度不会超过沸点，通过在更新活动之前检查计算出的温度升高是否可以在不超出沸点的情况下执行。如果这个温度升高会超过沸点，则计划无效。执行间隔结束时的温度是根据当前温度、加热速率以及加热动作已激活的持续时间计算得出的（见示例2的进一步讨论）。

Example 2 *If a plan attempts to further heat the pan (say by applying a blowtorch to the pan), during the heat-water interval then, provided that the concurrent action ends before the end of the heat-water action, the duration constraint will be seen to have been violated if the duration has been chosen so that the overall increase in temperature would exceed boiling. If the concurrent activity ends simultaneously with the heat-water action then the no-moving-targets rule would be violated because the duration constraint would attempt to access the temperature at the same time point as the concurrent action attempted to update it.*
Figure 13 depicts these two situations. In this figure, apply-blowtorch is a durative action that applies heat to an object (in this case, the pan). In part (a) of the figure the duration constraint will be violated if the duration of the heat-water action is sufficient to cause the temperature to increase beyond boiling when combined with the heat increase caused by the blowtorch — in that case that the plan will be invalid. The planner can choose a value for duration that avoids this violation. In part (b) the plan will be determined invalid regardless of the duration of the action because of the no-moving-targets rule. Notice that this model does not attempt to model the consequences of continued heating of the pan after the boiling point, so plans with actions that cause this to occur are simply invalid. However, PDDL2.1 *can be used to model more of the physical situation, so that the consequences are explicit and the planner can choose to exploit them or avoid them accordingly.*

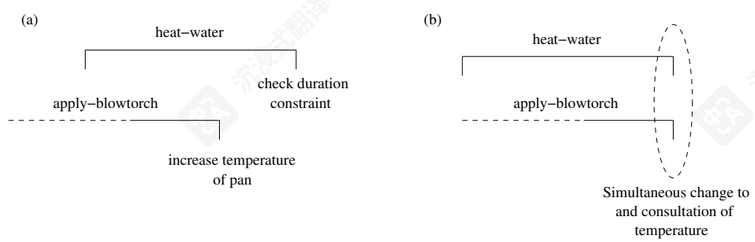

Figure 13: Heating a pan with a discrete durative action, concurrently with another heating activity.

示例 2 如果一个计划试图进一步加热锅（比如通过向锅上吹风），在加热水期间，只要并发动作在加热水动作结束之前结束，如果选择的持续时间使得整体温度升高超过沸腾，那么就会看到持续时间约束被违反。如果并发活动与加热水动作同时结束，那么会因为持续时间约束试图在并发动作试图更新它的同一时间点访问温度而违反无移动目标规则。图 13 描绘了这两种情况。在这个图中，apply-blowtorch 是一个持续动作，它向一个对象（在这种情况下是锅）施加热量。在图的 (a) 部分，如果加热水动作的持续时间足够长，导致温度在结合吹风造成的温度升高后超过沸腾，那么持续时间约束将被违反——在这种情况下，计划将是无效的。规划器可以选择一个持续时间值来避免这种违反。在 (b) 部分，由于无移动目标规则，无论动作的持续时间如何，计划都将被确定为无效。请注意，该模型不试图模拟锅在沸腾点后继续加热的后果，因此导致这种情况发生的计划简单地无效。然而，PDDL2.1 可以用来模拟更多的物理情况，以便后果是明确的，规划器可以相应地利用它们或避免它们。

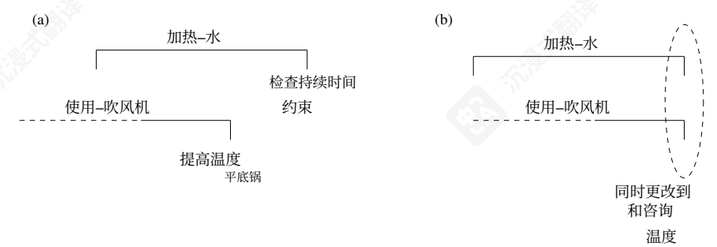

图13：使用离散持续动作加热锅，同时进行另一项加热活动。

```
(:durative-action heat-water
   :parameters (?p - pan)
   :duration ()
   :condition (and (at start (full ?p))
                   (at start (onHeatSource ?p))
                   (at start (byPan))
                   (over all (full ?p))
                   (over all (onHeatSource ?p))
                   (over all (heating ?p))
                   (over all (<= (temperature ?p) 100))
                   (at end (byPan)))
   :effect (and    (at start (heating ?p))
                   (at end (not (heating ?p)))
                   (increase (temperature ?p) (* #t (heat-rate))))
)
```

Figure 14: A continuous durative action for heating a pan of water.

The use of duration inequalities adds significant expressive power even when using discrete durative actions. For example, the plan depicted in part (a) of Figure 13, which illustrates the use of the water-heating action shown in Figure 12 while concurrently heating the pan with a blowtorch, will be considered valid provided that there is a duration value that satisfies the duration constraint in the water-heating action. This brings us very close to the expressive power available with continuous durative actions because it gives the planner the power to exploit concurrent interacting activities enacting changes on the same numeric valued variable (see Example 3). Attempting to express continuous change using only duration inequalities does not give precisely equivalent behaviour, because the discretisation forces actions that access changing numeric values to be separated, by some small temporal interval, from the actions that change those values in order to resolve their mutual exclusion. In a continuous model this is not necessary because the true value of a numeric variable is available for consultation at any time during the continuous process of change.

In the discrete semantics presented in Section 8 we exploit the fact that the only changes that can occur when a plan is executed are at points corresponding to the times of happenings, so the plan can be checked by looking at the activity focussed in this finite happening sequence. In fact, provided continuous effects are restricted to linear functions of time with only first order effects (which requires that no continuous effects can affect numeric expressions contributing to the rate of change of another numeric valued variable), and invariants are restricted to linear functions of changing quantities, it is still possible to restrict attention to the happening sequence even when using continuous actions.

Non-linear effects and higher-order rates of change create difficulties since it is possible for an invariant to be satisfied at the end points of an interval, without having necessarily been satisfied throughout the interval. In these cases it is no longer sufficient to insert invariant checking actions at fixed mid-points in the happening sequence of a plan in order to validate its behaviour. However, provided that effects are first-order and linear, and invariants are linear in continuously changing values, then, despite the fact that arbitrary


```
(:durative-action heat-water
   :parameters (?p - pan)
   :duration ()
   :condition (and (at start (full ?p))
                   (at start (onHeatSource ?p))
                   (at start (byPan))
                   (over all (full ?p))
                   (over all (onHeatSource ?p))
                   (over all (heating ?p))
                   (over all (<= (temperature ?p) 100))
                   (at end (byPan)))
   :effect (and    (at start (heating ?p))
                   (at end (not (heating ?p)))
                   (increase (temperature ?p) (* #t (heat-rate))))
)
```


图14：对一锅水进行加热的持续时态动作。

使用时态不等式即使在使用离散时态动作时也能增加显著的表意能力。例如，图13(a)部分所示计划，该计划说明了在同时用吹风机加热锅的情况下使用图12中所示的水加热动作，只要存在一个满足水加热动作中时态约束的时态值，该计划将被视为有效。这使我们的规划器非常接近于连续时态动作所能提供的表意能力，因为它赋予了规划器利用并发交互活动对同一数值变量执行更改的能力（参见示例3）。仅使用时态不等式来表达连续变化不会给出精确等效的行为，因为离散化迫使访问变化数值的动作与改变这些数值的动作之间必须存在一个小的时态间隔，以便解决它们的相互排斥。在连续模型中这不是必要的，因为在连续变化过程中，任何时间点都可以获取数值变量的真实值。

在第八节中介绍的离散语义中，我们利用了这样一个事实：当一个计划被执行时，可能发生的变化仅发生在对应事件时间的点上，因此可以通过观察这个有限的事件序列中聚焦的活动来检查计划。事实上，只要将连续效应限制为仅具有一阶效应的时间的线性函数（这要求没有连续效应可以影响对其他数值变量变化率有贡献的数值表达式），并将不变量限制为对变化量的线性函数，即使使用连续动作，仍然可以将注意力限制在事件序列上。

非线性效应和高阶变化率会导致困难，因为一个不变量可能在区间的端点得到满足，而未必在整个区间内都满足。在这种情况下，在计划的事件序列的固定中点插入不变量检查操作已不足以验证其行为。然而，只要效应是一阶且线性的，并且不变量在连续变化的值中是线性的，那么，尽管任意

Example 3 *It is possible with discrete durative actions, with duration inequalities, to model the effects of adding an egg to the heating water when the water is at, say, 90 degrees. We do this by applying two heat-water actions, around an add-egg action, in such a way that the overall duration of the two heat-water actions is exactly the duration required to boil the water from its original temperature. However, the way the heat-water action is currently modelled means that the heat will be turned off before the egg is added, and then turned on again to complete the heating, since the temperature is only updated when the durative action terminates. With continuous durative actions the egg can be added whilst the single heat-water action is in progress since the temperature of the pan is continuously updated. So, discrete durative actions with duration inequalities allow us to approximate continuous activity by appending a finite sequence of discrete intervals in an appropriate way. The no moving targets rule means that the end points of these intervals will be separated by non-zero, arbitrarily small, time gaps. This is not required when using continuous actions because, in contrast to the step-function effects of discrete actions, continuous effects are not localised at a single point.*

time points within action intervals are accessible to the planner, it is only necessary to gain access to numeric values at the start- and end-points of the actions in the plan that refer to them, together with finitely many mid-points for invariant-checking actions. The values are not required at all other points. This is so because continuous durative actions do not support the modelling of exogenous events, so it is not necessary to take into account the exogenous activity of the environment in determining the validity of a plan.

5.4 Related Approaches

Time is an important numerically varying quantity. The simplest way to reason about time is to adopt a *black box* durative action model in which change happens at the ends of their durative intervals. This is the approach taken in the language used by TGP (Smith & Weld, 1999), for example, in which durative actions encapsulate continuous change so that the correct values of any affected variables are guaranteed only at the end points of the implied intervals. All of the logical and numeric effects of a durative action are enacted at the end of the action and are undefined during the interval of its execution. All undeleted preconditions must remain true throughout the interval. There is no syntactic distinction between *pre*conditions and *invariant* conditions in this action representation. A simplistic way of ensuring correct action application is to prevent concurrent actions that refer to the same facts, but this excludes many intuitively valid plans.

A more sophisticated approach allows preconditions to be annotated with time points, or intervals, so that the requirement that a condition be true at some point, or over some interval, within the duration of the action can be expressed. This is the approach taken in Sapa (Do & Kambhampati, 2001). For example, using such an annotated precondition it would be possible to express the requirement that some chemical additive be added within two minutes of the start of a tank-filling action. If effects can also be specified to occur at arbitrary points within the duration of the action then it is possible to express effects that

示例 3 使用离散持续动作，以及持续时间不等式，可以在水处于，例如，90 摄氏度时，模拟向加热水中添加鸡蛋的效果。我们通过在添加鸡蛋动作周围应用两个加热水动作来实现这一点，并确保这两个加热水动作的整体持续时间正好是水从原始温度沸腾所需的持续时间。然而，目前对加热水动作的建模方式意味着在添加鸡蛋之前会关闭加热，然后在添加鸡蛋后再次开启以完成加热，因为温度仅在持续动作终止时更新。对于连续持续动作，可以在单个加热水动作进行过程中添加鸡蛋，因为锅的温度是持续更新的。因此，具有持续时间不等式的离散持续动作允许我们通过以适当的方式附加有限数量的离散区间来近似连续活动。不移动目标规则意味着这些区间的端点将被非零、任意小的时间间隙隔开。在使用连续动作时不需要这样做，因为与离散动作的阶跃函数效果不同，连续效果不会在单个点上局部化。

动作区间内的任意时间点是可被规划器访问的，只需获取计划中指向它们的动作的起止点的数值，以及有限多个用于不变量检查动作的中点即可，其他点的值则完全不需要。这是因为在持续性的持续动作中不支持对环境事件的建模，因此在确定计划的有效性时无需考虑环境的外部活动。

5.4 相关方法

时间是重要的数值变化量。思考时间最简单的方法是采用一个黑盒持续动作模型，其中变化发生在它们的持续区间端点。这是TGP（Smith & Weld, 1999）所使用的语言采用的方法，例如，在持续动作中封装连续变化，以便任何受影响变量的正确值仅保证在隐含区间的端点。持续动作的所有逻辑和数值效果都在动作结束时发生，并在其执行区间内未定义。所有未删除的前置条件必须在整个区间内保持为真。在这种动作表示中，前置条件和不变条件之间没有句法区别。确保正确动作应用的一种简单方法是防止引用相同事实的并发动作，但这排除了许多直观上有效的计划。

一种更复杂的方法允许将前置条件标注为时间点或时间间隔，以便能够表达在动作持续时间内某个条件在某个时间点或某个时间间隔内为真的要求。这就是 Sapa (Do & Kambhampati, 2001) 采用的方法。例如，使用这样的标注前置条件，可以表达在罐装动作开始后的两分钟内添加某种化学添加剂的要求。如果效果也可以指定在动作持续时间的任意点发生，那么就可以表达那些

occur before the end of the specified duration. It is also possible to distinguish between conditions that are local to specific points in the duration of the action and those that are invariant throughout the action.

Allowing reference to finitely many time points between the start and end of actions makes the language more complex without adding to its expressive power. Where time points are strictly scheduled relative to the start of the action the effect can be achieved through the use of a sequence of linked durative actions. We decided to keep PDDL2.1 simple by restricting access to only the end points of actions.

In TLPlan (Bacchus & Ady, 2001) a similar, but more constrained, approach is adopted in which actions are applied instantaneously but can have delayed effects. The delays for effects can be arbitrary and different for each effect. However, invariants cannot be specified because the preconditions are checked at the instant of application and subsequent delayed effects are separated from the action which initiated them.

Several planners have been developed to use networks of temporal constraints (Ghallab & Laruelle, 1994; Jonsson, Morris, Muscettola, & Rajan, 2000; El-Kholy & Richards, 1996) to handle temporal structure in planning problems. Efficient algorithms exist for handling such constraints (Dechter, Meiri, & Pearl, 1991) which make them practical for managing large networks. The domain models constructed using PDDL2.1 certainly lend themselves to treatment by similar techniques, but are not constrained to be handled in this way.

6. Introduction to the Semantics of PDDL2.1

In Sections 7 and 8 we provide a formal semantics for the numeric extension and temporal extension of PDDL2.1. Together these sections contain 20 definitions. The lengthy treatment is necessary because the semantics we have developed adds four significant extensions over classical planning and the semantics Lifschitz developed for STRIPS (Lifschitz, 1986). These are:

- the introduction of time, so that plans describe behaviour relative to a real time line;
- related to the first extension, the treatment of concurrency — actions can be executed in parallel, which can lead to plans that contain concurrent interacting processes (although these processes are encapsulated in durative actions in PDDL2.1);
- an extension to handle numeric-valued fluents;
- the use of conditional effects, both alone and in conjunction with all of the above extensions.

The semantics is built on a familiar state-transition model. The requirements of the semantics can be reduced to four essential elements.

1. To define what is a state. The introduction of both time and numeric values complicate the usual definition of a state as a set of atoms.
2. To define when a state satisfies a propositional formula representing a goal condition or precondition of an action. An extension of the usual interpretation of a state as a valuation in which an atom is true if and only if the atom is in the state (the Closed World Assumption) is required in order to handle the numeric values in the state.

在指定持续时间结束之前发生的效果。还可以区分在动作持续时间的特定点局部生效的条件和在整个动作期间保持不变的条件。

允许在动作的开始和结束之间引用有限多个时间点会使语言变得更加复杂，而没有增加其表达能力。当时间点相对于动作的开始严格调度时，可以通过使用一系列链接的持续动作来实现效果。我们决定通过仅限制对动作的终点访问来保持 PDDL2.1简单。

在TLPlan（Bacchus & Ady，2001）中，采用了类似但更受约束的方法，其中动作是即时应用的，但可以有延迟效果。效果的延迟可以是任意的，并且每个效果都不同。然而，无法指定不变式，因为前提条件是在应用时检查的，而后续的延迟效果与启动它们的动作是分开的。

已经开发了几个规划器，用于使用时间约束网络（Ghallab & Laruelle，1994；Jonsson，Morris，Muscettola，& Rajan，2000；El-Kholy & Richards，1996）来处理规划问题中的时间结构。存在用于处理此类约束的有效算法（Dechter，Meiri，& Pearl，1991），这使得它们在实际管理大型网络时具有实用性。使用 PDDL2.1 构建的领域模型当然适合用类似技术进行处理，但不受约束必须以这种方式处理。

6. PDDL2.1的语义介绍

在7节和8节中，我们为 PDDL2.1的数值扩展和时序扩展提供了形式化语义。这两节共包含20个定义。详细的阐述是必要的，因为我们所发展的语义在经典规划以及Lifschitz为 STRIPS (Lifschitz, 1986)开发的语义基础上增加了四个重要的扩展。这些扩展是：

- 引入时间，使得计划描述相对于真实时间线的行为；
- 与第一个扩展相关，即并发处理——动作可以并行执行，这可能导致包含并发交互进程的计划（尽管这些进程在 PDDL2.1中以持续动作的形式封装）；
- 扩展以处理数值型流动态；
- 使用条件效果，单独使用以及与上述所有扩展结合使用。

语义学建立在一种熟悉的状态转换模型之上。语义学的需求可以简化为四个基本要素。

1. 定义什么是状态。时间和数值的引入使状态通常被定义为原子集合的定义变得复杂。

2. 为了定义一个状态何时满足一个表示目标条件或动作的前置条件的命题公式。为了处理状态中的数值，需要在将状态解释为赋值（原子仅在原子位于状态中时为真，即闭世界假设）的通常解释的基础上进行扩展。

3. To define the state transition induced by application of an action. The update rule for the logical state must be supplemented with an explanation of the consequences for the numeric part of the state.

4. To define when two actions can be applied concurrently and how their concurrent application affects the application of those actions individually.

The structure of the definitions is as follows. Definitions 1 to 15, given in Section 7, define what it means for a plan to be valid when the plan consists of only non-durative actions. Definitions 1 to 6 set up the basic terminology, the foundational structures and the framework for handling conditional effects and primitive numeric expressions. Definition 2 meets the first requirement identified above, defining states. Definition 9 meets the second requirement, defining when a goal description is satisfied in a state. Definition 11 defines a simple plan, extending the classical notion of a sequence of actions by adding time. Definitions 12 meets the fourth requirement, by defining when two actions cannot be executed concurrently. Definition 13 meets the third requirement, defining what we mean by execution of actions, including concurrent execution of actions. Definitions 14 and 15 define the execution of a plan and what it means for a plan to be valid, given the basis laid in the previous definitions.

In Section 8 the semantics is extended to give meaning to durative actions. We begin with Definition 16, which defines ground durative actions analogously to Definition 6 for simple (that is, non-durative) actions. Similarly, Definition 17 parallels the definition of a simple plan (Definition 11) and Definitions 19 and 20 parallel those for the execution and validity of simple plans (Definitions 14 and 15). Definition 18 is the critical definition for the semantics of plans with durative actions, supplying a transformation of temporal plans into simple plans, whose validity according to the semantics of purely simple plans, can be used to determine the validity of the original temporally structured plans.

7. The Semantics of Simple Plans

The semantics we define in this section extends the essential core of Lifschitz' STRIPS semantics (1986) to handle temporally situated actions, possibly occurring simultaneously, with numeric and conditional effects.

Definition 1 Simple Planning Instance *A* simple planning instance *is defined to be a pair*

$$I = (Dom, Prob)$$

where $Dom = (Fs, Rs, As, arity)$ *is a 4-tuple consisting of (finite sets of)* function symbols, relation symbols, actions *(non-durative), and a function arity mapping all of these symbols to their respective arities.* $Prob = (Os, Init, G)$ *is a triple consisting of the objects in the domain, the initial state specification and the goal state specification.*

The primitive numeric expressions *of a planning instance,* $PNEs$*, are the terms constructed from the function symbols of the domain applied to (an appropriate number of) objects drawn from* Os*. The* dimension *of the planning instance,* dim*, is the number of distinct primitive numeric expressions that can be constructed in the instance.*

3. 为了定义应用一个动作所引起的状态转换。逻辑状态的更新规则必须补充对状态数值部分后果的解释。

4. 要定义何时两个动作可以同时应用以及它们的同时应用如何影响这些动作的单独应用。

定义的结构如下。第7节中给出的定义1到15定义了当计划仅由非持续性动作组成时，什么意味着计划有效。定义1到6建立了基本术语、基础结构和处理条件效果和原始数值表达式的框架。定义2满足了上述第一个要求，定义了状态。定义9满足了第二个要求，定义了在状态中何时满足目标描述。定义11定义了一个简单计划，通过添加时间扩展了动作序列的经典概念。定义12通过定义何时两个动作不能同时执行满足了第四个要求。定义13满足了第三个要求，定义了我们所说的动作执行，包括动作的同时执行。定义14和15定义了计划的执行以及给定先前定义的基础时，计划有效的含义。

在8节中，语义被扩展以赋予持续性动作意义。我们从定义16开始，它类似于定义6为简单（即非持续性）动作定义了基础持续性动作。类似地，定义17平行于简单计划（定义11）的定义，定义19和20平行于简单计划的执行和有效性（定义14和15）。定义18是具有持续性动作的计划语义的关键定义，它提供了一种将时间计划转换为简单计划的方法，根据纯粹简单计划的语义，其有效性可用于确定原始时间结构化计划的有效性。

7. 简单计划的语义

本节中我们定义的语义扩展了Lifschitz' STRIPS se- mantics (1986) 的核心，以处理时间定位的动作，这些动作可能同时发生，具有数字和条件效果。

定义1 简单规划实例 简单规划实例定义为一个对

$$I = (Dom, Prob)$$

where $Dom = (Fs, Rs, As, arity)$ 是一个由（有限集合的）函数符号、关系符号、动作（非持续性）以及一个将所有这些符号映射到它们各自阶数的函数 $arity$ 组成的4元组。$Prob = (Os, Init, G)$是一个由域中的对象、初始状态规范和目标状态规范组成的3元组。

规划实例的原始数值表达式， $PNEs$，是由领域中的函数符号应用于（适当数量的）来自 Os的对象构成的项。规划实例的维度， dim，是在实例中可以构造的不同原始数值表达式的数量。

The atoms *of the planning instance, Atms, are the (finitely many) expressions formed by applying the relation symbols in Rs to the objects in Os (respecting arities).*

Init consists of two parts: $Init_{logical}$ is a set of literals formed from the atoms in Atms. $Init_{numeric}$ is a set of propositions asserting the initial values of a subset of the primitive numeric expressions of the domain. These assertions each assign to a single primitive numeric expression a constant real value. The goal condition is a proposition that can include both atoms formed from the relation symbols and objects of the planning instance and numeric propositions between primitive numeric expressions and numbers.

As is a collection of action schemas (non-durative actions) each expressed in the syntax of PDDL*. The primitive numeric expression schemas and atom schemas used in these action schemas are formed from the function symbols and relation symbols (used with appropriate arities) defined in the domain applied to objects in Os and the schema variables.*

The semantics shows how instantiated action schemas can be interpreted as state transitions, in a similar way to the familiar state transition semantics defined by Lifschitz. An important difference is that states can no longer be seen as simply sets of propositions, but must also account for the numeric expressions appearing in the planning instance and the time at which the state holds. This is achieved by extending the notion of state.

Definition 2 Logical States and States *Given the finite collection of atoms for a planning instance I, $Atms_I$, a* logical state *is a subset of $Atms_I$. For a planning instance with dimension dim, a* state *is a tuple in $(\mathbb{R}, \mathbb{P}(Atms_I), \mathbb{R}_\perp^{dim})$ where $\mathbb{R}_\perp = \mathbb{R} \cup \{\perp\}$ and $\perp$ denotes the undefined value. The first value is the* time *of the state, the second is the logical state and the third value is the vector of the dim values of the dim primitive numeric expressions in the planning instance.*

The initial state for a planning instance is $(0, Init_{logical}, \mathbf{x})$ where $\mathbf{x}$ is the vector of values in $\mathbb{R}_\perp$ corresponding to the initial assignments given by $Init_{numeric}$ (treating unspecified values as $\perp$).

Undefined values are included in the numeric ranges because there are domains in which some terms start undefined but can nevertheless be initialised and exploited by actions.

To interpret actions as state transition functions it is necessary to achieve two steps. Firstly, since (in PDDL2.1) plans are only ever constructed from fully instantiated action schemas, the process by which instantiation affects the constructs of an action schema must be defined and, secondly, the machinery that links primitive numeric expressions to elements of the vector of real values in a state and that allows interpretation of the numeric updating behaviours in action effects must be defined. Since the mechanisms that support the second of these steps also affect the process in the first, the treatment of numeric effects is described first.

Definition 3 Assignment Proposition *The syntactic form of a numeric effect consists of an assignment operator (*`assign`*,* `increase`*,* `decrease`*,* `scale-up` *or* `scale-down`*), one primitive numeric expression, referred to as the* lvalue*, and a numeric expression (which is an arithmetic expression whose terms are numbers and primitive numeric expressions), referred to as the* rvalue*.*

规划实例的原子， $Atms$，是由在 Rs 中应用于 Os 中的对象（尊重元数）形成的（有限个）表达式。

$Init$ 由两部分组成： $Init_{logical}$ 是由 $Atms$中的原子形成的字面量集合。$Init_{numeric}$ 是断言领域一部分原始数值表达式的初始值的命题集合。这些断言分别将一个常量实数值赋给单个原始数值表达式。目标条件是一个可以包含从关系符号形成的原子和规划实例的对象以及原始数值表达式与数字之间的数值命题的命题。

As是使用 PDDL语法表达的行动模式（非持续性行动）的集合。这些行动模式中使用的原始数值表达式模式和原子模式是由应用于 Os中的对象和模式变量定义的函数符号和关系符号（使用适当的元数）形成的。

语义学展示了如何将实例化的行动模式解释为状态转换，类似于Lifschitz定义的熟悉的状态转换语义。一个重要区别是状态不再可以简单地被视为命题的集合，而必须考虑规划实例中出现的数值表达式以及状态保持的时间。这是通过扩展状态的概念来实现的。

定义 2 逻辑状态和状态 给定规划实例的有限原子集合 I，$Atms_I$，逻辑状态是 $Atms_I$ 的一个子集。对于具有维度 dim 的规划实例，状态是一个元组 $(\mathbb{R}, \mathbb{P}(Atms_I)\, \mathbb{R}_\perp^{dim})$，其中 $\mathbb{R}_\perp = \mathbb{R} \cup \{\perp\}$ 和 $\perp$ 表示未定义值。第一个值是状态的时间，第二个是逻辑状态，第三个值是规划实例中 dim 原始数值表达式的 dim 值的向量。

规划实例的初始状态是 $(0, Init_{logical}, \mathrm{x})$，其中 x 是 $\mathbb{R}_\perp$ 中对应于 $Init_{numeric}$ 给出的初始分配的值的向量（将未指定值视为 $\perp$）。

未定义值包含在数值范围内，因为在某些域中，某些项初始时未定义，但仍然可以被动作初始化和利用。

为了将动作解释为状态转换函数，需要完成两个步骤。首先，由于（在 PDDL2.1 中）计划仅从完全实例化的动作模式构建，必须定义实例化如何影响动作模式结构的进程，其次，必须定义将原始数值表达式链接到状态中实数值向量的元素，并允许解释动作效果中的数值更新行为的机制。由于支持第二个步骤的机制也影响第一个步骤的过程，因此首先描述数值效果的处理。

定义3 赋值命题 数字效果的表达式形式包括一个赋值运算符（赋值、增加、减少、扩大或缩小），一个原始数字表达式，称为左值，以及一个数字表达式（这是一个项为数字和原始数字表达式的算术表达式），称为右值。

The assignment proposition *corresponding to a numeric effect is formed by replacing the assignment operator with its equivalent arithmetic operation (that is* `(increase p q)` *becomes* `(= p (+ p q))` *and so on) and then annotating the lvalue with a "prime".*

A numeric effect in which the assignment operator is either `increase` *or* `decrease` *is called an* additive assignment effect*, one in which the operator is either* `scale-up` *or* `scale-down` *is called a* scaling assignment effect *and all others are called* simple assignment effects*.*

A numeric effect defines a function of the numeric values in the state to which an action is applied determining the value of a primitive numeric expression in the resulting state. For the convenience of a uniform treatment of numeric expressions appearing in pre- and post-conditions, we transform the functions into propositions that assert the equality of the post-condition value and the expression that is intended to define it. That is, rather than writing an effect `(increase p q)` as a function $f(p) = p+q$, we write it as the proposition $(= p' (+ p\ q))$. The "priming" distinguishes the postcondition value of a primitive numeric expression from its precondition value (a convention commonly adopted in describing state transition effects on numeric values). The binding of the primitive numeric expressions to their values in states is defined in the following definition.

Definition 4 Normalisation *Let I be a planning instance of dimension dim_I and let*

$$index_I : PNEs_I \rightarrow \{1, \ldots, dim\}$$

be an (instance-dependent) correspondence between the primitive numeric expressions and integer indices into the elements of a vector of dim_I real values, $\mathbb{R}_\perp^{dim_I}$.

The normalised form of a ground proposition, p, in I is defined to be the result of substituting for each primitive numeric expression f in p, the literal $X_{index_I(f)}$. The normalised form of p will be referred to as $\mathcal{N}(p)$. Numeric effects are normalised by first converting them into assignment propositions. Primed primitive numeric expressions are replaced with their corresponding primed literals. **X** *is used to represent the vector $\langle X_1 \ldots X_n \rangle$.*

In Definition 4, the replacement of primitive numeric expressions with indexed literals allows convenient and consistent substitution of the vector of actual parameters for the vector of literals **X** appearing in a state.

With the machinery supporting treatment of numeric expressions complete, it is now possible to consider the process of instantiating action schemas. This process is managed in two steps. The first step is to remove constructs that we treat as syntactic sugar in the definition of a domain. These are conditional effects and quantified formulae. We handle both of these by direct syntactic transformations of each action schema into a set of action schemas considered to be equivalent. The transformation is similar to that described by Gazen and Knoblock (1997). Although it would be possible to give a semantic interpretation of the application of conditional effects directly, the transformation allows us to significantly simplify the question of what actions can be performed concurrently.

Definition 5 Flattening Actions *Given a planning instance, I, containing an action schema $A \in As_I$, the set of action schemas $flatten(A)$, is defined to be the set S, initially containing A and constructed as follows:*

与数字效果对应的赋值命题是通过用其等效的算术运算替换赋值运算符（即（增加 p q）变为 (= p (+ p q)) 等等）然后标注左值带有“撇号”。

一种赋值运算符为增加或减少的数值效果称为加法赋值效果，一种运算符为放大或缩小称为缩放赋值效果，其他所有效果称为简单赋值效果。

数值效果定义了应用于状态的动作所定义的数值函数，以确定结果状态中原始数值表达式的值。为了方便对前置条件和后置条件中出现的数值表达式进行统一处理，我们将函数转换为断言后置条件值与预期定义它的表达式的等式的命题。也就是说，我们不是将效果（增加 p q）作为函数 $f(p) = p+q$ 写出，而是将其作为命题 $(= p' (+ p\ q))$ 写出。“priming”区分了原始数值表达式的后置条件值与其前置条件值（在描述数值状态转换效果时通常采用的约定）。原始数值表达式与其在状态中的值之间的绑定在以下定义中定义。

定义 4 规范化 令 I 为维度 dim_I 的一个规划实例，并令

$$index_I : PNEs_I \rightarrow \{1, \ldots, dim\}$$

是原始数值表达式和向量 dim_I 实际值 $\mathbb{R}_\perp^{dim_I}$ 的元素索引之间的（实例依赖的）对应关系。

在 I 中，一个接地命题 p 的规范形式被定义为对 p 中的每个原始数值表达式 f 替换为文字 $X_{index_I(f)}$ 的结果。 p 的规范形式将被称为 $\mathcal{N}(p)$。数值效果通过首先将它们转换为赋值命题来进行规范化。带撇号的原始数值表达式被替换为它们对应的带撇号的文字。X 用于表示向量 $\langle X_1 \ldots X_n \rangle$。

在定义 4 中，用索引文字替换原始数值表达式允许方便且一致地替换状态中出现的文字向量 X 的实际参数向量。

随着支持数值表达式处理的机制完成，现在可以考虑实例化动作模式的过程。该过程分为两步管理。第一步是移除我们在领域定义中视为语法糖的结构。这些是条件效果和量化公式。我们通过将每个动作模式直接转换为被认为等效的一组动作模式来处理这两种情况。该转换类似于 Gazen 和 Knoblock (1997) 描述的转换。尽管可以直接对条件效果的适用性给出语义解释，但转换使我们能够显著简化可以同时执行哪些动作的问题。

定义 5 平坦化操作 给定一个规划实例， I，包含一个动作模式 $A \in As_I$，动作模式集 $flatten(A)$ 被定义为集合 S，初始包含 A 并按如下方式构建：

- *While S contains an action schema, X, with a conditional effect,* `(when P Q)`*, create two new schemas which are copies of X, but without this conditional effect, and conjoin the condition* `P` *to the precondition of one copy and* `Q` *to the effects of that copy, and conjoin* `(not P)` *to the precondition of the other copy. Add the modified copies to S.*
- *While S contains an action schema, X, with a formula containing a quantifier, replace X with a version in which the quantified formula* `( Q ( var1 ... vark ) P)` *in X is replaced with the conjunction (if the quantifier, Q, is* `forall`*) or disjunction (if Q is* `exists`*) of the propositions formed by substituting objects in I for each variable in $var_1 \ldots var_k$ in* `P` *in all possible ways.*

These steps are repeated until neither step is applicable.

Once flattened, actions can be grounded by the usual substitution of objects for parameters:

Definition 6 Ground Action *Given a planning instance, I, containing an action schema $A \in As_I$, the set of* ground actions for *A, GA_A, is defined to be the set of all the structures, a, formed by substituting objects for each of the schema variables in each schema, X, in $flatten(A)$ where the components of a are:*

- Name *is the name from the action schema, X, together with the values substituted for the parameters of X in forming a.*
- Pre_a*, the* precondition *of a, is the propositional precondition of a. The set of ground atoms that appear in* Pre_a *is referred to as* GPre_a*.*
- Add_a*, the* positive postcondition *of a, is the set of ground atoms that are asserted as positive literals in the effect of a.*
- Del_a*, the* negative postcondition *of a,is the set of ground atoms that are asserted as negative literals in the effect of a.*
- NP_a*, the* numeric postcondition *of a, is the set of all assignment propositions corresponding to the numeric effects of a.*

The following sets of primitive numeric expressions are defined for each ground action, $a \in GA_A$:

- $L_a = \{f | f$ *appears as an lvalue in* $a\}$
- $R_a = \{f | f$ *is a primitive numeric expression in an rvalue in a or appears in* $Pre_a\}$
- $L_a^* = \{f | f$ *appears as an lvalue in an additive assignment effect in* $a\}$

Some comment is appropriate on the last definition: an action precondition might be considered to have two parts — its logical part and its numeric expression-dependent part. Unfortunately, these can be interdependent. For example:

```
(or (clear ?x) (>= (room-in ?y) (space-for ?z)))
```


- 当 S 包含一个具有条件效果的动作模式，(当 P Q)，创建两个新的模式，它们是 X 的副本，但不包含这个条件效果，并将条件 P 连接到其中一个副本的前置条件，将 Q 连接到该副本的效果，并将 (不 P) 连接到另一个副本的前置条件。将修改后的副本添加到 S。
- 虽然 S 包含一个动作模式， X，其中包含一个量化公式，将X替换为在该量化公式（ Q($var_1 \ldots var_k$) P）在 X中替换为合取（如果量化器， Q，是forall）或析取（如果 Q 是exists）的命题的版本，这些命题是通过以所有可能的方式将对象 I 代入P中的每个变量$var_1 \ldots var_k$形成的。

重复这些步骤，直到没有步骤适用。

展平后，动作可以通过通常的对象替换参数来接地：

定义6 接地动作 给定一个规划实例， I，包含一个动作模式$A \in As_I$， A， GA_A的接地动作集被定义为所有结构，a，这些结构是通过在每个模式， X，在$flatten$（A）中替换每个模式变量中的对象形成的，其中 a 的组件是：

- 名称是来自动作模式的名字， X，与用于形成 a的 X 参数的替换值一起。
- 预$_a$， a的前置条件，是 a的命题前置条件。在Pre$_a$ 中出现的地面原子集合称为GPre$_a$。
- 添加$_a$， a的正后置条件，是在 a的效果中作为正文字断言的地面原子集合。
- 删除$_a$， a的负后置条件，是在 a的效果中作为负文字断言的地面原子集合。
- NP$_a$， a的数值后置条件，是所有对应于 a数值效果的赋值命题的集合。

为每个地面动作,$a \in GA_A$定义了以下原始数值表达式集合：

- $L_a = \{f | f$ 作为左值出现在 $a\}$ 中
- $R_a = \{f | f$是 a 中作为右值的原始数值表达式，或出现在 $Pre_a\}$ 中
- $L_a^* = \{f | f$ 作为左值出现在 $a\}$ 中的加性赋值效果中

在最后一个定义上适当添加一些注释：一个动作前提条件可能被认为包含两个部分——其逻辑部分和其数值表达式相关的部分。不幸的是，这些部分可能是相互依赖的。例如：

```
（或（清除? x）（>= （房间内? y）（空间为? z））)
```

might be a precondition of an action. In order to handle such conditions, we need to check whether they are satisfied given not only the current logical state, but also the current values of the domain numeric expressions. The inclusion of a numeric component in the state makes it necessary to ensure the correct substitution of the numeric values for the expressions used in the action precondition. This is achieved using the normalisation process from Definition 4 in Definition 9. In contrast, the postcondition of an action cannot contain interlocked numeric and logical effects, so it *is* possible to separate the effects into the distinct numeric and logical components.

Definition 7 Valid Ground Action *Let a be a ground action. a is* valid *if no primitive numeric expression appears as an lvalue in more than one simple assignment effect, or in more than one different type of assignment effect.*

Definition 7 ensures that an action does not attempt inconsistent updates on a numeric value. Unlike logical effects of an action which cannot conflict, it is possible to write a syntactic definition of an action in which the effects are inconsistent, for example by assigning two different values to the same primitive numeric expression.

Definition 8 Updating Function *Let a be a valid ground action. The* updating function *for a is the composition of the set of functions:*

$$\{\mathrm{NPF}_p : \mathbb{R}_\perp^{dim} \rightarrow \mathbb{R}_\perp^{dim} \,|\, p \in NP_a\}$$

such that $\mathrm{NPF}_p(\mathbf{x}) = \mathbf{x}'$ where for each primitive numeric expression x_i' that does not appear as an lvalue in $\mathcal{N}(p)$, $x_i' = x_i$ and $\mathcal{N}(p)[\mathbf{X}' := \mathbf{x}', \mathbf{X} := \mathbf{x}]$ is satisfied.

The notation $\mathcal{N}(p)[\mathbf{X}' := \mathbf{x}', \mathbf{X} := \mathbf{x}]$ should be read as the result of normalising p and then substituting the vector of actual values $\mathbf{x}'$ for the parameters $\mathbf{X}'$ and actual values $\mathbf{x}$ for formal parameters $\mathbf{X}$.

Definition 8 defines the function describing the update effects of an action. The function ensures that all of the reals in the vector describing the numeric state remain unchanged if they are not affected by the action (this is the numeric-state equivalent of the persistence achieved for propositions by the STRIPS assumption). For other values in the vector, the normalisation process is used to substitute the correctly indexed vector elements for the primitive numeric expressions appearing as lvalues (which are the primed vector elements corresponding to values in the post-action state) and rvalues (the unprimed values appearing in the pre-action state). The tests that must be satisfied in order to ensure correct behaviour of the functions in the composition simply confirm that the arithmetic on the rvalues is correctly applied to arrive at the lvalues. The requirement that the action be valid ensures that the composition of the functions in Definition 8 is well-defined, since all of the functions in the set commute, so the composition can be carried out in any order.

The various sets of primitive numeric expressions defined in the Definition 6 allow us to conveniently express the conditions under which two concurrent actions might interfere with one another. In particular, we are concerned not to allow concurrent assignment to the same primitive numeric expression, or concurrent assignment and inspection. We *do* allow concurrent increase or decrease of a primitive numeric expression. To allow this we will

可能是一个动作的前提条件。为了处理这些条件，我们需要检查它们是否在当前逻辑状态以及当前领域数值表达式的值下得到满足。状态中包含数值组件使得必须确保正确地替换动作前提条件中使用的表达式的数值。这是通过使用定义9中的定义4的规范化过程实现的。相比之下，动作的后置条件不能包含相互锁定的数值和逻辑效果，因此可以将效果分离为不同的数值和逻辑组件。

定义 7 有效地面动作 令 a 为一个地面动作。 a 是有效的，如果没有任何原始数值表达式作为多个简单赋值效果中的 lvalue，或者出现在多种不同类型的赋值效果中。

定义 7 确保一个动作不会尝试对数值进行不一致的更新。与动作的逻辑效果不能冲突不同，可以编写一个动作的句法定义，其中效果不一致，例如通过将两个不同的值赋给同一个原始数值表达式。

定义 8 更新函数 令 a 为一个有效地面动作。 a 的更新函数是以下函数集的组合：

$$\{\mathrm{NPF}_p : \mathbb{R}_\perp^{dim} \rightarrow \mathbb{R}_\perp^{dim} \,|\, p \in NP_a\}$$

使得 $\mathrm{NPF}_p(\mathbf{x}) = \mathbf{x}'$ 其中对于每个不作为 $\mathcal{N}(p)$、 $x_i' = x_i$ 和 $\mathcal{N}(p)[\mathbf{X}' := \mathbf{x}', \mathbf{X} := \mathbf{x}]$ 中的 lvalue 出现的原始数值表达式 x_i'， $x_i' = x_i$ 和 $\mathcal{N}(p)[\mathbf{X}' := \mathbf{x}', \mathbf{X} := \mathbf{x}]$ 满足。

符号 $\mathcal{N}(p)[\mathbf{X}' := \mathbf{x}', \mathbf{X} := \mathbf{x}]$ 应解释为对 p 进行规范化后的结果，然后用实际值向量 x' 替换参数 X' 和实际值 x。

定义8定义了描述动作更新效果的函数。该函数确保，如果向量描述的数值状态中的所有实数未受动作影响，则保持不变（这是对命题通过 STRIPS 假设实现的持久性在数值状态中的等效）。对于向量中的其他值，使用归一化过程来用正确索引的向量元素替换作为lvalue出现的原始数值表达式（这些是与后动作状态中的值对应的带撇号的向量元素）和rvalue（出现在前动作状态中的未带撇号的值）。为了确保组合中函数的正确行为，必须满足的测试仅确认对rvalue的算术运算正确地应用于得到lvalue。要求动作有效确保定义8中函数的组合是良定义的，因为集合中的所有函数都交换，因此组合可以在任何顺序下进行。

定义6中定义的各种原始数值表达式使我们能够方便地表达两个并发操作可能相互干扰的条件。特别是，我们不允许对同一个原始数值表达式进行并发赋值，或并发赋值和检查。我们允许对原始数值表达式的并发增加或减少。为了允许这一点，我们将

have to apply collections of concurrent updating functions to the primitive numeric expressions. This can be allowed provided that the functions commute. Additive assignments do commute, but other updating operations cannot be guaranteed to do so, except if they do not affect the same primitive numeric expressions or rely on primitive numeric expressions that are affected by other concurrent assignment propositions. It would be possible to make a similar exception for scaling effects, but additive assignment effects have a particularly important role in durative actions that is not shared by scaling effects, so for simplicity we allow concurrent updates only with these effects. We use the three sets of primitive numeric expressions to determine whether we are in a safe situation or not. Within a single action it is possible for the rvalues and lvalues to intersect. That is, an action can update primitive numeric expressions using current values of primitive numeric expressions that are also updated by the same action. All rvalues will have the values they take in the state prior to execution and all lvalues will supply the new values for the state that follows.

Definition 9 Satisfaction of Propositions *Given a logical state, s, a ground propositional formula of* PDDL2.1*, p, defines a predicate on $\mathbb{R}_{\perp}^{dim}$, $Num(s,p)$, as follows:*

$$Num(s,p)(\mathbf{x}) \quad iff \quad s \models \mathcal{N}(p)[\mathbf{X} := \mathbf{x}]$$

where $s \models q$ means that q is true under the interpretation in which each atom, a, that is not a numeric comparison, is assigned true iff $a \in s$, each numeric comparison is interpreted using standard equality and ordering for reals and logical connectives are given their usual interpretations. p is satisfied *in a state $(t, s, \mathbf{X})$ if* $\mathrm{Num}(s,p)(\mathbf{X})$.

Comparisons involving $\perp$, including direct equality between two $\perp$ values are all undefined, so that enclosing propositions are also undefined and not satisfied in any state.

Definition 10 Applicability of an Action *Let a be a ground action. a is* applicable *in a state s if the Pre_a is satisfied in s.*

7.1 Semantics of a Simple Plan

A simple plan, in PDDL2.1, is a sequence of timed actions, where a timed action has the following syntactic form:

$$t : (action\, p_1 \ldots p_n)$$

In this notation t is a positive rational number in floating point syntax and the expression (action $p_1 \ldots p_n$) is the name and actual parameters of the action to be executed at that point in time. In more complex plans simple and durative actions, with or without numeric-valued effects or preconditions, can co-occur — the semantics of such plans is discussed in Section 8. No special separators are required to separate timed actions in the sequence and the actions are not required to be presented in time-sorted order. It is possible for multiple actions to be given the same time stamp, indicating that they should be executed concurrently. It should be emphasised that the earliest point at which activity occurs within a plan must be strictly after time 0. This constraint follows from the decision to make the initial state be the state existing at time 0, together with the decision, in the semantics, that actions have their effects in the interval that is *closed on the left*, starting at the time when the action is applied, while preconditions are tested in the interval that is *open on the right* that precedes the action.

不得不将一组并发更新函数应用于原始数值表达式。只要函数可交换，这就可以允许。加法赋值是可交换的，但其他更新操作不能保证可交换，除非它们不影响的原始数值表达式相同，或依赖于受其他并发赋值命题影响的原始数值表达式。对于缩放效应，也可以做出类似的例外，但加法赋值效应在持续动作中具有特别重要的作用，而缩放效应并不共享这一作用，因此为了简化，我们仅允许使用这些效应进行并发更新。我们使用三组原始数值表达式来确定我们是否处于安全状态。在一个单一动作中，rvalues和lvalues可能相交。也就是说，一个动作可以使用当前值更新原始数值表达式，而这些原始数值表达式也由同一个动作更新。所有rvalues将具有执行前的状态中的值，而所有lvalues将为后续状态提供新值。

定义 9 满足命题 给定一个逻辑状态 s，一个关于 PDDL2.1 的 p 基本命题公式定义了 $\mathbb{R}_{\perp}^{dim}$ 上的谓词 $\mathrm{Num}(s,p)$，如下：

$$Num(s,p)(\mathbf{x}) \quad iff \quad s \models \mathcal{N}(p)[\mathbf{X} := \mathbf{x}]$$

其中 $s \models q$ 表示在解释中，每个不是数值比较的原子 a，当 $a \in s$ 时被赋予 true；每个数值比较使用实数的标准相等性和排序进行解释；逻辑连接符被赋予它们通常的解释。 p 在状态 (t, s, X) 中被满足，如果 $\mathrm{Num}(s,p)(\mathrm{X})$。

涉及 $\perp$ 的比较，包括两个 $\perp$ 值之间的直接相等性都是未定义的，因此包含的命题也是未定义的，并且在任何状态下都不被满足。

定义 10 动作的可应用性 设 a 是一个基本动作。 a 在状态 s 中可应用，如果 Pre_a 在 s 中被满足。

7.1 简单计划的语义

一个简单的计划，在 PDDL2.1中，是一个定时动作的序列，其中定时动作具有以下句法形式：

$$t : (action\, p_1 \ldots p_n)$$

在这种表示法中 t 是一个浮点语法中的正有理数，表达式（动作 $p_1 \ldots p_n$）是该时间点要执行的动作用名和实际参数。在更复杂的计划中，简单和持续的动作可以共存，无论它们是否有数值效果的先决条件——这种计划的语义在第8节中讨论。在序列中分隔定时动作不需要特殊的分隔符，动作也不需要按时间排序呈现。多个动作可以给相同的时钟标记，表示它们应该同时执行。应该强调的是，计划中活动发生的最早时间点必须在时间0之后。这个约束来自将初始状态设为时间0时存在的状态的决策，以及语义中关于动作在应用时开始、左闭右开的区间内产生效果的决策，而先决条件在动作之前的右开区间内被测试。

In order to retain compatibility with the output of current planners the following concession is made: if the plan is presented as a sequence of actions with no time points, then it is inferred that the first action is applied at time 1 and the succeeding actions apply in sequence at integral time points one unit apart.

A simple plan is a slight generalisation of the more familiar STRIPS-style classical plan, since actions are labelled with the time at which they are to be executed.

Definition 11 Simple Plan *A* simple plan*, SP, for a planning instance, I, consists of a finite collection of* timed simple actions *which are pairs (t, a), where t is a rational-valued time and a is an action name.*

The happening sequence*, $\{t_i\}_{i=0\ldots k}$ for SP is the ordered sequence of times in the set of times appearing in the timed simple actions in SP. All t_i must be greater than 0. It is possible for the sequence to be empty (an empty plan).*

The happening *at time t, E_t, where t is in the happening sequence of SP, is the set of (simple) action names that appear in timed simple actions associated with the time t in SP.*

A plan thus consists of a sequence of happenings, each being a set of action names applied concurrently at a specific time, the sequence being ordered in time. The times at which these happenings occur forms the happening sequence. It should be noted that action names are ambiguous when action schemas contain conditional effects — the consequence of flattening is to have split these actions into multiple actions with identical names, differentiated by their preconditions. However, at most one of each set of actions with identical names can be applicable in a given logical state, since the precondition of each action in such a set will necessarily be inconsistent with the precondition of any other action in the set, due to the way in which the conditional effects are distributed between the pairs of action schemas they induce.

In order to handle concurrent actions we need to define the situations in which the effects of those actions are consistent with one another. This issue was first discussed in Section 5.1. The mutual exclusion rule for PDDL2.1 is an extension of the idea of *action mutex* conditions for GraphPlan (Blum & Furst, 1995). The extension handles two extra features: the extended expressive power of the language (to include arbitrary propositional connectives) and the introduction of numeric expressions. We make a very conservative condition for actions to be executed concurrently, which ensures that there is no possibility of interaction. This rules out cases where intuition might suppose that concurrency is possible. For example, the actions:

```
(:action a
  :precondition (or p q)
  :effect (r))

(:action b
  :precondition (p)
  :effect (and (not p) (s)))
```

could, one might suppose, be executed simultaneously in a state in which both p and q hold. The following definition asserts, however, that the two actions are mutex. The reason we have chosen such a constrained definition is because checking for mutex actions must be

为了与当前规划器的输出保持兼容性，做出了以下让步：如果计划被呈现为没有时间点的动作序列，则推断第一个动作在时间1应用，后续动作按整数时间点依次间隔一单位时间应用。

一个简单的计划是更熟悉的 STRIPS-风格经典计划的一个轻微推广，因为动作被标记为它们要被执行的时间。

定义 11 简单计划 一个简单计划，SP，对于一个规划实例，I，由一个有限的时间简单动作集合组成，这些动作是 pairs (t, a)，其中 t 是一个有理值时间，a 是一个动作名称。

发生序列，$\{t_i\}_{i=0\ldots k}$ 对于 SP 是时间集合中在时间简单动作中出现的有序时间序列，SP。所有的 t_i 必须大于 0。序列可能为空（一个空计划）。

在时间 t，E_t发生，其中 t 在 SP的发生序列中，是出现在与时间 t 在 SP相关联的时间简单动作中的（简单）动作名称集合。

因此，一个计划由一个发生序列组成，每个发生是一个在特定时间并发应用的动名名称集合，序列按时间顺序排列。这些发生发生的时间形成发生序列。应该注意的是，当动名模式包含条件效果时，动名名称是模糊的——扁平化结果的后果是将这些动作分成具有相同名称的多个动作，通过它们的先决条件来区分。然而，在给定的逻辑状态中，每个具有相同名称的动作集合中最多只有一个可以适用，因为该集合中每个动作的先决条件必然与其他集合中的任何其他动作的先决条件不一致，这是由于条件效果在它们诱导的动作模式对之间分布的方式。

为了处理并发操作，我们需要定义那些操作的效果彼此一致的情况。这个问题最早在5.1节中讨论。 PDDL2.1 的互斥规则是GraphPlan（Blum & Furst，1995）中操作互斥条件的扩展。这个扩展处理了两个额外的特性：语言的扩展表达能力（包括任意命题连接词）和数值表达式的引入。我们对并发执行的操作设定了一个非常保守的条件，以确保没有交互的可能性。这排除了直觉上可能认为并发是可能的那些情况。例如，操作：

(:action a
  :precondition (or p q)
  :effect (r))

(:action b
  :precondition (p)
  :效果 (and (not p) (s)))

或许，在 p 和 q 同时成立的状态下，这可以同时执行。然而，以下定义断言这两个操作是互斥的。我们选择这种受限定义的原因是必须检查互斥操作。

tractable and handling the case implied by this example would appear to require checking the consequence of interleaving preconditions and effects in all possible orderings. The condition on primitive numeric expressions has already been discussed — it determines that the update effects can be executed concurrently and that they do not affect values which are then tested by preconditions (regardless of whether the results of those tests matter to the satisfaction of their enclosing proposition). This is the rule of *no moving targets*: no concurrent actions can affect the parts of the state relevant to the precondition tests of other actions in the set, regardless of whether those effects might be harmful or not. It might be considered odd that the preconditions of one action cannot refer to literals in the add effects of a concurrent action. We require this because preconditions can be negative, in which case their interaction with add effects is analogous to the interaction between positive preconditions and delete effects. The no moving targets rule makes the cost of determining whether a set of actions can be applied concurrently polynomial in the size of the set of actions and their pre- and post-conditions.

Definition 12 Mutex Actions *Two grounded actions, a and b are* non-interfering *if*

$$GPre_a \cap (Add_b \cup Del_b) = GPre_b \cap (Add_a \cup Del_a) = \emptyset$$
$$Add_a \cap Del_b = Add_b \cap Del_a = \emptyset$$
$$L_a \cap R_b = R_a \cap L_b = \emptyset$$
$$L_a \cap L_b \subseteq L_a^* \cup L_b^*$$

If two actions are not non-interfering they are mutex.

The last clause of this definition asserts that concurrent actions can only update the same values if they both do so by additive assignment effects.

We are now ready to define the conditions under which a simple plan is valid. We can separate the executability of a plan from whether it actually achieves the intended goal. We will say that a plan is valid if it is executable *and* achieves the final goal. Executability is defined in terms of the sequence of states that the plan induces by sequentially executing the happenings that it defines.

Definition 13 Happening Execution *Given a state, $(t, s, \mathbf{x})$ and a happening, H, the* activity *for H is the set of grounded actions*

$$A_H = \{a | \textit{the name for } a \textit{ is in } H, a \textit{ is valid and } Pre_a \textit{ is satisfied in } (t, s, \mathbf{x})\}$$

The result of executing a happening, *H, associated with time t_H, in a state $(t, s, \mathbf{x})$ is undefined if $|A_H| \neq |H|$ or if any pair of actions in A_H is mutex. Otherwise, it is the state $(t_H, s', \mathbf{x}')$ where*

$$s' = (s \setminus \bigcup_{a \in A_H} Del_a) \cup \bigcup_{a \in A_H} Add_a$$

and $\mathbf{x}'$ is the result of applying the composition of the functions $\{\mathrm{NPF}_a \,|\, a \in A_H\}$ to $\mathbf{x}$.

Since the functions $\{NPF_a \,|\, a \in A_H\}$ must affect different primitive numeric expressions, except where they represent additive assignment effects, these functions will commute and

可行的，并且处理这个例子所隐含的情况似乎需要检查所有可能的顺序中并发前提条件和效果的结果。关于原始数值表达式的条件已经讨论过——它决定了更新效果可以并发执行，并且它们不会影响随后由前提条件测试的值（无论这些测试的结果是否对其包含的命题的满足有影响）。这是“目标不移动”的规则：没有任何并发操作可以影响其他操作在集合中相关的前提条件测试的状态部分，无论这些效果是否可能有害。有人可能会觉得奇怪，一个操作的前提条件不能引用并发操作的添加效果中的文字。我们要求这一点，因为前提条件可以是负面的，在这种情况下，它们与添加效果的交互类似于正前提条件与删除效果的交互。目标不移动规则使得确定一组操作是否可以并发应用的成本是关于操作集合及其前提条件和后置条件的多项式。

定义 12 互斥操作 两个接地操作， a 和 b 是非干扰的，如果

$$GPre_a \cap (Add_b \cup Del_b) = GPre_b \cap (Add_a \cup Del_a) = \emptyset$$
$$Add_a \cap Del_b = Add_b \cap Del_a = \emptyset$$
$$L_a \cap R_b = R_a \cap L_b = \emptyset$$
$$L_a \cap L_b \subseteq L_a^* \cup L_b^*$$

如果两个操作不是非干扰的，它们就是互斥的。

这个定义的最后一句断言，并发操作只能通过加性赋值效应更新相同的值。

我们现在准备好定义简单计划有效性的条件。我们可以将计划的可行性与其是否实际实现预期目标分开。我们将说一个计划是有效的，如果它是可行的并且实现了最终目标。可行性是在计划通过顺序执行其定义的事件序列所诱导的状态序列方面定义的。

定义 13 事件执行 给定一个状态，$(t, s,$x$)$ 和一个事件， H，的
活动 H 是接地操作的集合

$$A_H = \{a | \, a \text{的名称在 } H\text{中}， \; a \text{是有效的}， \; Pre_a \text{在}(t, s, \mathrm{x})\text{中满足}\}$$

执行与时间 t_H相关的事件 H在状态$(t, s,$ x$)$中的结果未定义 *if* $|A_H| \neq |H|$ ，或者如果 A_H 中的任何一对操作是互斥的。否则，它是状态$(t_H, s', \mathbf{x}')$，其中

$$s' = (s \setminus \bigcup_{a \in A_H} Del_a) \cup \bigcup_{a \in A_H} Add_a$$

并且x′是应用函数 $\{\mathrm{NPF}_a \,|\, a \in A_H\}$ 的组合到x的结果。

由于函数 $\{NPF_a \,|\, a \in A_H\}$ 必须影响不同的原始数值表达式，除了它们表示加法赋值效果的情况外，这些函数将交换，

therefore the order in which the functions are applied is irrelevant. Therefore, the value of $\mathbf{x}'$ is well-defined in this last definition. The requirement that the activity for a happening must have the same number of elements as the happening itself is simply a constraint that ensures that each action name in the happening leads to a valid action that is applicable in the appropriate state. We have already seen that conditional effects induce the construction of families of grounded actions, but that at most one of each family can be applicable in a state. If none of them is applicable for a given name, then this must mean that the precondition is unsatisfied, regardless of the conditional effects. In this case, we are asserting that the attempt to apply the action has undefined interpretation.

Definition 14 Executability *A simple plan, SP, for a planning instance, I, is executable if it defines a happening sequence, $\{t_i\}_{i=0\ldots k}$, and there is a sequence of states, $\{S_i\}_{i=0\ldots k+1}$ such that S_0 is the initial state for the planning instance and for each $i = 0\ldots k$, S_{i+1} is the result of executing the happening at time t_i in SP.*

The state S_{k+1} is called the final state *produced by SP and the state sequence $\{S_i\}_{i=0\ldots k+1}$ is called the* trace *of SP. Note that an executable plan produces a unique trace.*

Definition 15 Validity of a Simple Plan *A simple plan (for a planning instance, I) is* valid *if it is executable and produces a final state S, such that the goal specification for I is satisfied in S.*

8. The Semantics of Durative Actions

Plans with durative actions with discrete effects can be given a semantics in terms of the semantics of simple plans. Handling durative actions that have continuous effects is more complex — we discuss this further in Section 9.

Durative actions appearing in a plan must be given with an additional field indicating the duration. This is given with the syntax:

$$t : (action\, p_1 \ldots p_n)\, [d]$$

where d is a rational valued duration, written in floating point syntax.

Durative actions are introduced into the framework we have defined so far by generalising Definition 1 to include durative action schemas. The definition of the grounded action must now be extended to define the form of grounded durative actions. However, this definition can be given in such a way that we associate with each durative action two simple (non-durative) actions, corresponding to the end points of the durative action. These simple actions can, together, simulate almost all of the behaviour of the durative action. The only aspects that are not captured in this pair of simple actions are the duration of the durative action and the invariants that must hold over that duration. These two elements can, however, be simply handled in a minor extension to the semantics of simple plans, and this is the approach we adopt. By taking this route we avoid any difficulties in establishing the effects of interactions between durative actions — this is all handled by the semantics for the concurrent activity within a simple plan. As we will see, one difficulty in this account is the handling of durative actions with conditional effects that contain conditions and effects that are associated with different times or conditions that must hold over the entire duration of

因此函数应用的顺序无关紧要。因此，在最后一个定义中，x' 的值是明确定义的。要求一个事件的活动必须具有与事件本身相同数量的元素，仅仅是一个确保事件中的每个动作名称都引导到一个在适当状态下适用的有效动作的约束。我们已经看到条件效果会导致构建一组扎根动作，但在任何状态下最多只有一个每个家族中的动作适用。如果给定的名称对它们都不适用，那么这意味着前提条件未满足，无论条件效果如何。在这种情况下，我们断言应用动作的尝试具有未定义的解释。

定义 14 可执行性 一个简单的计划，SP，对于一个规划实例，I，是可执行的，如果它定义了一个事件序列，$\{t_i\}_{i=0\ldots k}$，并且存在一个状态序列，$\{S_i\}_{i=0\ldots k+1}$，使得 S_0 是规划实例的初始状态，并且对于每个 $i = 0\ldots k$，S_{i+1} 是在时间 t_i 在 SP 中执行事件的结果。

状态 S_{k+1} 被称为由 SP 生成的最终状态，状态序列 $\{S_i\}_{i=0\ldots k+1}$ 被称为 SP 的轨迹。注意，一个可执行的计划产生一个唯一的轨迹。

定义 15 简单计划的有效性 一个简单计划（对于一个规划实例，I）是有效的，如果它是可执行的并且产生一个最终状态 S，使得 I 的目标规范在 S 中得到满足。

8. 持续动作的语义

具有离散效果的持续动作的计划可以基于简单计划的语义来给出语义。处理具有连续效果的持续动作更复杂——我们在第 9 节中进一步讨论这一点。

在计划中出现的持续动作必须使用一个附加字段来指示持续时间。这是使用以下语法给出的：

$$t : (action\, p_1 \ldots p_n)\, [d]$$

其中 d 是一个有理值持续时间，使用浮点语法表示。

持续性行为通过将定义1推广到包含持续性行为模式被引入到我们迄今定义的框架中。现在必须扩展接地行为的定义来定义接地持续性行为的形式。然而，这个定义可以以这种方式给出，即我们为每个持续性行为关联两个简单（非持续）行为，对应于持续性行为的端点。这些简单行为可以一起模拟持续性行为的几乎所有行为。这对简单行为没有捕捉到的方面是持续性行为的持续时间和必须在该持续时间上保持的不变量。然而，这两个元素可以在简单计划的语义的微小扩展中简单地处理，这就是我们采用的方法。通过采取这种方法，我们避免了在建立持续性行为之间交互作用的效果方面的任何困难——所有这些都是由简单计划内的并发活动的语义处理的。正如我们将看到的，在这个解释中的一个困难是处理具有条件效果的持续性行为，这些条件包含与不同时间或必须在整个持续时间上保持的条件相关联的效果

the action. Since these cases complicate the semantics we will postpone treatment of them until the next section and begin with durative actions without conditional effects.

The mapping from durative actions to non-durative actions has the important consequence that the mutex relation implied between non-durative actions is (advantageously) weaker than the strong mutex relation used in, for example, TGP (Smith & Weld, 1999). Two durative actions can be applied concurrently provided that the end-points of one action do not interact either with the end-points (if simultaneous) or the invariants of the other action.

Definition 16 Grounded Durative Actions *Durative actions are grounded in the same way as simple actions (see Definition 6), by replacing their formal parameters with constants from the planning instance and expanding quantified propositions. The definition of durative actions requires that the condition be a conjunction of temporally annotated propositions. Each temporally annotated proposition is of the form* `(at start p)`, `(at end p)` *or* `(over all p)`, *where p is an unannotated proposition. Similarly, the effects of a durative action (without continuous or conditional effects) are a conjunction of temporally annotated simple effects.*

The duration field of DA defines a conjunction of propositions that can be separated into DC^{DA}_{start} *and* DC^{DA}_{end}, *the* duration conditions *for the start and end of DA, with terms being arithmetic expressions and* `?duration`. *The separation is conducted in the obvious way, placing* at start *conditions into* DC^{DA}_{start} *and* at end *conditions into* DC^{DA}_{end}.

Each grounded durative action, DA, with no continuous effects and no conditional effects defines two parameterised simple actions DA_{start} *and* DA_{end}, *where the parameter is the* `?duration` *value, and a single additional simple action* DA_{inv}, *as follows.*

DA_{start} *(*DA_{end}*) has precondition equal to the conjunction of the set of all propositions, p, such that* `(at start p)` *(*`(at end p)`*) is a condition of DA, together with* DC^{DA}_{start} *(*DC^{DA}_{end}*), and effect equal to the conjunction of all the simple effects, e, such that* `(at start e)` *(*`(at end e)`*) is an effect of DA (respectively).*

DA_{inv}, *is defined to be the simple action with precondition equal to the conjunction of all propositions, p, such that* `(over all p)` *is a condition of DA. It has an empty effect.*

Every conjunct in the condition of DA contributes to the precondition of precisely one of DA_{start}, DA_{end} *or* DA_{inv}. *Every conjunct in the effect of DA contributes to the effect of precisely one of* DA_{start} *or* DA_{end}. *For convenience,* DA_{start} *(*DA_{end}, DA_{inv}*) will be used to refer to both the entire (respective) simple action and also to just its name.*

The actions DA_{start} and DA_{end} are parameterised by `?duration` and this parameter must be substituted with the correct duration value in order to arrive at the two simple actions corresponding to the start and end of a durative action.

Definition 17 Plans *A* plan, *P, with durative actions, for a planning instance, I, consists of a finite collection of* timed actions *which are pairs, each either of the form* (t, a), *where t is a rational-valued time and a is a simple action name – an action schema name together with the constants instantiating the arguments of the schema, or of the form* $(t, a[t'])$, *where t is a rational-valued time, a is a durative action name and* t' *is a non-negative rational-valued duration.*

的行为。由于这些情况使语义复杂化，我们将推迟处理它们，直到下一节，并从没有条件效果的持续性行为开始。

从持续动作到非持续动作的映射有一个重要的后果，即非持续动作之间隐含的互斥关系（有利地）比TGP（Smith & Weld, 1999）中使用的强互斥关系弱。只要一个动作的终点不与另一个动作的终点（如果是同时的）或其不变式相互作用，两个持续动作就可以并发应用。

定义16 基于持续动作 持续动作与简单动作一样（见定义6）一样被基于，通过用规划实例中的常量替换它们的正式参数并扩展量化命题。持续动作的定义要求条件是时间标记命题的合取。每个时间标记命题的形式为(at start p)，(at end p)或(over all p)，其中 p 是一个未标记的命题。类似地，持续动作的效果（没有连续或条件效果）是时间标记简单效果的合取。

DA 的持续时间字段定义了一个可以分离为DC^{DA}_{start} 和 DC^{DA}_{end}的命题合取，即 DA的开始和结束持续时间条件，其中术语是算术表达式和? 持续时间。分离以明显的方式进行，将开始条件放入 DC^{DA}_{start} ，将结束条件放入 DC^{DA}_{end}。

每个有持续效果且无条件效果的根持续时间动作， DA，定义了两个参数化简单动作 DA_{start} 和 DA_{end}，其中参数是? 持续时间值，以及一个额外的简单动作 DA_{inv}，如下所示。

DA_{start} （DA_{end})的前置条件等于所有命题 p的合取，其中（在开始时 p）((在结束时 p))是 DA的条件之一，以及DC^{DA}_{start} （DC^{DA}_{end})，效果等于所有简单效果 e的合取，其中（在开始时 e）((在结束时 e))是 DA的效果（分别）。

DA_{inv}, 被定义为具有前提条件等于所有命题 p的合取的简单动作，使得（对于所有p）是 DA的条件。它没有空效果。

在 DA 的条件中的每个合取项都贡献给 DA_{start}、 DA_{end} 或 DA_{inv}中恰好一个的前提条件。在 DA 的效果中的每个合取项都贡献给 DA_{start} 或 DA_{end}中恰好一个的效果。为了方便起见，DA_{start} (DA_{end}, DA_{inv})将用于指代整个（相应的）简单动作，以及仅指代其名称。

动作 DA_{start} 和 DA_{end} 由? duration参数化，并且此参数必须用正确的持续时间值替换，以得到与持续动作的开始和结束相对应的两个简单动作。

定义 17 计划 A 计划， P，具有持续性行动，对于一个规划实例， I，由一个有限的时间行动集合组成，这些行动是成对的，每个要么是形式（t， a），其中 t 是一个实数值时间并且 a 是一个简单行动名称——一个行动模式名称以及实例化模式参数的常量，要么是形式（t， $a[t']$），其中 t 是一个实数值时间， a 是一个持续性行动名称并且 t' 是一个非负实数值持续时间。

Definition 18 Induced Simple Plan *If P is a plan then the* happening sequence *for P is $\{t_i\}_{i=0\ldots k}$, the ordered sequence of time points formed from the set of times*[1]

$$\{t \mid (t,a) \in P \, or \, (t,a[t']) \in P \, or \, (t-t',a[t']) \in P\}$$

The induced simple plan *for a plan P, simplify(P), is the set of pairs defined as follows:*

- *(t,a) for each $(t,a) \in P$ where a is a simple (non-durative) action name.*
- *$(t, a_{start}[?\textbf{duration} := t'])$ and $(t+t', a_{end}[?\textbf{duration} := t'])$ (these expressions are simple timed actions – the square brackets denote substitution of t' for* `?duration` *in this case) for all pairs $(t,a[t']) \in P$, where a is a durative action name.*
- *$((t_i + t_{i+1})/2, a_{inv})$ for each pair $(t,a[t']) \in P$ and for each i such that $t \leq t_i < t+t'$, where t_i and t_{i+1} are in the happening sequence for P.*

The process of transforming a plan into a simple plan involves introducing actions to represent the end points of the intervals over which the durative actions in the plan are applicable. Duration constraints convert into simple preconditions on start or end actions, requiring the substitution of a numeric value for the `?duration` field to complete the conversion into simple actions. The complication to this process is that invariants cannot be associated with the end points, but must be checked throughout the interval. This is achieved by adding to the simple plan a collection of special actions responsible for checking the invariants. These actions are added between each pair of happenings in the original plan lying between the start and end point of the durative action. Because the semantics of simple plans requires that the preconditions of actions in the plan be satisfied, even though they might have no effects, the consequence of putting these monitoring actions into the simple plan is to ensure that the original plan is judged valid only if the invariants remain true, firstly, after the start of the durative action and, subsequently, after each happening that occurs throughout the duration of the durative action. One possibility is to make these monitoring actions occur at the same times as the updating actions, but this would require values to be accessed at the same time as they might be being updated, violating the no moving targets rule. In order to avoid this problem the monitoring actions are interleaved with the updating actions by inserting them midway between pairs of successive happenings in the interval over which each durative action is executed. Only happenings in the original plan need be considered when carrying out this insertion, since the invariant-checking actions themselves cannot have any effect on the states in which they are checked.

Alternative treatments of invariants are possible, but an important advantage of the approach we have taken is that the semantics rests, finally, on a state-transition model in a form that is familiar to the planning community. That is, plans can be seen as recipes for state-transition sequences, with each state-transition being a function from the current state of the world to the next. However, durative actions complicate this picture because they rely on a commitment, once a durative action has been started, to follow it through to completion. That commitment involves some sort of communication across the duration of the plan. The communication can be managed by structures outside the plan, that examine

1. Care should be taken in reading this definition — the last disjunct allows the time corresponding to the *end* of execution of a durative action to be included as a happening time.

定义 18 诱导简单计划 如果 P 是一个计划那么 P 的事件序列是 $\{t_i\}_{i=0\ldots k}$，由时间集合 [1] 形成的有序时间点序列

$$\{t \mid (t,a) \in P \, or \, (t,a[t']) \in P \, or \, (t-t',a[t']) \in P\}$$

为计划 P 生成的简单计划 simplify(P) 是如下定义的对集合：

- (t,a) 对每个 $(t,a) \in P$，其中 a 是一个简单（非持续）动作名称。
- $(t, a_{start}[?\text{duration} := t'])$ 和 $(t+t', a_{end}[?\text{duration} := t'])$（这些表达式是简单时序动作——方括号表示在此情况下用 t' 替换 ?duration）对所有对 $(t,a[t']) \in P$，其中 a 是一个持续动作名称。
- $((t_i + t_{i+1})/2, a_{inv})$ 对每个对 $(t,a[t']) \in P$ 和对每个 i，使得 $t \leq t_i < t+t'$，其中 t_i 和 t_{i+1} 在 P 的发生序列中。

将计划转换为简单计划的过程涉及引入动作来表示计划中持续动作适用的区间端点。持续时间约束转换为简单动作的开始或结束的前置条件，需要用数值替换 ?duration 字段以完成转换为简单动作。这个过程的一个复杂之处是，不变式不能与端点关联，但必须在整个区间内进行检查。这是通过向简单计划添加一组特殊动作来实现的，这些动作负责检查不变式。这些动作被添加到原始计划中位于持续动作开始和结束点之间的每对发生之间。由于简单计划的语义要求计划中的动作的前置条件必须得到满足，即使它们可能没有效果，将这些监控动作添加到简单计划的结果是确保只有当不变式在持续动作开始后首先保持为真，并且在持续动作持续期间每次发生后仍然保持为真时，原始计划才被判断为有效。一种可能性是让这些监控动作与更新动作在同一时间发生，但这将需要在它们可能被更新的同时访问值，违反了无移动目标规则。为了避免这个问题，监控动作通过在每对连续发生之间插入它们来与更新动作交错进行，这些发生位于每个持续动作执行的区间内。在执行此插入时，只需要考虑原始计划中的发生，因为不变式检查动作本身不能对它们被检查的状态产生任何影响。

对不变量的替代治疗方法是可能的，但我们所采取的方法的一个重要优势在于，其语义最终基于一个为规划领域所熟悉的状变模型。也就是说，计划可以被看作是状态转换序列的食谱，其中每个状态转换是从当前世界状态到下一个状态的函数。然而，持续动作使这一图景复杂化，因为它们依赖于一种承诺，一旦持续动作开始，就必须坚持完成它。这种承诺涉及某种跨计划持续时间的通信。这种通信可以由计划外部的结构管理，这些结构检查

1. 阅读此定义时应注意——最后一个合取项允许将一个持续动作执行所对应的时间 *end* 包括为发生时间。

the trace, or by artificial modification of the plan itself to ensure that states carry extra information from the start to the end of the durative action. The latter approach has the disadvantage that as durative actions become more complex the artificial components that must be added to the plan become more intrusive. This is particularly apparent in the treatment of conditional effects that require conditions tested at the start of a durative action, or across its duration, but effects that are triggered at the end, since these cases require some sort of "memory" in the state to remember the status of the tested conditions from the start of the durative action to the end point. These memory conditions allow us to avoid embedding an entire execution history in a state by substituting an *ad hoc* memory of the history for just those propositions and at just those times it is required. The management of conditional effects of this form, in the mapping from durative actions to simple actions, is discussed further in Section 8.1.

We can now conclude the definitions supporting the validity of a plan with durative actions.

Definition 19 Executability of a Plan *A plan, P (for a planning instance), is executable if the induced simple plan for P, simplify(P) is executable, producing the trace $\{S_i = (t_i, s_i, \mathbf{v}_i)\}_{i=0\ldots k}$.*

Definition 20 Validity of a Plan *A plan, P (for a planning instance), is valid if it is executable and if the goal specification is satisfied in the final state produced by its induced simple plan.*

8.1 Durative Actions with Conditional Effects

We now explain how the mapping described in the previous section is extended to deal with durative actions containing conditional effects.

First, we observe that temporally annotated conditions and effects can be accumulated, because the temporal annotation distributes through logical conjunction. Therefore, we can convert conditional effects so that their conditions are simple conjunctions of at most one *at start* condition, at most one *at end* condition and at most one *over all* condition. It should be noted that we do not allow logical connectives other than conjunction in combining temporally annotated propositions. Allowing other connectives would create significant further complexity in the semantics and create potentially paradoxical opportunities for communication from future states to earlier states. Similarly to conditions, durative action effects can be reduced to a conjunction of at most one *at start* effect and at most one *at end* effect. Treatment of conditional effects then divides into three cases. The first case is very straightforward: any effect in a durative action of the form `(when (at t p) (at t q))`, where the condition and the effect bear the same single temporal annotation, can be transformed into a simple conditional effect of the form `(when p q)` attached to the start or end simple action according to whether t is *start* or *end*. Since this case is straightforward we will not explicitly extend the previous definitions to cope with it. The second case is one in which the condition of a condition effect has *at start* conditions and the effect has *at end* effects.

Note that we consider conditional effects in which the effects occur at the start, but with conditions dependent on the state at the end or over the duration of the action, to be

轨迹，或者通过人工修改计划本身来确保状态从持续动作的开始到结束携带额外的信息。后者方法的缺点是，随着持续动作变得越来越复杂，必须添加到计划中的人工组件变得越来越侵入性。这在处理需要持续动作开始时测试的条件或在其持续时间内测试的条件，但效果在结束时被触发的条件时尤其明显，因为这些情况需要在状态中保留某种“记忆”，以记住从持续动作开始到结束点所测试的条件的状态。这些记忆条件使我们能够通过用针对需要时仅那些命题和仅那些时间的临时历史记忆来代替整个执行历史，来避免将整个执行历史嵌入状态中。在将持续动作映射到简单动作时，管理这种形式的条件效果在8.1节中进一步讨论。

现在我们可以总结支持具有持续性动作的计划有效性的定义。

定义 19 计划的可执行性 计划 P (针对一个规划实例)，如果由 P 生成的简单计划 simplify(P) 是可执行的，并产生轨迹$\{S_i = (t_i, s_i, \mathbf{v}_i)\}_{i=0\ldots k}$，则是可执行的。

定义 20 计划的有效性 计划 P (针对一个规划实例)，如果它是可执行的，并且其生成的简单计划在最终状态中满足目标规范，则是有效的。

8.1 具有条件效果的持续性动作

我们现在解释上一节中描述的映射如何扩展以处理包含条件效果的持续性动作。

首先，我们观察到时间标注的条件和效果可以累积，因为时间标注通过逻辑合取传播。因此，我们可以将条件效果转换为它们的条件是至多一个起始条件、至多一个终止条件和至多一个所有条件的简单合取。应该注意的是，我们不允许在组合时间标注命题时使用除合取以外的逻辑连接词。允许其他连接词会在语义中产生显著的进一步复杂性，并可能为从未来状态到早期状态的通信创造潜在的悖论性机会。类似于条件，持续性动作的效果可以简化为至多一个起始效果和至多一个终止效果的合取。条件效果的处理分为三种情况。第一种情况非常简单：任何形式为 (when (at t p) (at t q)) 的持续性动作中的效果，其中条件和效果具有相同的时间标注，可以转换为形式为 (when p q) 的简单条件效果，根据 t 是起始还是终止附加到起始或终止简单动作上。由于这种情况很简单，我们将不会明确扩展以前定义来处理它。第二种情况是条件效果的条件具有起始条件，而效果具有终止效果。

请注意，我们认为在开始时发生但条件取决于结束状态或行动持续期间的状态的条件下发生的效果为

meaningless. This is because they reverse the expected behaviour of causality, where cause precedes effect. In any attempt to validate a plan by constructing a trace such reversed causality would be a huge problem, since we could not determine the initial effects of applying a durative action until we had seen what conditions held over the subsequent interval and conclusion of its activity, but, equally, we could not see what the effects of activity during the interval would be without seeing the initial effects of applying the durative action. This paradox is created by the opportunity for an action to change the past.

To handle this second case we need to modify the state after the start of the durative action to "remember" whether the start conditions were satisfied and communicate this to the end of the durative action where it can then be simply looked up in the (then) current state to determine whether the conditional effect should be applied. We apply a transformation to conditional effects of the form `(when (and (at start ps) (at end pe)) (at end q))` into a conditional effect added to the start simple action, `(when ps (M`$_{ps}$`))`, and a conditional effect added to the end simple action, `(when (and pe (M`$_{ps}$`)) q)`, where `M`$_{ps}$ is a special new proposition, unique to the particular conditional effect of the particular application of the durative action being transformed. By ensuring that this proposition is unique in this way, there is no possibility of any other action in the plan interfering with it, so it represents an isolated memory of the fact that `ps` held in the state at which the durative action was started. If a conditional effect does not have *at end* conditions, the same transformation can be applied, simply ignoring `pe` in the previous discussion. Figure 15 depicts the transformation of a single durative action, A, with a conditional effect, into a collection of level 2 actions, complete with the appropriate "memory" proposition (in this case called P^*).

The importance of the memory introduced in this transformation is explained in Figures 16 and 17. Figure 16 shows the ambiguity that results from not remembering how a state, on the trajectory of a plan, was reached. The figure illustrates that if one is in a state $(P, Q, \neg R)$ at the point when durative action A (as described in Figure 15) ends, it is impossible to determine from the state alone whether R should be added or not. This is because it is possible to have reached the state $(P, Q, \neg R)$ by at least two different paths, with at least one path having seen A started in a state in which P held and at least one path having seen A started in a state in which P did not hold (using an action, achieve-P, with P as its only effect). The state $(P, Q, \neg R)$ does not contain any information to disambiguate which path was used to reach it, and hence cannot determine the correct value of R after A ends.

The third, and final, case is where the durative action has conditional effects of the form:

```
(when (and (at start ps) (over all pi) (at end pe)) (at end q)).
```

Again, if the effect has no *at start* or *at end* conditions the following transformation can be applied simply ignoring `ps` or `pe` as appropriate. In this case we need to construct a transformation that "remembers" not only whether `ps` held in the state at which the durative action is first applied, but also whether `pi` holds throughout the interval from the start to the end of the durative action. Unlike the invariants of durative actions, these conditions are not *required* to hold for the plan to be valid, but only determine what effects will occur at the end of the durative action. The idea is to use intervening monitoring actions, rather as we did for invariants in definition 18. This is achieved by adding a further effect to the start

这是因为它们颠倒了因果关系预期的行为，即原因先于结果。在尝试通过构建一个反向因果关系的轨迹来验证计划时，这将是一个巨大的问题，因为我们无法在看到后续时间段及其活动结论中保持的条件之前确定应用持续行动的初始效应，但同样，我们无法在没有看到应用持续行动的初始效应的情况下看到作用期间的活动效应。这个悖论是由行动改变过去的机会造成的。

要处理这种情况下的第二种情况，我们需要在持续动作开始后修改状态，以“记住”是否满足开始条件，并将此信息传达给持续动作的结束部分，在那里可以简单地在其（当时的）当前状态中查找以确定是否应应用条件效果。我们将形式为（当（和（在开始 ps）（在结束 pe））（在结束 q））的条件效果转换为一个添加到开始简单动作的条件效果（当 ps (M$_{ps}$)），以及一个添加到结束简单动作的条件效果（当（和 pe (M$_{ps}$)） q），其中 M$_{ps}$ 是一个特殊的命题，仅限于特定条件效果，该条件效果是特定持续动作应用转换的特定实例。通过确保该命题以这种方式是唯一的，没有任何其他计划中的动作会干扰它，因此它代表了在持续动作开始时状态中持有的 ps 的孤立记忆。如果一个条件效果没有在结束条件，相同的转换可以应用，只需忽略之前讨论中的 pe。图 15 描绘了一个持续动作 A 和一个条件效果的转换，转换为一个包含适当的“记忆”命题（在这种情况下称为 P^*）的 2 级动作集合。

这种转换中引入的内存的重要性在图16和图17中解释。图16显示了由于不记得计划轨迹上的状态是如何达到的而导致的歧义。该图说明，如果在持续动作 A（如图15所述）结束时处于状态 $(P, Q, \neg R)$，仅从状态本身无法确定是否应添加 R。这是因为可以通过至少两条不同的路径达到状态 $(P, Q, \neg R)$，其中至少有一条路径在状态中看到了 A 的启动，且 P 持有，而至少有一条路径在状态中看到了 A 的启动，且 P 未持有（使用动作 achieve-P，其唯一效果为 P）。状态 $(P, Q, \neg R)$ 不包含任何信息来消除哪种路径被用来达到它，因此无法确定 A 结束后 R 的正确值。

第三个，也是最后一个，情况是持续性行为具有条件效果，形式如下：

(当 (并且 (在开始时 ps) (覆盖所有 pi) (在结束时 pe)) (在结束时 q))

同样地，如果效果没有 at start 或 at end 条件，可以简单地忽略 ps 或 pe。在这种情况下，我们需要构建一个“记住”不仅 ps 在持续性行为首次应用的状态中是否成立，而且 pi 在持续性行为从开始到结束的整个区间内是否始终成立的转换。与持续性行为的不变性不同，这些条件不是计划有效的必要条件，而只是决定持续性行为结束时会发生什么效果。想法是使用中间监控行为，就像我们在定义 18 中为不变性所做的那样。这是通过在开始处添加进一步的效果来实现的

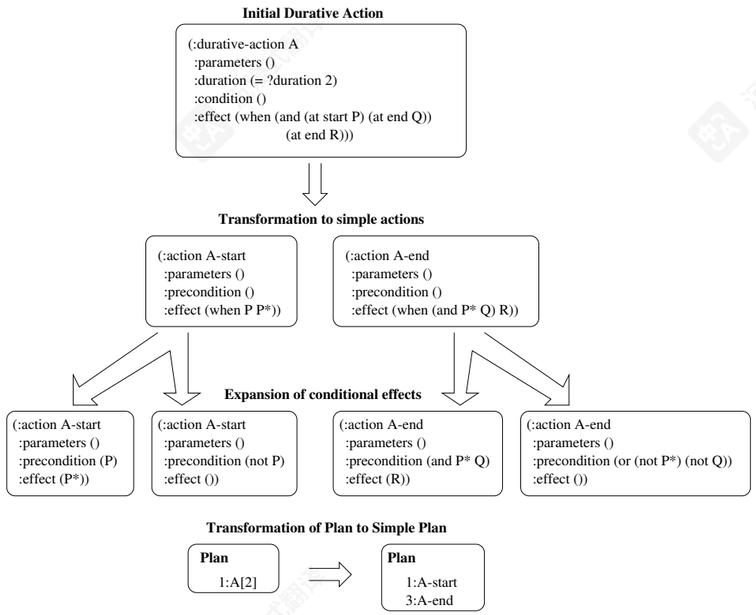

Figure 15: Conversion of a durative action into non-durative actions and their grounded forms.

PDDL2.1: Expressing Temporal Planning Domains

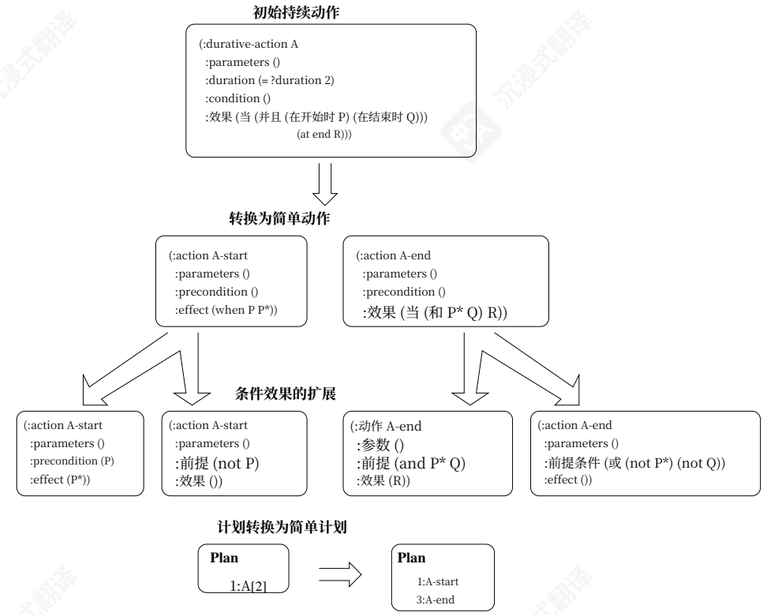

图15：将持续性动作转换为非持续性动作及其 grounding 形式。

99

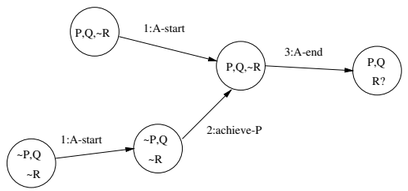

Figure 16: Flawed state space resulting from failure to record the path traversed when conditional effects span the interval of a durative action. The arc labelled *achieve-P* indicates the possible application of some action that achieves the proposition P.

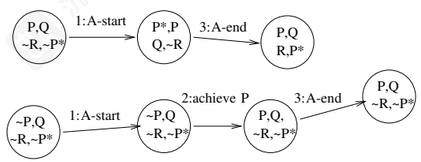

Figure 17: Correct state space showing use of "memory" proposition P*. The arc labelled *achieve P* indicates the possible application of some action that achieves the proposition P.

action: `(M`$_{pi}$`)`. Then, the monitoring (simple) actions that are required have no precondition, but a single conditional effect: `(when (and (M`$_{pi}$`) (not pi)) (not (M`$_{pi}$`)))`. Once again, `M`$_{pi}$ is a special new proposition unique to the conditional effect for the application instance of the durative action being transformed. The monitoring actions are added at all the intermediate points that are used for the monitoring actions in Definition 18. The same transformation used in the second case above is required again for the *at start* condition, `ps`, so `(when ps (M`$_{ps}$`))` is added as a conditional effect to the start simple action. Finally, we add a conditional effect to the end (simple) action: `(when (and (M`$_{ps}$`) (M`$_{pi}$`) pe) q)`. The effect of this machinery is to ensure that if the proposition `pi` becomes false at any time between the start and end of the durative action then `M`$_{pi}$ will be deleted, but otherwise at the end of the durative action `M`$_{ps}$ will hold precisely if `ps` held at the start of the action and `M`$_{pi}$ will hold precisely if `pi` has held over the entire duration of the durative action. Therefore, the conditional effect of the end action achieves the intuitively correct behaviour of asserting its conditional effect precisely when the *at start* condition held at the start of the durative action, the *at end* condition holds at the end of the durative action and its *over all* condition has held throughout the duration of the action.

The addition of these new memory-checking actions means that it is no longer true to claim that the added actions cannot change the state. However, memory propositions are unique to the task of communication for a single action instance, so the effects that memory-checking actions might have on these have no implications for other invariants.

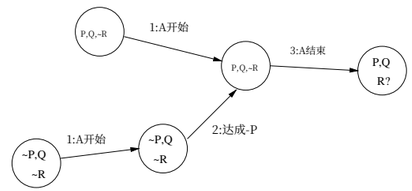

图16：由于未记录条件效果跨越持续性动作区间时遍历的路径而产生的缺陷状态空间。标有achieve-P的弧表示可能应用某个实现命题P的动作。

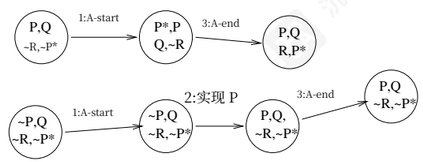

图 17：正确的状态空间，显示了使用“记忆”命题 P* 的方法。标有 achieve P 的弧表示可能应用某个实现命题 P 的动作。

动作: (M^{pi})。然后，所需的监控（简单）动作没有前提条件，但有一个条件效果：(当 (和 (M^{pi}) (not pi)) (not (M^{pi})))。再次，M^{pi} 是一个特殊的、仅用于该持续动作应用实例的条件效果的新命题。监控动作被添加到定义 18 中用于监控动作的所有中间点。同样，上述第二例中使用的转换也适用于 at start 条件 ps，因此 (当 ps (M^{ps})) 被添加为开始简单动作的条件效果。最后，我们为结束（简单）动作添加一个条件效果：(当 (和 (M^{ps}) (M^{pi}) pe) q)。这种机制的效果是确保如果命题 pi 在持续动作开始和结束之间的任何时候变为假，则 M^{pi} 将被删除，否则在持续动作结束时 M^{ps} 将精确地成立，如果 ps 在动作开始时成立，M^{pi} 将精确地成立，如果 pi 在整个持续动作期间都成立。因此，结束动作的条件效果实现了直观上正确的行为，即在持续动作开始时 at start 条件成立时、在持续动作结束时 at end 条件成立时以及其 overall 条件在整个动作期间都成立时，断言其条件效果。

添加这些新的内存检查操作意味着，声称添加的操作不能改变状态的说法不再成立。然而，内存命题是针对单个操作实例的通信任务所特有的，因此内存检查操作可能对这些产生的影响对其他不变式没有影响。

9. The Semantics of Continuous Durative Actions

The introduction of continuous durative actions complicates the semantics. It is no longer possible to handle invariants by insertion of simple actions between other happenings in a plan to test their continued satisfaction. In fact, continuous effects can, in principle, cause an invariant to be satisfied over some parts of an interval and not over others. Ignoring invariants for a moment, updates to numeric values caused by continuous effects can be applied as discrete updates at time points within the interval over which they apply. These updates behave slightly differently to the discrete updates we have seen in durative actions with discrete effects, since it *is* possible for a continuous update to affect a variable that is concurrently affected by a discrete update, or examined by a precondition, without creating an inconsistency. For example, if the water heating action in Figure 14 is applied with the concurrent addition of an egg to the pan with a precondition that the temperature of the water is between 90 and 95 degrees then the value of the temperature can be examined at the moment of application of the action adding the egg. This is because the temperature change is actually happening over the interval between the start of the heating and the point at which the egg is added, rather than as a discrete update at the point the egg is added. The temperature is not actually changed at the instant of the addition of the egg.

In this section we summarise the semantics for continuous actions. Where the semantics for discrete durative actions is defined in terms of the familiar state-transition semantics, the continuous semantics introduces a different formulation.

Definition 21 A Continuous Durative Action *A* continuous effect *is an effect expression that includes the symbol* `#t`*. A* continuous durative action *is a durative action with at least one continuous effect.*

Definition 22 Continuous Update Function *Let C be a set of ground continuous effects for a planning instance, I, and $St = (t, S, \mathbf{X})$ be a state. The* continuous update function *defined by C for state St is the function $f_C : \mathbb{R} \to \mathbb{R}^n$, where n is dim_I, such that:*

$$\frac{df_C}{dt} = g$$

and

$$f_C(0) = \mathbf{X}$$

where g is the update function generated for an action a with:

$$NP_a = \{ \texttt{(<op> P Q)} \mid \texttt{(<op> P (* \#t Q))} \in C\}$$

Definition 22 shows how the continuous effects of several continuous durative actions can be combined to create a single system of simultaneous differential equations whose solution, given an appropriate starting point, defines the evolution of the continuously varying values.

Definition 23 Induced Continuous Plan *Let I be a planning instance that includes continuous durative actions and P be a plan for I. The* induced continuous plan *for P is a triple, $(S, Invs, Cts)$, where S is $simplify(P)$, $Invs$ is the set of* invariant constraints*:*

$$Invs = \{(Q, t, t+d) \mid (t, a[d]) \in P \text{ and } \texttt{(over all Q)} \text{ is an invariant for } A\}$$

9. 连续持续动作的语义

连续持续操作的出现使语义变得复杂。不再可能通过在计划中的其他事件之间插入简单操作来处理不变式以测试其持续满足。实际上，连续效应原则上可以导致一个不变式在某个时间间隔的部分区域得到满足而在其他区域不满足。暂时忽略不变式，由连续效应引起的数值更新可以应用为在它们生效的时间间隔内的离散更新。这些更新与我们在具有离散效应的持续操作中看到的离散更新行为略有不同，因为连续更新可能影响一个同时受离散更新影响的变量，或被前置条件检查，而不会产生不一致。例如，如果图14中的水加热操作与同时向锅中添加一个鸡蛋的操作一起应用，并且有一个前提条件，即水的温度在90到95度之间，那么可以在添加鸡蛋的操作应用时刻检查温度的值。这是因为温度变化实际上发生在加热开始和鸡蛋添加点之间的时间间隔内，而不是在鸡蛋添加点作为一个离散更新发生。温度实际上在鸡蛋添加的瞬间并没有改变。

在本节中，我们总结了连续动作的语义。离散持续动作的语义是用熟悉的状态转换语义定义的，而连续语义引入了一种不同的表述方式。

定义 21 连续持续动作 一个连续效果是一个包含符号 #t 的效果表达式。一个连续持续动作是一个具有至少一个连续效果的持续动作。

定义 22 连续更新函数 令 C 是一个规划实例 I 的地面连续效果集合， $St = (t, S, \mathbf{X})$ 是一个状态。由 C 在状态 St 定义的连续更新函数是函数 $f_C : \mathbb{R} \to \mathbb{R}^n$，其中 n 是 $\dim_I$，使得：

$$\frac{df_C}{dt} = g$$

and

$$f_C(0) = \mathbf{X}$$

其中 g 是为动作 a 生成的更新函数：

$$NP_a = \{ \texttt{(<op> P Q)} \mid \texttt{(<op> P (* \#t Q))} \in C\}$$

定义 22 显示了如何将多个连续持续动作的连续效果组合起来，以创建一个同时微分方程系统，其解，给定一个适当的起点，定义了连续变化值的演化。

定义23诱导连续计划令 I 为一个包含连续持续动作的计划实例，且 P 为 I的计划。 P 的诱导连续计划是一个三元组，$(S, Invs, Cts)$，其中 S是 $simplify(P)$， $Invs$是不变约束的集合：

$$Invs = \{(Q, t, t+d) \mid (t, a[d]) \in P \text{ and } \texttt{(over all Q)} \text{ is an invariant for } A\}$$

Let t_i and t_{i+1} be two consecutive times in the happening sequence for $simplify(P)$. The set of active continuous effects *over (t_i, t_{i+1}) is:*

$$\{Q \mid (t, a[d]) \in P, (t_i, t_{i+1}) \subseteq [t, t+d] \text{ and } Q \text{ is a continuous effect of } a\}$$

and Cts is the set of systems of continuous effects*:*

$$Cts = \{(C, t_i, t_{i+1}) \mid C \text{ is the set of active continuous effects over } (t_i, t_{i+1})\}$$

The components of a continuous plan separate out the invariant conditions and continuous effects from the rest of the simple plan in order to allow correct application of the continuous updates and to allow confirmation that the invariants hold in the face of the continuous effects.

Definition 24 Trace *Let I be a planning instance that includes continuous durative actions, P be a plan for I, (SP, Inv, Cts) be the induced continuous plan for P, $\{t_i\}_{i=0...k}$ be the happening sequence for S and S_0 be the initial state for I. The* trace *for P is the sequence of states $\{S_i\}_{i=0...k+1}$ defined as follows:*

- *If there is no element $(C, t_i, t_{i+1}) \in Cts$ then S_{i+1} is the state resulting from applying the happening at t_i in the simple plan SP to the state S_i.*
- *If $(C, t_i, t_{i+1}) \in Cts$ then let T_i be the the state formed by substituting $f(t_{i+1} - t_i)$ for the numeric part of state S_i, where f is the continuous update function defined by C for state S_i. Then S_{i+1} is the state resulting from applying the happening at t_i in the simple plan SP to the state T_i. If f is undefined for any element in Cts then so is the trace.*

Definition 24 defines a trace in a similar fashion to the traces for simple plans and plans with durative actions. The key difference is the need to apply the continuous updates. These are handled by solving the systems of simultaneous differential equations across each interval in which they are active and then applying the result to update the numeric values across that interval. Of course, this is easier to describe than it is to do, since solving arbitrary simultaneous differential equations algorithmically is not generally possible. Under certain constraints this semantics can be implemented in order to confirm the validity of a plan automatically.

Definition 25 Invariant Safe *Let I be a planning instance that includes continuous durative actions, P be a plan for I, (S, Inv, Cts) be the induced continuous plan for P and $\{S_i\}_{i=0...k+1}$ be the trace for P. For each $(C, t_i, t_{i+1}) \in Cts$ let f_i be the continuous update function defined by C for S_i. P is* invariant safe *if, for each f_i that is defined and for each $(Q, t, u) \in Inv$ such that $[(t_i, t_{i+1})] = I \subseteq (t, u)$, then $\forall x \in I$, $\mathbb{N}um(s, Q)(f_i(x))$ where s is the logical state in S_i.*

In this definition, the symbols $[(..)]$ are used to mean that the interval I can be closed or open at either end.

令 t_i和 t_{i+1}为 $simplify(P)$发生序列中的两个连续时间。(t_i, t_{i+1}) 上的活跃连续效应的集合是:

$$\{Q \mid (t, a[d]) \in P, (t_i, t_{i+1}) \subseteq [t, t+d] \text{ and } Q \text{ is a continuous effect of } a\}$$

并且 Cts 是连续效应系统的集合:

$$Cts = \{(C, t_i, t_{i+1}) \mid C \text{ is the set of active continuous effects over } (t_i, t_{i+1})\}$$

连续计划的组件将不变条件与连续效应从简单计划的其余部分中分离出来，以允许正确应用连续更新，并允许确认不变条件在连续效应面前仍然成立。

定义 24 追踪 让 I 是一个包含连续持续动作的计划实例， P 是 I 的计划，(SP, Inv, Cts) 是为 P 诱导的连续计划，$\{t_i\}_{i=0...k}$ 是 S 的发生序列， S_0 是 I 的初始状态。 I 的追踪是状态序列 $\{S_i\}_{i=0...k+1}$ ，定义如下:

- 如果没有元素 $(C, t_i, t_{i+1}) \in Cts$，则 S_{i+1} 是在简单计划 SP 中将发生 t_i 应用于状态 S_i 得到的状态。
- 如果 $(C, t_i, t_{i+1}) \in Cts$，则让 T_i 是通过用 $f(t_{i+1} - t_i)$ 替换状态 S_i 的数字部分形成的状态，其中 f 是由 C 为状态 S_i 定义的连续更新函数。然后 S_{i+1} 是在简单计划 SP 中将发生 t_i 应用于状态 T_i 得到的状态。如果对于 Cts 中的任何元素 f 是未定义的，则追踪也是未定义的。

定义24以与简单计划和使用持续动作的计划类似的方式定义了跟踪。关键区别是需要应用连续更新。这些是通过在每个它们活动的区间内解决同步微分方程组来处理的，然后将结果应用于更新该区间内的数值。当然，这比实际做起来更容易描述，因为算法ically解决任意同步微分方程通常是不可能的。在一定的约束下，这种语义可以实施，以自动确认计划的有效性。

定义25 不变安全 让 I 是一个包含持续持续动作的计划实例， P 是 I 的计划，(S, Inv, Cts) 是 P 的诱导连续计划，$\{S_i\}_{i=0...k+1}$ 是 P 的跟踪。对于每个 $(C, t_i, t_{i+1}) \in Cts$，让 f_i 是由 C 定义的 S_i 的连续更新函数。 P 是不变安全的，如果对于每个定义的 f_i 和每个 $(Q, t, u) \in Inv$ 使得 $[(t_i, t_{i+1})] = I \subseteq (t, u)$，则 $\forall x \in I$，$\mathbb{N}um(s, Q)(f_i(x))$，其中 s 是 S_i 中的逻辑状态。

在这个定义中，符号 $[(..)]$ 用于表示区间 I 的两端可以是闭区间或开区间。

From a semantic point of view, invariants must be checked at every point in the interval over which they apply. When the interval contains only finitely many discrete changes then the obligation can be met by considering only the finite number of points at which change occurs (a fact that is exploited for discrete durative action plan semantics in Definition 18). When there is continuous change the obligation is much harder to meet. In practice, the invariants can be checked by examining the possible roots of the function describing continuous change, but finding those roots can be very difficult in general. Again, suitable constraints on the forms of differential equations expressed in a domain can make the validation problem tractable.

The last two definitions simply assemble the components to arrive at analogous definitions to those for executability and validity of simple plans and plans with durative actions.

Definition 26 Executability of a Plan *A plan P containing continuous durative actions, for planning instance I, with induced continuous plan $(S, Invs, Cts)$. P is* executable *if the trace for P is defined, $\{S_i\}_{i=0\ldots k+1}$, and it is invariant safe.*

Definition 27 Plan Validity *A plan P containing durative actions, for planning instance I is* valid *if it is executable, with trace $\{S_i\}_{i=0\ldots k+1}$ and S_{k+1} satisfies the goal in I.*

10. Plan Validation

Plan validation is an important part of the use of PDDL, particularly in its role for the competition. With approximately 5000 plans to consider in the competition in 2002, it can be seen that automation is essential. The validation problem is tractable for propositional versions of PDDL because plans are finite and can be validated simply by simulation of their execution. The issue is more complicated for PDDL2.1 because the potential for concurrent activity, possibly in the face of numeric change, makes it necessary to ensure that invariant properties are protected and that concurrent activity is non-interfering.

When durative actions are used there is a question over whether a plan should be considered valid if it does not contain all of the end points of the actions initiated in the plan. When an action is exploited in a plan for the effect it has at the end of its duration it is clear that that end point will be present in the plan, but when an action was selected for its start effect this is less clear. A match-striking action is performed for its start effect, not in order to have a burned out match at the end of a brief interval. It could be argued that, having obtained the desired start effect the end of the action is irrelevant and the plan can terminate (as soon as all goals are achieved) without ensuring that all initiated actions end safely. Indeed, the plan search process in Sapa (Do & Kambhampati, 2001) can terminate whilst there are still queued events awaiting the advancement of time. However, it is possible to conceive of situations in which the end point of an action, incorporated only for its start effect, introduces inconsistencies in the plan so that its inclusion would make the plan invalid. In these cases it seems that plan validity could be compromised by ignoring end effects.

In order to avoid having to resolve these complexities, we have taken the view that a PDDL2.1 plan is valid only if all action start and end points are explicit within the plan. Having identified that this is the case we then proceed to confirm that all happenings within the plan are mutex-free.

从语义角度来看，必须在整个适用区间内的每一点上检查不变式。当区间只包含有限多个离散变化时，可以通过考虑变化发生的有限个点来满足这一要求（这一事实被用于离散持续动作计划语义的定义18中）。当存在连续变化时，满足这一要求要困难得多。在实践中，可以通过检查描述连续变化的函数的可能根来检查不变式，但在一般情况下，找到这些根可能非常困难。同样，对域中微分方程形式的适当约束可以使验证问题变得可行。

最后两个定义简单地组装了组件，以得出与可执行性和简单计划以及具有持续动作的计划的有效性类似定义。

定义 26 计划的可执行性 一个包含持续持续动作的计划 P，对于规划实例 I，具有诱导的持续计划 $(S, Invs, Cts)$。 P 如果 P 的轨迹已定义， $\{S_i\}_{i=0\ldots k+1}$，并且它是保持不变的，则它是可执行的。

定义 27 计划的有效性 一个包含持续动作的计划 P，对于规划实例 I，如果它是可执行的，并且轨迹 $\{S_i\}_{i=0\ldots k+1}$ 和 S_{k+1} 满足 I 中的目标，则它是有效的。

10. 计划验证

计划验证是使用 PDDL 的重要部分，特别是在其作为竞争角色时。在 2002 年的比赛中要考虑大约 5000 个计划，可以看出自动化是必不可少的。验证问题对于 PDDL 的命题版本是可行的，因为计划是有限的，并且可以通过模拟其执行来简单地验证。对于 PDDL2.1 的问题更加复杂，因为并发活动的可能性，可能面对数字变化，使得必须确保不变属性得到保护，并且并发活动是非干扰的。

当使用持续性行动时，存在一个问题，即如果计划不包含计划中启动的所有行动的终点，是否应将其视为有效。当在一个计划中利用一个行动以达到其在持续时间结束时的效果时，很明显那个终点将存在于计划中，但当选择一个行动是为了其开始效果时，这就不太清楚了。一个点燃火柴的行动是为了其开始效果而执行的，而不是为了在短暂间隔结束时有一个烧尽的火柴。可以论证的是，在获得了所需的开端效果后，行动的终点无关紧要，计划可以在（一旦所有目标都实现）终止，而无需确保所有启动的行动都安全结束。事实上，Sapa（Do & Kambhampati，2001）中的计划搜索过程可以在仍有排队事件等待时间推进时终止。然而，可以设想一些情况，其中行动的终点仅因其开始效果而被包含，从而在计划中引入不一致性，以至于其包含会使计划无效。在这些情况下，似乎忽略终点效果可能会损害计划的有效性。

为了避免解决这些复杂性，我们认为只有当计划中所有行动的起始和结束点都明确时，aPDDL2.1 计划才有效。在确认这一点后，我们继续确认计划中所有事件都是无互斥的。

Plan validation is decidable for domains including discretized and, under certain constraints, continuous durative actions because all activity is encapsulated with the durative actions explicitly identified by a plan. This makes the trace induced by a plan finite and hence checkable. We therefore observe that the validation problem for PDDL2.1 is decidable even when actions contain duration inequalities. This is because the work in determining how the duration inequalities should be solved has already been completed in the finished plan so validation of the plan can proceed by simulation of its execution, as is the case for PDDL plans. The problem is tractable for domains without continuous effects, but the introduction of continuous effects can, in principle, allow expression of domains with very complex functions describing numeric change (Howey & Long, 2002). Under the assumption that continuous effects are restricted to description in terms of simple linear or quadratic functions, without any interactions between concurrent continuous effects, plan validity is tractable. The cost in practice is increased however, since it may be necessary to solve polynomials in order to check invariants. Validation of plans containing more complex expressions of change is being explored.

Although plan validity checking is tractable, there is a subtlety that arises because of the need to represent plans syntactically and the difficulties involved in expressing numbers with arbitrary precision. In principle, all of the values that are required to describe valid plans are algebraic (assuming we constrain continuous effects as indicated above), and therefore finitely representable. In practice, expecting planners to handle numbers as algebraic expressions seems unnecessarily complicated and it is far more reasonable to assume that numbers will be represented as finite precision floating point values. Indeed, the syntax we have adopted for the expression of plans restricts planners to expressing times as finite precision floating point values. With this constraint, and because of limitations on the precision of floating point computations in implementations of plan validation systems, it is necessary to take a more pragmatic view of the validation process and accept that numeric conditions will have to be evaluated to a certain tolerance. Otherwise, it can occur that there is no way to report a plan to the necessary degree of accuracy for it to have a valid interpretation under the semantics we defined in Section 8. In most cases, a plan that specifies time points to, for example, four significant digits, is a reasonable abstraction of the execution time activity that will be needed to control the flow system. No plan can specify time points absolutely precisely, so abstraction is forced upon the planner by the fact that it is working with models of the world and not the physical world itself. The problem, then, is one of the relationship between the theoretical semantics and the pragmatic concerns of automated validation.

In Figure 18 this relationship is depicted in terms of what kinds of plans can be automatically validated. The left side of the picture describes the theoretical semantics, with the arrow indicating the link between plans and their interpretation under the theoretical semantics. For example, it is possible to construct a domain and problem for which a plan that requires an action to happen at time $\sqrt{2}$ is a meaningful semantic object, but for which a plan that specifies that the action happen at time 1.41 is not a meaningful semantic object because 1.41 is not equal to $\sqrt{2}$. These two plans are distinct, and only one is correct (under the assumed constraints). The right side of the picture depicts the pragmatic validation of syntactic plan objects. The two control plans, though distinct in the semantics, can map to the same syntactic object if we assume that the validation is subject to a tolerance of 0.01.

对于包括离散化和在特定约束下连续持续行动的领域，计划验证是可判定的，因为所有活动都被封装在计划明确标识的持续行动中。这使得计划引起的跟踪是有限的，因此是可检查的。因此，我们观察到对于 PDDL2.1 的验证问题，即使行动包含持续时间不等式也是可判定的。这是因为确定持续时间不等式应如何解决的工作已经在完成的计划中完成，因此可以通过模拟其执行来验证计划，就像 PDDL 计划的情况一样。对于没有连续效应的领域，该问题是可处理的，但是引入连续效应原则上可以表达具有非常复杂函数描述数值变化的领域（Howey & Long, 2002）。在假设连续效应仅限于用简单的线性或二次函数描述，且没有并发连续效应之间的相互作用的情况下，计划有效性是可处理的。然而，在实践中成本会增加，因为可能需要求解多项式来检查不变式。正在探索包含更复杂变化表达式的计划验证。

尽管计划有效性检查是可行的，但由于需要以句法形式表示计划以及表达任意精度数字所涉及的困难，存在一个微妙之处。原则上，描述有效计划所需的所有值都是代数的（假设我们像上面那样将连续效应进行约束），因此是有限可表示的。在实践中，要求规划器以代数表达式处理数字似乎是不必要的复杂，而假设数字将表示为有限精度的浮点值则更为合理。实际上，我们为表达计划所采用的句法限制了规划器只能以有限精度的浮点值表达时间。有了这个约束，并且由于计划验证系统实现中浮点计算精度的限制，有必要对验证过程采取更务实的观点，并接受数字条件必须以一定的容差进行评估。否则，可能无法以必要的精度报告计划，使其在我们在第8节中定义的语义下具有有效的解释。在大多数情况下，指定时间点为例如四位有效数字的计划，是对控制流系统所需执行时间活动的一个合理抽象。没有计划可以绝对精确地指定时间点，因此规划器由于它处理的是世界的模型而不是物理世界本身，被迫进行抽象。那么，问题就是理论语义与自动化验证的务实问题之间的关系。

在图18中，这种关系以可以自动验证哪些计划为标准进行了描述。图片的左侧描述了理论语义，箭头指示了计划及其在理论语义下的解释之间的链接。例如，可以构建一个领域和问题，其中需要某个动作在时间$\sqrt{2}$发生的一个计划是一个有意义的语义对象，但对于指定该动作在时间1.41发生的计划则不是一个有意义的语义对象，因为1.41不等于 $\sqrt{2}$。这两个计划是不同的，并且只有一个（在假设的约束下）是正确的。图片的右侧描绘了句法计划对象的实用验证。虽然这两个控制计划在语义上是不同的，但如果我们假设验证受0.01的容差限制，它们可以映射到同一个句法对象。

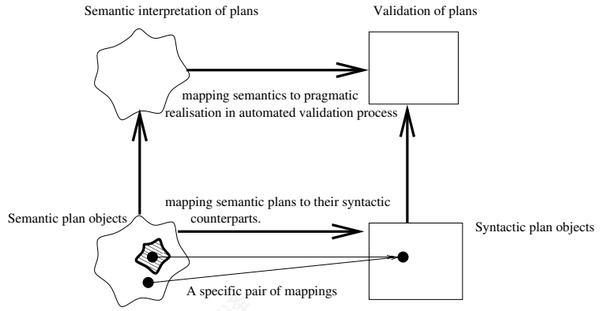

Figure 18: The pragmatic mapping between semantics of plans and their validation by automated computational processes. The shaded area contains plans that cannot be interpreted within the theoretical semantics. It can be seen that a plan in this collection is indistinguishable from a meaningful plan when mapped to the syntactic side of the picture.

These plans both map to the syntactic object in which 1.41 approximates the value $\sqrt{2}$. This syntactic plan can be validated using the pragmatic validation processes necessary for automatic validation of describable syntactic plans, which will check for validity subject to the tolerance of 0.01. The pragmatic constraints on the representations of plans, the expectations about representations of numeric values in planners and validators and their consequences are all reasonable assumptions given that the models against which we check validity are, in any case, abstractions, at some non-zero tolerance, of the world. In practice, it is a problem to accept plans at specified tolerance levels only in pathological cases, while arithmetic precision in computer representations of floats has an immediate and negative impact if one tries to take the stronger line that plans should only be accepted if they are strictly valid according to the formally precise evaluation of expressions.

Finally, there is an interesting philosophical issue that arises and is discussed by Henzinger and his co-authors (1997, 2000). It is, in fact, not possible to achieve exact precision in the measurement of time or other continuous numeric quantities. Henzinger *et al.* have considered this problem through the development of *robust automata*. Robust automata only accept a trace if there exists a *tube* of traces within a distance $\epsilon > 0$ of the original trace, all of which are acceptable by the original acceptance criteria. These are called *fuzzy tubes* indicating that time is fuzzily, rather than precisely, detectable. This idea offers a path to a formal semantics that is closer to defining plans that are robust to the imprecision in an executive's ability to measure time. Unfortunately, checking fuzzy tubes is intractable. We currently compromise by adopting an ϵ value, used as the tolerance in checking that numeric values fulfil numeric constraints during plan execution, to also represent the minimum separation of conflicting end points within plans. This is consistent with the idea that if the planner assumes that an executive is willing to abstract to the indicated tolerance level in the checking of preconditions for actions then it is unreasonable to suppose that a plan can make use of finer grained measurements in determining when actions should be

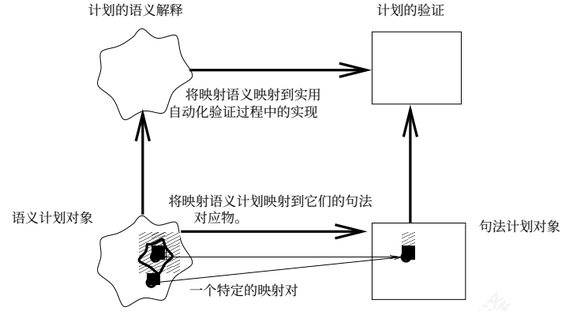

图18：计划语义与其通过自动化计算过程验证之间的实用映射。阴影区域包含在理论语义中无法解释的计划。可以看出，当映射到图片的句法一侧时，该集合中的一个计划与一个有意义的计划无法区分。

这两个计划都映射到句法对象，其中1.41近似于值$\sqrt{2}$。这个句法计划可以使用自动验证可描述句法计划所必需的实用验证过程进行验证，该过程将根据0.01的容差检查其有效性。对计划表示的实用约束、规划器和验证器中数值表示的期望及其后果，在所检查有效性的模型无论如何都是世界在某些非零容差下的抽象的情况下，都是合理的假设。在实践中，仅在病态情况下接受指定容差级别的计划是一个问题，而浮点数在计算机表示中的算术精度如果采取更强硬的立场，即只有当计划根据表达式的形式精确评估严格有效时才接受它们，则会产生立即且负面的影响。

最后，存在一个有趣的哲学问题，由Hen- zinger及其合著者（1997年，2000年）提出并讨论。事实上，在时间或其他连续数值量的测量中，无法实现精确的精度。Henzinger等人通过开发鲁棒自动机来考虑这个问题。鲁棒自动机仅接受一个轨迹，如果在该轨迹附近存在一个长度为 $\epsilon > 0$ 的轨迹管，并且这些轨迹都满足原始的接受标准。这些被称为模糊管，表明时间模糊地而不是精确地可检测。这个想法为一种更接近定义对执行者测量时间能力中的不精确性具有鲁棒性的计划的正式语义提供了途径。不幸的是，检查模糊管是不可行的。我们目前通过采用一个 ϵ 值来妥协，该值用作在计划执行期间检查数值是否满足数值约束的容差，以表示计划中冲突端点之间的最小分离。这与这样一种观点一致，即如果规划者假设执行者愿意在检查动作的前置条件时抽象到指示的容差级别，那么假设计划可以利用更细粒度的测量来确定动作应该

applied. At the moment the value of ϵ is set in the validation process, and only communicated informally to planner-engineers, but it might be better to allow a domain designer to define an ϵ appropriate for use in the particular domain. There remain several issues concerning the correct management of the buffers during validation (particularly the usual problem concerning the transitivity of "fuzzy closeness") which are important issues for temporal reasoning as a whole and are not restricted to the planning context. We do not yet have solutions to all of these problems.

11. Related Work: Representing and Reasoning about Time

Representation of, and reasoning with, statements about time and the temporal extent of propositions has long been a subject of research in AI including planning research (Allen, 1984; McDermott, 1982; Sandewall, 1994; Kowalski & Sergot, 1986; Laborie & Ghallab, 1995; Muscettola, 1994; Bacchus & Kabanza, 2000). Important issues raised during the extension of PDDL to handle temporal features have, of course, already been examined by other researchers, for example in Shanahan's work (1990) on continuous change within the event calculus, in Shoham's work (1985) and Reichgelt's work (1989) on temporal reasoning and work on non-reified temporal systems (Bacchus, Tenenberg, & Koomen, 1991). Vila (1994) provides an excellent survey of work in temporal reasoning in AI. In this section we briefly review some of the central issues that have been addressed, and their treatment in the literature, and set PDDL2.1 in the context of research in temporal logics.

Several researchers in temporal logics have considered the problems of reasoning about concurrency, continuous change and temporal extent. These works have focussed on the problem of reasoning about change when the world is described using arbitrary logical formulae, and most have been concerned with making meta-level statements (such as that effect cannot precede cause). The need to handle complex logical formulae makes the frame problem difficult to resolve, and an approach based on circumscription (McCarthy, 1980) and default reasoning (Reiter, 1980) is typical. The STRIPS assumption provides a simple solution to the frame problem when states are described using atomic formulae. The classical planning assumption is that states can be described atomically but this is not a general view of the modelling of change. Although simplifying, this assumption is surprisingly expressive. The bench mark domains introduced in the third International Planning Competition suggest that atomic modelling is powerful enough to capture some complex domains which closely approximate real problems. The temporal reasoning issues we confront are not simplified as a consequence of having made a simplifying assumption about how states are updated. We remain concerned with the major issues of temporal reasoning: concurrency, continuous change and temporal extent.

In the development of PDDL2.1 we made a basic decision to consider the end points of durative actions as instantaneous state transitions. This allows us to concentrate on the truth of propositions at points instead of over intervals. The decision to consider actions in this way is similar to that made by many temporal reasoning researchers (Shanahan, 1990; McCarthy & Hayes, 1969; McDermott, 1982). In the context of PDDL2.1 the approach has the advantage of smoothly integrating with the classical planning view of actions as state transitions. Nevertheless, Allen has shown that a temporal ontology based on intervals can be a basis for planning (Allen, 1984, 1991) and several planning systems have been

应用是不合理的。目前， ϵ 的值在验证过程中设置，并且仅非正式地传达给规划器工程师，但最好允许领域设计者定义一个适用于特定领域的 ϵ 。在验证期间正确管理缓冲区仍然存在几个问题（尤其是关于“模糊接近”传递性的通常问题），这些问题对时间推理整体都很重要，并且不仅限于规划上下文。我们还没有解决所有这些问题。

11. 相关工作：表示和推理时间

对时间以及命题的时间范围的陈述的表示和推理，长期以来一直是人工智能研究（包括规划研究）的课题（Allen, 1984; McDermott, 1982; Sandewall, 1994; Kowalski & Sergot, 1986; Laborie & Ghallab, 1995; Muscettola, 1994; Bacchus & Kabanza, 2000）。在将 PDDL 扩展以处理时间特征时提出的重要问题，当然已经由其他研究人员进行了研究，例如在 Shanahan（1990）关于事件演算中连续变化的工作中，在 Shoham（1985）和 Reichgelt（1989）关于时间推理以及非实体化时间系统（ Bacchus, Tenenberg, & Koomen, 1991）的工作中。Vila(1994) 对人工智能中的时间推理工作提供了优秀的综述。在本节中，我们简要回顾了已经解决的一些核心问题及其在文献中的处理方式，并将 PDDL2.1 放在时间逻辑研究的背景下。

时序逻辑中的许多研究人员已经考虑了关于并发、连续变化和时序范围的问题。这些工作主要集中在当世界被任意逻辑公式描述时，如何进行变化推理的问题，并且大多数都关注于做出元级声明（例如，效果不能先于原因）。处理复杂的逻辑公式使得框架问题难以解决，基于限制（McCarthy，1980）和默认推理（Reiter，1980）的方法是典型的。当状态使用原子公式描述时， STRIPS 假设为框架问题提供了一个简单的解决方案。经典规划假设是状态可以被原子地描述，但这并不是对变化建模的普遍观点。尽管这种假设简化了问题，但它却出奇地具有表达能力。第三届国际规划竞赛中引入的基准域表明，原子建模足够强大，能够捕捉一些与实际问题紧密近似的一些复杂域。我们所面临的时间推理问题并没有因为我们对状态更新方式做出了简化假设而得到简化。我们仍然关注时间推理的主要问题：并发、连续变化和时序范围。

在 PDDL2.1 的开发中，我们做出了一个基本决定，将持续动作的端点视为瞬时状态转换。这使我们能够专注于命题在点上的真值，而不是在区间上。这种考虑动作的方式与许多时间推理研究人员所做的决定相似（Shanahan，1990；McCarthy & Hayes，1969；McDermott，1982）。在 PDDL2.1 的背景下，这种方法的优势在于能够与经典规划中动作被视为状态转换的观点平滑集成。然而，Allen已经表明，基于区间的时态本体可以作为规划的基础（Allen，1984，1991），并且几个规划系统都受到了区间方法的强烈影响（Muscettola，1994；Rabideau，Knight，Chien，Fukunaga，& Govindjee，1999）。Allen后来放弃了最初认为不需要瞬时的立场，引入了“时刻”的概念（Hayes & Allen，1987），这是一个试图调和 “ 没有任何是瞬时的 ” （因此只应该有区间）的立场以及观察到的离散值变量（如命题变量）的变化显然无法避免在瞬时发生变化的观点。这种观点与我们建模连续持续动作时所采取的方法一致，并且与变化由离散和连续两个方面组成的观点一致（Henzinger，1996）。

strongly influenced by the intervals approach (Muscettola, 1994; Rabideau, Knight, Chien, Fukunaga, & Govindjee, 1999). Allen later moved away from his initial position that instants are not required, introducing the notion of *moments* (Hayes & Allen, 1987), which are a concept that attempts to reconcile the stance that nothing is instantaneous (so there should only be intervals) and the observation that changes in values of discrete-valued variables, such as propositional variables, apparently cannot avoid changing at instants. This view is consistent with the approach we take in the modelling of continuous durative actions, and with the view of change as consisting of both discrete and continuous aspects (Henzinger, 1996).

In the remainder of this section we compare the PDDL extensions that we propose with previous work in temporal reasoning by considering the three central issues identified above. Our objective is not to claim that our extensions improve on previous work, but instead to demonstrate that the implementation of solutions to these three problems within the PDDL framework makes their exploitation directly accessible to planning in a way that they are not when embedded within a logic and accompanying proof theory.

11.1 Continuous change

Several temporal reasoning frameworks began with consideration of discrete change and, later, were extended to handle continuous change. For example, Shanahan (1990) extended the event calculus of Kowalski and Sergot (1986) to enable the modelling of continuous change. This process of extension mirrors the situation faced in extending PDDL, where a system modelling discrete change already existed. It is, therefore, interesting to compare the use of PDDL2.1 with the use of systems such as the extended event calculus.

In his sink-filling example Shanahan (1990) discusses the issues of termination of events (self-termination and termination by other events), identification of the level of water in the sink during the filling process and the effect on the rate of change in the level of water in a sink when it is being filled from two sources simultaneously. The behaviour of the filling process and its effects on the state of the sink over time are modelled as axioms which would allow an inference engine to predict the state of the sink at points during the execution of the process.

PDDL2.1 allows the representation of the complex interactions that arise when a sink is filled from multiple independently controlled water sources by means of concurrent durative actions with continuous effects that encapsulate the initiation of the filling process, from a single water source, the change in the level of water in the sink and the termination of the process when the water source is turned off. This model is robust, since it easily accommodates multiple water sources, simply modifying the rate of flow appropriately by commutative updates. Since the actions have additive effects and the model provides the rate at which water enters the tank from a source, it is possible to compute the level of water in the sink at any point in the filling interval at which a concurrent action might consult the level. In contrast to Shanahan's extension to the event calculus, this approach does not require that the filling process be (at least from the point of view of the logical axiomatisation) terminated and restarted at a new rate when a water source is opened or closed, since the process simply remains active throughout. The change in rate of filling is

strongly influenced by the intervals approach (Muscettola，1994；Rabideau，Knight，Chien，Fukunaga，& Govindjee，1999)。Allen后来放弃了最初认为不需要瞬时的立场，引入了“时刻”的概念（Hayes & Allen，1987），这是一个试图调和“没有任何是瞬时的”（因此只应该有区间）的立场以及观察到的离散值变量（如命题变量）的变化显然无法避免在瞬时发生变化的观点。这种观点与我们建模连续持续动作时所采取的方法一致，并且与变化由离散和连续两个方面组成的观点一致（Henzinger，1996）。

在本节的其余部分，我们将我们提出的 PDDL 扩展与先前在时序推理方面的工作进行比较，方法是考虑上述三个核心问题。我们的目标不是声称我们的扩展优于先前的工作，而是证明在 PDDL框架内解决这三个问题的解决方案，使它们能够直接用于规划，而在逻辑和伴随的证明理论中嵌入时则无法做到这一点。

11.1 连续变化

几个时序推理框架最初考虑的是离散变化，后来扩展以处理连续变化。例如，Shanahan (1990) 扩展了 Kowalski 和 Sergot (1986) 的事件演算，以实现连续变化的建模。这个过程与扩展 PDDL时面临的情况类似，其中已经存在一个模拟离散变化的系统。因此，将 PDDL2.1 的使用与扩展的事件演算等系统的使用进行比较是很有趣的。

在他的水槽注水示例中，Shanahan (1990) 讨论了事件终止的问题（自我终止和由其他事件终止）、在注水过程中识别水槽水位以及当水槽同时从两个水源注水时水位变化速率的影响。注水过程的行为及其对水槽随时间状态的影响被建模为公理，这些公理将允许推理引擎预测水槽在执行过程中特定点的状态。

PDDL2.1 允许表示当从多个独立控制的水源通过并发持续作用填充一个容器时产生的复杂交互，这些持续作用封装了填充过程的启动，从一个单一的水源开始，容器的水位变化，以及当水源关闭时过程的终止。该模型是稳健的，因为它很容易适应多个水源，只需通过交换律更新适当修改流量即可。由于这些作用具有加性效果，并且模型提供了从水源进入水箱的速率，因此可以在填充间隔中的任何一点计算水位，这时并发作用可能会查询水位。与Shanahan对事件演算的扩展相比，这种方法不需要在打开或关闭水源时（至少从逻辑公理化的角度来看）终止并重新以新速率启动填充过程，因为该过程简单地保持活动状态。填充速率的变化

then reflected in a piecewise-linear profile for the depth of water in the sink, just as it is in Shanahan's model.

It is not possible to model the multiple water sources situation if the filling process is completely encapsulated within a discretized durative action. In the discretized action the true level of water is not accessible during the filling process but only at its end or its start. Step-function behaviour only coarsely approximates true behaviour, so the consequence is that complex interactions cannot be properly modelled.

One of the important consequences of continuous behaviour is the triggering of events. In Shanahan's extensions this is achieved through the axiomatisation of causal relationships — events are not distinguished syntactically from actions, but only by the fact that their happening is axiomatically the consequence of certain conditions. In PDDL2.1 some events (such as the flooding of the sink if the filling continues after its capacity has been reached) can be modelled by using a combination of conditional effects and duration inequalities. However, not all events can be modelled in this way, since it is not always possible to predict when spontaneous events will occur. PDDL2.1 could be extended to allow the expression of causal axioms, but an alternative approach is to modify the language to enable the representation of events within the action-oriented tradition. This can be achieved by breaking up continuous durative actions into their instantaneous start and end points and the processes they encapsulate. This would enable the execution of a process to be initiated by a start action and ended by an instantaneous state transition that is either an action under the control of the planner or an *event*. A simple extension to the language is needed to distinguish actions from events and to prevent the planner from deliberately selecting an event. We refer to this approach as the *start-process-stop* model, and we have extended PDDL2.1 to support it (Fox & Long, 2002). The resulting language, PDDL+, is more difficult to plan with than PDDL2.1, and there are still open questions, concerning the complexity of the plan validation problem for this language, which remain topics for future work.

11.2 Concurrency

The opportunity for concurrent activities complicates several aspects of temporal reasoning. Firstly, it is necessary to account for which actions can be concurrent and secondly it is necessary to describe how concurrent activities interact in their effects on the world.

In most formalisms the first of these points is achieved by relying on the underlying logic to deliver an inconsistency when an attempt is made to apply two incompatible actions simultaneously. For example, the axioms of the event calculus will yield the simultaneous truth and falsity of a fluent if incompatible actions are applied simultaneously and consequently yield an inconsistency. Unfortunately, recognising inconsistency is, in general, undecidable, for a sufficiently expressive language. In PDDL2.1 we adopt a solution that exploits the restricted form of the action-centred formalism, defining the circumstances in which two actions could lead to inconsistency and rejecting the simultaneous application of such actions. We favour a conservative restriction on compatibility of actions (the *no moving targets* rule), in order to support efficient determination of incompatibility, rather than a more permissive but elusive ruling. An alternative approach, adopted by Bacchus in TLplan (2001), for example, is to allow multiple actions to occur at the same instant, but nevertheless to be executed in sequence. We find this solution counter-intuitive and, more

然后在容器的深度随时间变化的分段线性曲线中反映出来，就像在Shanahan的模型中一样。

如果填充过程完全封装在一个离散的持续动作中，则无法对多种水源情况建模。在离散动作中，水的真实水平在填充过程中是不可访问的，只有在开始或结束时才能访问。阶跃行为只能粗略地近似真实行为，因此结果是复杂的交互无法被正确建模。

连续行为的一个重要后果是事件的触发。在Shanahan的扩展中，这是通过因果关系的公理化来实现的——事件在句法上与动作没有区别，但只有在其发生是公理上某些条件的后果这一事实上才被区分。在 PDDL2.1 某些事件（例如，如果填充在达到容量后继续，则水槽的洪水）可以通过使用条件效应和持续时间不等式的组合来建模。然而，并非所有事件都能以这种方式建模，因为无法总是预测自发事件何时发生。 PDDL2.1 可以扩展以允许因果公理的表达，但另一种方法是修改语言以在面向动作的传统中表示事件。这可以通过将连续的持续动作分解为其瞬时开始和结束点以及它们封装的过程来实现。这将允许通过一个开始动作启动一个过程，并通过一个瞬时状态转换结束，该转换要么是规划者控制下的动作，要么是事件。需要对该语言进行简单的扩展，以区分动作和事件，并防止规划者故意选择一个事件。我们将这种方法称为开始-过程-停止模型，并且我们已经扩展了 PDDL2.1 来支持它（Fox & Long, 2002）。生成的语言， PDDL+，比 PDDL2.1更难规划，并且仍然存在关于该语言计划验证问题复杂性的未解决的问题，这些问题仍然是未来工作的主题。

11.2 并发

并发活动的机会使时间推理的几个方面变得复杂。首先，有必要考虑哪些动作可以并发，其次，有必要描述并发活动如何在其对世界的影响中相互作用。

在大多数形式化系统中，这一点是通过依赖底层逻辑在尝试同时应用两个不兼容的操作时产生不一致性来实现的。例如，事件演算的公理将导致一个流同时为真和假，如果同时应用了不兼容的操作并因此产生不一致性。不幸的是，识别不一致性通常是不可判定的，对于足够表达力的语言而言。在 PDDL2.1 中，我们采用了一种利用以操作为中心的形式主义的有限形式的解决方案，定义了两个操作可能导致不一致的情况，并拒绝同时应用这些操作。我们倾向于对操作兼容性进行保守的限制（即“无移动目标”规则），以支持高效的不兼容性确定，而不是更宽松但难以捉摸的规则。Bacchus在TLplan（2001）中采用的另一种方法，例如，是允许多个操作在同一瞬间发生，但仍然按顺序执行。我们发现这种解决方案有违直觉，更重要的是，我们认为不可能将此类计划用作执行指令——没有执行者能够同时执行操作并按指定顺序执行。我们的观点是，如果执行顺序很重要，那么执行者必须确保操作按顺序执行，并且只能在其测量时间和对时间流逝做出反应的能力限制内做到这一点。

importantly, consider that it would be impossible to use a plan of this sort as an instruction to an executive — no executive could be equipped to execute actions simultaneously and yet in a specified order. Our view is that if the order of execution matters then the executive must ensure that the actions are sequenced and can only do so within the limitations of its capability to measure time and react to its passing.

Shanahan (1999) discusses Gelfond's (1991) example of the soup bowl in which the problem concerns raising a soup bowl without spilling the soup. Two actions, lift left and lift right, can be applied to the bowl. If either is applied on its own the soup will spill, but, it is argued, if they are applied simultaneously then the bowl is raised from the table and no soup spills. Shanahan considers this example within the event calculus, where he uses an explicit assertion of the interaction between the lift left and lift right actions to ensure that the spillage effect is cancelled when the pair is executed together. The assumption is that the two actions can be executed at precisely the same moment and that the reasoner can rely on the successful simultaneity in order to exploit the effect.

In PDDL2.1 we take the view that precise simultaneity is outside the control of any physical executive. A plan is interpreted as an instruction to some executive system and we hold that no executive system is capable of measuring time and controlling its activity at arbitrarily fine degrees of accuracy. In particular, it is not possible for an executive to ensure that two actions that must be independently initiated are executed simultaneously. If a plan were to rely on such precision in measurement then, we claim, it could not be executed with any reliable expectation of success and should not, therefore, be considered a valid plan.

PDDL2.1 supports the modelling of the soup bowl situation in the following way. Two durative actions, *lift left* and *lift right*, both independently initiate tilting intervals which, when complete, will result in spillage of the soup if their effects have not been counteracted. Provided that the two lift actions start within an appropriate tolerance of one another the tilting will be corrected and the spillage avoided without the need to model cancellation of effects. We argue that an executive can execute the two actions to within a fine but non-zero tolerance of one another, and can therefore successfully lift the bowl. The event calculus model presented by Shanahan insists on precise synchronization of the two actions, incorrectly allowing it to be inferred that the soup will be spilled even if the time that elapses between the two lifts is actually small enough to allow for correction of the tilting of the bowl. Worse, Shanahan's axioms would allow lack of precise synchronization to be exploited to achieve spillage, using an amount of time smaller than that correctly describing the physical situation being modelled.

If one considers it unnecessary to model the precise interaction between the two lifts, one has the alternative in PDDL2.1 to abstract out the interaction and see the soup-bowl lifting action as a single discretized action that achieves the successful raising of the bowl.

11.3 Temporal extent

A common concern in temporal reasoning frameworks, discussed in detail by Vila and others (Vila, 1994; van Bentham, 1983), is the *divided instant problem*. This is the problem that is apparent when considering what happens at the moment of transition from, say, truth to falsity of a propositional variable. The question that must be addressed is whether

重要的是，我们认为不可能使用此类计划作为执行指令——没有执行者能够同时执行操作并按指定顺序执行。我们的观点是，如果执行顺序很重要，那么执行者必须确保操作按顺序执行，并且只能在其测量时间和对时间流逝做出反应的能力限制内做到这一点。

Shanahan (1999) 讨论了 Gelfond (1991) 的汤碗例子，其中问题在于如何抬起汤碗而不洒出汤。可以施加两个动作，即抬起左和抬起右，到碗里。如果单独施加任何一个动作，汤就会洒出来，但是，有人认为，如果它们同时施加，那么碗就会从桌子上抬起，而且不会洒出汤。Shanahan 将这个例子放在事件演算中考虑，他使用对抬起左和抬起右动作之间交互的明确断言，以确保当这一对动作一起执行时，溢出效果被取消。假设这两个动作可以在精确的同一时刻执行，并且推理器可以依赖成功的同步性来利用这种效果。

在 PDDL2.1 我们认为精确的同步性超出了任何物理执行者的控制范围。一个计划被解释为对某个执行系统的指令，我们坚持认为没有执行系统能够以任意精细的精度测量时间并控制其活动。特别是，执行者不可能确保必须独立启动的两个动作同时执行。如果一个计划依赖于这种测量精度，那么，我们声称，它不可能以任何可靠的成功预期来执行，因此不应被视为一个有效的计划。

PDDL2.1 支持以下方式对汤碗情况进行建模。两个持续动作，左举和右举，都独立地启动倾斜区间，当这些区间完成时，如果它们的效果没有被抵消，会导致汤溢出。如果这两个举升动作在适当的时间范围内开始，倾斜将被纠正，并且可以避免溢出，而无需建模效果的取消。我们认为，一个执行者可以将这两个动作执行到非常精细但非零的时间范围内，因此可以成功地举起碗。Shanahan提出的事件演算模型坚持两个动作的精确同步，错误地允许推断即使两次举升之间经过的时间实际上足够小以允许纠正碗的倾斜，汤也会被溢出。更糟的是，Shanahan的公理允许利用精确同步的缺乏来实现溢出，使用的时间量小于正确描述所建模的物理情况的时间量。

如果认为没有必要模拟两个电梯之间的精确交互，那么在 PDDL2.1 中，可以选择抽象出交互，并将碗举起的动作视为一个单一的离散化动作，该动作实现了碗的成功举起。

11.3 时间跨度

时序推理框架中一个常见的关注点，由Vila等人详细讨论过（Vila, 1994; van Bentham, 1983），是分裂瞬间问题。这个问题在考虑命题变量从真到假的转变时刻时变得明显。必须回答的问题是，在转变瞬间命题是真、假、未定义，还是不一致地既真又假。

the proposition is true, false, undefined or inconsistently both true and false at the instant of transition. Clearly the last of these possibilities is undesirable. The solution we adopt is a combination of the pragmatic and the philosophically principled. The pragmatic element is that we choose to model actions as instantaneous transitions with effects beginning at the instant of application. Thus, the actions mark the end-points of intervals of persistence of state which are closed on the left and open on the right. This ensures that the intervals nest together without inconsistency and the truth values of propositions are always defined. The same half-open-half-closed solution is adopted elsewhere. For example, Shanahan (1999) observes that a similar approach is used in the event calculus, although there the intervals are closed on the right. Although the two choices are effectively equivalent, we slightly prefer the closed-on-the-left choice since this allows the validation of a plan to conclude with the state at the point of execution of its final action, making the determination of the temporal span of the plan unambiguous.

From a philosophical point of view the truth value of the proposition at the instant of application of an action cannot be exploited by any other action, by virtue both of the no moving targets rule and our position, outlined above, that a valid plan cannot depend on precise synchronisation of actions. This forces actions that require a proposition as a precondition to sit at the open end of a half-open interval in which the proposition holds.

11.4 Planning with Time

In classical planning models, time is treated as relative. That is, the only temporal structuring in a plan, and in reasoning about a plan, is in the ordering between actions. This is most clearly emphasised by the issues that dominated planning research in the late 1980s and early 1990s, when classical planning was mainly characterised by the exploration of partial plan spaces, in planners such as TWEAK (Chapman, 1987), SNLP (McAllester & Rosenblitt, 1991) and UCPOP (Penberthy & Weld, 1992). Partial plans include a collection of actions representing the activity thus far determined to be part of a possible plan and a set of temporal constraints on those actions. The temporal constraints used in a partial plan are all of the form $A < B$ where A and B are time points corresponding to the application of actions.

Classical linear planners (Fikes & Nilsson, 1971; Russell & Norvig, 1995) rely on the simple fact that a total ordering on the points at which actions are applied can be trivially embedded into a time line. Again, the duration between actions is not considered. The role of time in planning becomes far more significant once metric time is introduced. With metric time it is possible to associate specific durations with actions, to set deadlines or windows of opportunity. The problems associated with relative time have still to be resolved in a metric time framework, but new problems are introduced. In particular, durations become explicit, so it is necessary to decide what the durations attach to: actions or states. Further, explicit temporal extents make it more important to confront the issue of concurrency in order to best exploit the measured temporal resources available to a planner.

In contrast to the simple ordering constraints required for relative time, metric time requires more powerful constraint management. Most metric time constraint handlers are built around the foundations laid by Dechter, Meiri and Pearl (1991). For example, IxTeT uses extensions of temporal constraint networks (Laborie & Ghallab, 1995). The language

命题在转变瞬间是真、假、未定义，还是不一致地既真又假。显然，最后这种可能性是不可取的。我们采用的解决方案是实用主义和哲学原则的结合。实用主义方面是，我们选择将动作建模为瞬时转变，其效果在应用瞬间开始。因此，动作标志着状态持续区间（这些区间在左侧闭合并右侧开）的端点。这确保了区间嵌套时不产生不一致，并且命题的真值始终被定义。同样的半开半闭解决方案在其他地方也被采用。例如，Shanahan (1999)观察到事件演算中使用了类似的方法，尽管在那里区间在右侧闭合。尽管这两种选择在效果上等效，我们略微倾向于左侧闭合的选择，因为这样可以允许计划验证以最终动作的执行点状态结束，从而使得计划的时序跨度确定无歧义。

从哲学角度来看，在某个动作应用瞬间的命题的真值无法被任何其他动作所利用，这既得益于不动目标规则，也得益于我们上述所阐述的立场：一个有效的计划不能依赖于动作的精确同步。这迫使那些以命题为前提条件的动作位于一个半开区间（命题成立）的开端。

11.4 基于时间的规划

在经典规划模型中，时间是作为相对的来处理的。也就是说，在计划中以及在关于计划的推理中，唯一的时间结构化体现在动作之间的顺序。这一点在主导了20世纪80年代末和90年代初规划研究的议题中最为明显，当时经典规划主要特征在于对部分计划空间的探索，例如 TWEAK (Chapman, 1987)， SNLP (McAllester & Rosenblitt, 1991)和 UCPOP (Penberthy & Weld, 1992)中的规划器。部分计划包括一组代表迄今为止确定为可能计划一部分的动作，以及一组对这些动作的时间约束。部分计划中使用的时间约束都是 $A < B$ 的形式，其中 A 和 B 是与动作应用对应的时间点。

经典的线性规划者（Fikes & Nilsson, 1971；Russell & Norvig, 1995）依赖于一个简单的事实：在动作应用点上的全序可以轻易地嵌入到时间轴中。同样，动作之间的持续时间不被考虑。一旦引入了度量时间，规划中时间的角色就变得更为重要。在度量时间下，可以将特定的持续时间与动作关联起来，设置截止日期或机会窗口。在度量时间框架下，与相对时间相关的问题仍然需要解决，但新的问题被引入。特别是，持续时间变得明确，因此需要决定持续时间依附于什么：动作还是状态。此外，明确的时间范围使得解决并发问题更为重要，以便最好地利用规划者可用的测量时间资源。

与相对时间所需的简单排序约束相比，度量时间需要更强大的约束管理。大多数度量时间约束处理器都是基于 Dechter、Meiri 和 Pearl（1991）奠定的基础构建的。例如，IxTeT 使用时间约束网络（Laborie & Ghallab，1995）的扩展。我

that IxTeT uses to represent planning domains is similar to PDDL2.1 as described in this paper, but more expressive because it allows access to time points within the interval of a durative action. This added expressive power is obtained at the cost of increased semantic complexity and, consequently, increased difficulty in the validation of plans. However, there are many similarities between the modelling of discretised durative actions in PDDL2.1 and in IxTeT, and similar modelling conventions are also found in the languages of Sapa (Do & Kambhampati, 2001) and Oplan (Drabble & Tate, 1994).

One of the earliest planners to consider the use of metric time was Deviser (Vere, 1983), which was developed from NONLIN (Tate, 1977). In Deviser, metric constraints on the times at which actions could be applied and deadlines for the achievements of goals were both expressible and the planner could construct plans respecting metric temporal constraints on the interactions between actions. Cesta and Oddi (1996) have explored various developments of temporal constraint network algorithms to achieve efficient implementation for planning and Galipienso and Sanchis (2002) consider extensions to manage disjunctive temporal constraints efficiently, which is a particularly valuable expressive element for plan construction as was observed above, since constraints preventing overlap of intervals translate into disjunctive constraints on time points. HSTS (Muscettola, 1994) also relies on a temporal constraint manager.

In systems that use continuous real-valued time it is possible to make use of linear constraint solvers to handle temporal constraints. In particular, constraints dictated by the relative placement of actions with durations on a timeline can be approached in this way (Long & Fox, 2003a). An alternative timeline that is often used is a discretised line based on integers. The advantage of this approach is that it is possible to advance time to a *next* value after considering activity at any given time point. The *next* modality can be interpreted in a continuous time framework by taking it to mean the state following the next logical change, regardless of the time at which this occurs (Bacchus & Kabanza, 1998). In planning problems in which no events can occur other than the actions dictated by the planner and no continuous change is modelled, plans are finite structures and therefore change can occur at only a finite number of time points during its execution. This makes it possible to embed the execution of the plan into the integer-valued discrete time line without any loss of expressiveness.

Various researchers have considered the problem of modelling continuous change. Pednault (1986) proposes explicit description of the functions that govern the continuous change of metric parameters, attached to actions that effect instantaneous change to initiate the processes. However, his approach is not easy to use in describing interacting continuous processes. For example, if water is filling a tank at a constant rate and then an additional water source is added to increase the rate of filling then the action initiating the second process must combine the effects of the two water sources. This means that the second action cannot be described simply in terms of its direct effect on the world — to increase the rate of flow into the tank — but with reference to the effects of other actions that have already affected the rate of change of the parameter. Shanahan (1990) also uses this approach, with the consequence that processes are modelled as stopping and then restarting with new trajectories as each interacting action is applied.

In Zeno (Penberthy & Weld, 1994), actions have effects that are described in terms of derivatives. This approach makes it easier to describe interacting processes, but complicates

xTeT 用于表示规划域的语言与本文中描述的 PDDL2.1 类似，但更具表现力，因为它允许访问持续动作区间内的时间点。这种额外的表现力是以增加语义复杂性和随之而来的计划验证难度为代价获得的。然而，在 PDDL2.1 和 IxTeT 中对离散持续动作的建模之间存在许多相似之处，Sapa（Do & Kambhampati，2001）和 Oplan（Drabble & Tate，1994）的语言中也发现了类似的建模约定。

最早考虑使用公制时间的规划器之一是Deviser（Vere，1983），它源自于 NONLIN（Tate，1977）。在Deviser中，对动作可以应用的时间以及目标实现的截止时间的公制约束都是可表达的，并且规划器可以构建尊重动作之间交互的公制时间约束的计划。Cesta和Oddi（1996）探索了各种时间约束网络算法的发展，以实现规划的高效实现，而Galipienso和Sanchis（2002）考虑了管理析取时间约束的高效扩展，如前所述，这对于计划构建是一个特别有价值的表达能力，因为防止区间重叠的约束转化为对时间点的析取约束。 HSTS （Muscettola，1994）也依赖于一个时间约束管理器。

在连续实值时间系统中，可以使用线性约束求解器来处理时间约束。特别是，可以通过这种方式处理由动作在时间线上的相对位置所规定的约束（Long & Fox，2003a）。另一种常用的时间线是基于整数的离散线。这种方法的优点是，在考虑任何给定时间点的活动后，都可以将时间推进到下一个值。通过将其解释为下一个逻辑变化后的状态，无论这种变化何时发生，都可以在连续时间框架中解释这种下一个模态（Bacchus & Kabanza，1998）。在规划问题中，如果没有事件可以发生，除了由规划器规定的动作外，并且没有对连续变化的建模，则计划是有限结构，因此在执行期间只能在有限数量的时间点发生变化。这使得可以将计划的执行嵌入到整数值的离散时间线中，而不会损失表达能力。

许多研究人员已经考虑了建模连续变化的问题。Ped- nault (1986) 提出了对控制度量参数连续变化的函数的明确描述，这些函数附加于导致瞬时变化的动作以启动过程。然而，他的方法在描述相互作用的连续过程时并不容易使用。例如，如果水以恒定速率填充一个水箱，然后又增加了一个额外的水源来提高填充速率，那么启动第二个过程的动作必须结合两个水源的影响。这意味着第二个动作不能仅通过其对世界的直接影响——即增加流入水箱的速率——来描述，而必须参考已经影响参数变化速率的其他动作的影响。
Shanahan (1990) 也使用了这种方法，其结果是，过程被建模为停止然后以新的轨迹重新启动，每当应用一个相互作用的动作时。

在 Zeno (Penberthy & Weld, 1994) 中，行为的效果用导数来描述。这种方法使得描述交互过程更加容易，但也复杂化了

the management of processes by making it necessary to solve differential equations. The complication has not deterred other authors from taking this approach: McDermott (2003) takes this approach in his process planner.

The introduction of continuous processes into the planning problem represents a considerable complication, even over a model that includes temporal features and supports concurrency. It is an area of active research and the community has not yet agreed on matters of representation, let alone semantics. There remain many open problems for the planning community to address, both in the development of languages and planning algorithms and also in in the development of plan verification tools that can embody a widely accepted semantics.

12. Conclusions

Recent developments in AI planning research have been leading the community closer to the application of planning technology to realistic problems. This has necessitated the development of a representation language capable of modelling domains with temporal and metric features. The approach we have taken towards the development of such a language is to extend McDermott's PDDL domain representation standard to support temporal and metric models.

The development of the PDDL sequence towards greater expressive power is important to the planning community because PDDL has provided a common foundation for a great deal of recent research effort. The problems involved in modelling the behaviour of domains with both discrete and continuous behaviours have been well explored in the temporal logic and model checking communities but there have been no widely adopted models within the planning community. Our work on PDDL2.1 provides a way of making the relevant developments in these communities accessible to planning. Furthermore, PDDL2.1 begins to bridge the gap between basic research and applications-oriented planning by providing the expressive power necessary to capture real problems.

PDDL2.1 has the expressive power to represent a class of deterministic mixed discrete-continuous domains as planning domains. The language introduces a form of durative action based on three connected parts: the initiation of an interval in which numeric change might occur and its explicit termination by means of an action that produces the state corresponding to the end of the durative interval. This form of action allows the modelling of both discrete and continuous behaviours — discretized change can be represented by means of step functions, whilst continuous change can be modelled using the $\#t$ variable. The language provides solutions to the critical issues of concurrency, continuous change and temporal extent. The semantics of the language is derived from the familiar state transition semantics of STRIPS, extended to interpret invariants holding over intervals in which continuous functions might also be active. Our semantics allows us to interpret more plans than we can efficiently validate. We describe the criteria that a plan must satisfy in order to be practically verifiable.

This paper has focussed primarily on a discussion of the numeric and discretised temporal features of PDDL2.1. However, the modelling capability of discretized durative actions is in some respects limited and it is important for the planning community to address the challenges presented by continuous change. Indeed, even using the continuous actions of

过程的管理，因为它需要解微分方程。这种复杂性并没有阻止其他作者采用这种方法：McDermott (2003) 在他的过程规划器中采用了这种方法。

将连续过程引入规划问题代表着相当大的复杂性，即使在包含时间特征并支持并发性的模型中也是如此。这是一个活跃的研究领域，社区尚未就表示形式达成一致，更不用说语义了。规划社区仍然有许多悬而未决的问题需要解决，无论是在语言和规划算法的开发中，还是在开发能够体现广泛接受的语义的计划验证工具中。

12. 结论

人工智能规划研究领域的最新发展正将社区引向将规划技术应用于现实问题的方向。这需要开发一种能够模拟具有时间和度量特征的领域的表示语言。我们为开发此类语言所采取的方法是扩展 McDermott 的 PDDL 领域表示标准以支持时间和度量模型。

开发 PDDL 序列以获得更大的表达能力对规划社区很重要，因为 PDDL 为大量最近的研究工作提供了一个共同的基础。在模拟具有离散和连续行为的领域行为方面所涉及的问题在时序逻辑和模型检验社区中得到了充分探索，但在规划社区中还没有广泛采用的模型。我们在 PDDL2.1 方面的工作为规划社区提供了使这些社区的相关发展可用的途径。此外， PDDL2.1 通过提供必要的表达能力来捕捉现实问题，开始弥合基础研究与面向应用的规划之间的差距。

PDDL2.1 具有表示一类确定性混合离散-连续域为规划域的表达能力。该语言引入了一种基于三个连接部分的持续动作形式：一个可能发生数值变化的区间启动及其通过一个产生对应持续区间结束状态的动作的明确终止。这种动作形式允许对离散和连续行为进行建模——离散变化可以通过阶跃函数表示，而连续变化可以使用 $\#t$ 变量进行建模。该语言为并发、连续变化和时间范围等关键问题提供了解决方案。该语言的语义基于熟悉的 STRIPS 状态转换语义扩展而来，扩展为解释在可能存在连续函数活跃的区间上保持的不变式。我们的语义允许我们解释比我们能高效验证的更多计划。我们描述了计划必须满足的标准，以便在实际中可以进行验证。

本文主要集中讨论了 PDDL2.1的数值和离散化时间特征。然而，离散化持续动作的建模能力在某些方面存在局限性，并且对于规划领域来说，解决连续变化带来的挑战至关重要。事实上，即使使用连续动作，

PDDL2.1 it is not possible to model episodes of change being terminated by spontaneous events in the world rather than by the deliberate choice of the planner. The future goals of the community should include addressing domains that require the continuous actions of PDDL2.1, then confronting the challenges of planning within more dynamic environments in which intervals of change can be terminated by the world as well as by the deliberate action of the planner. This will constitute an important step towards planning within dynamic and unpredictable environments.

Acknowledgements

We would like to thank the members of the committee for the third International Planning Competition. In particular, discussions with Drew McDermott, Fahiem Bacchus, David Smith and Hector Geffner in turns infuriated, intrigued and delighted us and contributed immeasurably to the strengths of this paper. Many others have offered comments and insights that have allowed us to develop the work we present here. We would like to thank Jörg Hoffmann, Malte Helmert, Antonio Garrido, Stefan Edelkamp, Nicola Muscettola, Mark Boddy, Keith Golden, Jeremy Frank, Ari Jónsson, Julie Porteous, Alex Coddington, Stephen Cresswell, Luke Murray, Keith Halsey and Richard Howey for the many helpful discussions we have shared.

PDDL2.1: Expressing Temporal Planning Domains

PDDL2.1 也无法模拟由世界中的自发事件而非规划者的有意选择而终止的变化事件。该领域未来的目标应包括解决需要PDDL2.1连续动作的领域，然后应对在更动态的环境中规划所面临的挑战，其中变化的时间间隔可以由世界或规划者的有意行动终止。这将构成在动态和不可预测环境中进行规划的重要一步。

致谢

我们感谢委员会的成员们参加了第三届国际规划竞赛。特别是，与Drew McDermott、Fahiem Bacchus、David Smith和Hector Geffner的讨论轮流让我们愤怒、着迷和高兴，并极大地增强了本文的强度。许多其他人提出了评论和见解，使我们能够发展我们在这里展示的工作。我们感谢J¨org Hoffmann、Malte Helmert、Antonio Garrido、Stefa n Edelkamp、Nicola Muscettola、Mark Boddy、Keith Golden、Jeremy Frank、Ari J´onsson、Julie Porteous、Alex Coddington、Stephen Cresswell、Luke Murray、Keith Halsey和Richard Howey，因为我们共同进行了许多有益的讨论。

113

Appendix A. BNF Specification of PDDL2.1

This appendix contains a complete BNF specification of the PDDL2.1 language. This is not a strict superset of PDDL1.x. For example, the use of local variables within action schemas has been left out of this specification. It is not a widely used part of the language and has not been used in any of the competition domains. The interpretation of local variables as proposed by McDermott is subtle, since it demands confirmation that a unique instantiation exists for each such variable. It is non-trivial to confirm that this is the case during plan validation for domains with significant expressive power and the fact that it has been largely ignored suggests that it is poorly understood. Other changes are discussed in the following sections.

A.1 Domains

Domain structures remain essentially as specified in PDDL1.x. The main alterations are to introduce a slightly modified syntax for numeric fluent expressions and to remove the syntax for hierarchical expansions. The latter is not necessarily abandoned, but it has not, to the best of our knowledge, been used in any publicly available planning systems or even domains. In the original PDDL specification, a distinction was drawn between *strict* PDDL and *non-strict* PDDL, where strict PDDL must follow the ordering of the fields specified below, while non-strict PDDL is not restricted in this way. In practice, there are relatively few fields that it is intuitive to accept in arbitrary orders — it is natural to expect declarations to precede use of symbols and for preconditions to precede effects. However, declarations of constants, predicates and function symbols are not naturally ordered, so in the current definition of PDDL the ordering of all fields must follow the specification below, with the exception of these three fields which are legal in any order with respect to one another, although the group must follow types (if there are any) and precede action specifications.

```
<domain>            ::= (define (domain <name>)
                            [<require-def>]
                            [<types-def>]:typing
                            [<constants-def>]
                            [<predicates-def>]
                            [<functions-def>]:fluents
                            <structure-def>*)
<require-def>       ::= (:requirements <require-key>+)
<require-key>       ::= See Section A.5
<types-def>         ::= (:types <typed list (name)>)
<constants-def>     ::= (:constants <typed list (name)>)
<predicates-def>    ::= (:predicates <atomic formula skeleton>+)
<atomic formula skeleton>
                    ::= (<predicate> <typed list (variable)>)
<predicate>         ::= <name>
<variable>          ::= ?<name>
<atomic function skeleton>
                    ::= (<function-symbol> <typed list (variable)>)
<function-symbol>   ::= <name>
<functions-def>     ::=:fluents (:functions <function typed list
                                  (atomic function skeleton)>)
<structure-def>     ::= <action-def>
<structure-def>     ::=:durative-actions <durative-action-def>
```


附录 A. PDDL2.1的BNF规范

本附录包含 PDDL2.1 语言的完整BNF规范。这不是 PDDL1.x的严格超集。例如，在动作模式中使用局部变量的功能已被从本规范中省略。这不是语言中广泛使用的一部分，并且没有在任何竞赛领域中使用过。McDermott提出的局部变量的解释很微妙，因为它要求确认每个此类变量都存在唯一的实例。对于具有显著表达能力的领域，在计划验证期间确认这一点并不简单，而且它基本上被忽视的事实表明人们对其理解不佳。其他更改将在以下部分中讨论。

A.1 域

域结构基本上保持如 PDDL1.x 中所述。主要更改是引入略微修改的数值流畅表达式语法，并移除层级扩展的语法。后者并非必然放弃，但据我们所知，它在任何公开可用的规划系统或领域中都未被使用。在原始 PDDL 规范中，区分了严格的 PDDL 和非严格的 PDDL，其中严格的 PDDL 必须遵循下面指定的字段顺序，而非严格的 PDDL 在此方面不受限制。实际上，直观上接受任意顺序的字段相对较少——声明自然地期望先于符号的使用，前置条件自然地期望先于效果。然而，常量、谓词和函数符号的声明在自然顺序上并不明确，因此在当前的 PDDL 定义中，所有字段的顺序必须遵循下面的规范，除了这三个字段可以以任何顺序相互排列，尽管该组必须遵循类型（如果有）并先于动作规范。

```
<域>                ::= (定义 (域 <名>)
                            [<要求定义>]
                            [<类型定义>]:typing
                            [<常量定义>]
                            [<谓词定义>]
                            [<函数定义>]:fluents
                            <结构定义>*)
<要求定义>          ::= (:要求 <要求键>+)
<require-key>       ::= 参见第A.5节
<types-def>         ::=(:types <类型列表 (名称)>)
<constants-def>     ::= (:constants <类型列表 (名称)>)
<谓词定义>          ::= (:谓词 <原子公式骨架>+)
<原子公式骨架>
                    ::=(<谓词> <带类型列表（变量）>)
<谓词>              ::= <名称>
<变量>              ::= ?<名称>
<原子函数骨架>
                    ::= (<函数符号> <类型列表（变量）>)
<函数符号>          ::= <名称>
<函数定义>          ::=:fluents (:函数 <函数类型列表
                                  (原子函数骨架)>)
<结构定义>          ::= <动作定义>
<结构定义>          ::=:durative-actions <持续动作定义>
```

A slight modification has been made to the type syntax – it is no longer possible to nest `either` expressions (a possibility that was never exploited, but complicates parsing). Numbers are no longer considered to be an implicit type – the extension to numbers is now handled only through functional expressions. This ensures that there are only finitely many ground action instances. A desirable consequence is that action selection choice points need never include choice over arbitrary numeric ranges. The use of finite ranges of integers for specifying actions is useful (see Mystery or FreeCell for example) and an extension of the standard syntax to allow for a more convenient representation of these cases could be useful. The syntax of function declarations allows functions to be declared with types. At present the syntax is restricted to number types, since we do not have a semantics for other functions, but the syntax offers scope for possible extension. Where types are not given for the function results they are assumed to be numbers.

```
<typed list (x)> ::= x*
<typed list (x)> ::=:typing x+- <type> <typed list(x)>
<primitive-type> ::= <name>
<type>           ::= (either <primitive-type>+)
<type>           ::= <primitive-type>

<function typed list (x)> ::= x*
<function typed list (x)> ::=:typing x+- <function type>
                                         <function typed list(x)>
<function type>           ::= number
```

A.2 Actions

The BNF for an action definition is given below. Again, this has been simplified by removing generally unused constructs (mainly hierarchical expansions). It should be emphasised that this removal is not intended to be a permanent exclusion — hierarchical expansion syntax has proved a difficult element of the language both to agree on and to exploit. As the other levels of the language stabilise we hope to return to this layer and redevelop it.

```
<action-def>      ::= (:action <action-symbol>
                          :parameters ( <typed list (variable)> )
                          <action-def body>)
<action-symbol>   ::= <name>
<action-def body> ::= [:precondition <GD>]
                      [:effect <effect>]
```

Goal descriptions have been extended to include fluent expressions.

```
<GD>                  ::= ()
<GD>                  ::= <atomic formula(term)>
<GD>                  ::=:negative-preconditions <literal(term)>
<GD>                  ::= (and <GD>*)
```


类型语法已进行微小修改——现在不再允许嵌套表达式（虽然这种可能性从未被利用，但会使解析复杂化）。数字不再被视为隐式类型——对数字的扩展现在仅通过函数表达式处理。这确保了只有有限数量的地面动作实例。一个可取的后果是，动作选择选择点无需包括对任意数值范围的选择。使用整数有限范围来指定动作是很有用的（例如 Mystery 或 FreeCell ），并且可以将标准语法扩展以更方便地表示这些情况可能是有用的。函数声明的语法允许声明具有类型的函数。目前语法仅限于数字类型，因为我们没有其他函数的语义，但语法为可能的扩展提供了空间。当函数结果未给出类型时，它们被假定为数字。

```
<类型列表 (x)>::= x*
<类型列表 (x)> ::=:typing x+- <类型> <类型列表(x)>
<原始类型>::= <名称>
<类型>              ::=(要么 <原始类型>+)
<类型>              ::= <基本类型>

<函数类型列表 (x)>::= x*
<函数类型列表 (x)> ::=:typing x+- <函数类型>
                                  <函数类型列表(x)>
<函数类型>                  ::= 数字
```

A.2 操作

动作定义的 BNF 语法如下。同样，这是通过移除通常不使用的结构（主要是层次扩展）而简化的。应该强调的是，这种移除并不是永久性的排除——层次扩展语法已被证明是语言中难以商定和利用的元素。随着语言其他层次的稳定，我们希望回归这一层并重新开发它。

```
<action-def>          ::=(:action <action-symbol>)
                                  :参数 ( <类型列表（变量）> )
                                  <action-def body>)
<动作符号>            ::= <名称>
<动作定义体> ::= [:前提 <GD>]
                      [:效果 <效果>]
```

目标描述已扩展，以包含流畅的表达。

```
<GD>                  ::= ()
<GD>                  ::= <原子公式（项）>
<GD>                  ::=:negative-preconditions <literal(term)>
<GD>                  ::= (和 <GD>*)
```

```
<GD>                  ::=:disjunctive-preconditions (or <GD>*)
<GD>                  ::=:disjunctive-preconditions (not <GD>)
<GD>                  ::=:disjunctive-preconditions (imply <GD> <GD>)
<GD>                  ::=:existential-preconditions
                         (exists (<typed list(variable)>*) <GD> )
<GD>                  ::=:universal-preconditions
                         (forall (<typed list(variable)>*) <GD> )
<GD>                  ::=:fluents <f-comp>
<f-comp>              ::= (<binary-comp> <f-exp> <f-exp>)
<literal(t)>          ::= <atomic formula(t)>
<literal(t)>          ::= (not <atomic formula(t)>)
<atomic formula(t)>   ::= (<predicate> t*)
<term>                ::= <name>
<term>                ::= <variable>
<f-exp>               ::= <number>
<f-exp>               ::= (<binary-op> <f-exp> <f-exp>)
<f-exp>               ::= (- <f-exp>)
<f-exp>               ::= <f-head>
<f-head>              ::= (<function-symbol> <term>*)
<f-head>              ::= <function-symbol>
<binary-op>           ::= +
<binary-op>           ::= -
<binary-op>           ::= *
<binary-op>           ::= /
<binary-comp>         ::= >
<binary-comp>         ::= <
<binary-comp>         ::= =
<binary-comp>         ::= >=
<binary-comp>         ::= <=
<number>              ::= Any numeric literal
                          (integers and floats of form n.n).
```

Effects have been extended to include functional expression updates. The syntax proposed here is a little different from the syntax proposed in the earlier version of PDDL. The syntax of conditional effects proposed by Fahiem Bacchus for AIPS 2000 has been adopted, in which the nesting of conditional effects is not supported. The assignment operators are prefix forms. Simple assignment is called `assign` (previously this was `change`) and operators corresponding to C update assignments, $+=$, $-=$, $*=$ and $/=$ are given the names `increase`, `decrease`, `scale-up` and `scale-down` respectively. The prefix form has been adopted in preference to an infix form in order to preserve consistency with the Lisp-like syntax and the non-C names to help the C and C++ programmers to remember that the operators are to be used in prefix form). We prefer `assign` to the original `change` because the introduction of `increase` and so on makes the nature of a *change* more ambiguous.

```
<effect>       ::= ()
<effect>       ::= (and <c-effect>*)
<effect>       ::= <c-effect>
<c-effect>     ::=:conditional-effects (forall (<variable>*) <effect>)
<c-effect>     ::=:conditional-effects (when <GD> <cond-effect>)
<c-effect>     ::= <p-effect>
<p-effect>     ::= (<assign-op> <f-head> <f-exp>)
<p-effect>     ::= (not <atomic formula(term)>)
<p-effect>     ::= <atomic formula(term)>
```



```
<GD>                  ::=:disjunctive-preconditions (or <GD>*)
<GD>                  ::=:disjunctive-preconditions (not <GD>)
<GD>                  ::=:disjunctive-preconditions (imply <GD> <GD>)
<GD>                  ::=:existential-preconditions
                         (exists (<类型列表(变量)>*) <GD> )
<GD>                  ::=:universal-preconditions
                         (∀<类型为列表的变量>*) <GD> )
<GD>                  ::=:fluents <f-comp>
<f-comp>              ::=(<二进制复合> <函数表达式> <函数表达式>)
<literal(t)>          ::= <原子公式(t)>
<literal(t)>          ::= (not <原子公式(t)>)
<原子公式(t)>          ::= (<谓词> t*)
<项>                  ::= <名称>
<项>                  ::= <变量>
<f-exp>               ::= <数字>
<f-exp>               ::= (<二进制运算符> <f-exp> <f-exp>)
<f-exp>               ::= (- <f-exp>)
<f-exp>               ::= <f-head>
<f-head>              ::= (<函数符号> <术语>*)
<f-head>              ::= <函数符号>
<二元运算符>           ::= +
<二元运算符>           ::= -
<二元运算符>           ::= *
<binary-op>           ::= /
<binary-comp>         ::= >
<binary-comp>         ::= <
<binary-comp>         ::= =
<binary-comp>         ::= >=
<binary-comp>         ::= <=
<数字>                ::=任何数字字面量
                          (整数和形式 n.n的浮点数)。
```

效果已扩展，以包含功能表达式更新。此处提出的语法与 PDDL 早期版本中提出的语法略有不同。采用的 Fahiem Bacchus 为 AIPS 2000 提出的条件效果语法中，不支持条件效果的嵌套。赋值运算符是前缀形式。简单赋值称为 assign（以前这是 change），与 C 更新赋值对应的运算符 $+=$, $-=$, $*=$ 和 $/=$ 分别命名为 increase、decrease、scale-up 和 scale-down。优先采用前缀形式而不是中缀形式，以保持与 Lisp 语法的一致性，并使用非 C 名称，以帮助 C 和 C++ 程序员记住运算符应以前缀形式使用）。我们更倾向于使用 assign 而不是原始的 change，因为引入 increase 等操作使变化的性质更加模糊。

```
<效果>          ::=()<效果>          ::=(和 <c效果>*)<效果>          ::= <c效果><
c效果> ::=:conditional-effects (对于(<变量>*) <效果>)<c效果> ::
=:conditional-effects (当 <GD> <条件效果>)<c效果>      ::= <p效果><p效果> ::=
(<赋值操作符> <f头> <f表达式>)<p效果> ::=(不是 <原子公式(项)>)<p效果> ::
= <原子公式(项)>
```

```
<p-effect>     ::=:fluents(<assign-op> <f-head> <f-exp>)
<cond-effect>  ::= (and <p-effect>*)
<cond-effect>  ::= <p-effect>
<assign-op>    ::= assign
<assign-op>    ::= scale-up
<assign-op>    ::= scale-down
<assign-op>    ::= increase
<assign-op>    ::= decrease
```

A.3 Durative Actions

Durative action syntax is built on a relatively conservative extension of the existing action syntax.

```
<durative-action-def> ::= (:durative-action <da-symbol>
                           :parameters ( <typed list (variable)> )
                           <da-def body>)
<da-symbol>           ::= <name>
<da-def body>         ::= :duration <duration-constraint>
                          :condition <da-GD>
                          :effect <da-effect>
```

The conditions under which a durative action can be executed are more complex than for standard actions, in that they specify more than the conditions that must hold at the point of execution. They also specify the conditions that must hold throughout the duration of the durative action and also at its termination. To distinguish these components we introduce a simple temporal qualifier for the preconditions. The use of the name "precondition" would be somewhat misleading given that the conditions described can include constraints on what must hold after the action has begun. This has motivated the adoption of `:condition` to describe the collection of constraints that must hold in order to successfully apply a durative action. The logical form of conditions for durative actions has been restricted to conjunctions of temporally annotated expressions, but there is clearly scope for future extension to allow more complex formulae.

```
<da-GD>                ::= ()
<da-GD>                ::= <timed-GD>
<da-GD>                ::= (and <timed-GD>+)
<timed-GD>             ::= (at <time-specifier> <GD>)
<timed-GD>             ::= (over <interval> <GD>)
<time-specifier>       ::= start
<time-specifier>       ::= end
<interval>             ::= all
```

The duration (`?duration`) of a durative action can be specified to be equal to a given expression (which can be a function of numeric expressions), or else it can be constrained with inequalities. This latter allows for actions where the conclusion of the action can be freely determined by the executive without necessarily having further side-effects. For example, a walk between two locations could be made to take as long as the executive


```
<p效果>  ::=:fluents(<赋值操作符>  <f头>  <f表达式>)<条件效果>  ::=
(和  <p效果>*)<条件效果>  ::=  <p效果><赋值操作符>      ::=赋值<赋
值操作符>      ::=放大<赋值操作符>      ::=缩小<赋值操作符>      ::=
增加<赋值操作符>      ::=减少
```

A.3 持续动作

持续动作语法建立在现有动作语法的相对保守扩展之上。

```
<持续动作定义>  ::=  (:持续动作  <da符号>)
                                  :参数 (  <类型列表（变量）>  )
                                  <da-def body>)
<da-symbol>                ::=  <名字>
<da-def 身体>              ::=  :持续时间  <持续时间约束>
                                :条件   <da-GD>
                                :效果   <da-effect>
```

一个持续动作可以在何种条件下被执行比标准动作更复杂，因为它们指定了在执行点必须保持的条件之外的条件。它们还指定了在持续动作的整个持续期间以及在其终止时必须保持的条件。为了区分这些组成部分，我们引入了一个简单的时序限定符来表示前提条件。鉴于所描述的条件可以包括动作开始后必须保持的约束，“前提条件”这个名称的使用会有点误导。这促使我们采用:condition来描述为了成功应用持续动作而必须保持的约束集合。持续动作的条件逻辑形式已被限制为时序标注表达式的合取，但显然有空间在未来扩展以允许更复杂的公式。

```
<da-GD>                          ::=()<da-GD>                          ::
=  <timed-GD><da-GD>  ::=(and   <timed-GD>+)<timed-GD>  ::=
(at  <时间指定符>  <GD>)<timed-GD>  ::=  (over  <间隔>  <GD>)<时间
指定符>              ::=开始<时间指定符>              ::=结束<间隔
>                     ::=全部
```

一个持续动作的持续时间（?duration）可以指定等于一个给定的表达式（该表达式可以是数值表达式的函数），或者可以用不等式进行约束。后者允许执行者自由决定动作的结束时间，而不必产生进一步的副作用。例如，两个地点之间的散步可以设置为持续尽可能长的时间，由执行者决定

considered convenient, provided it was at least as long as the time taken to walk between the locations at the fastest walking speed possible. Constraints that do not specify the exact duration of a durative action might prove harder to handle, so we have introduced a label (`:duration-inequalities`) to signal that a domain makes use of them. A duration constraint is supplied to dictate or limit the temporal extent of the durative action. The duration is an implicit parameter of the durative action and must be supplied in a plan that uses durative actions. To denote this, a durative action is denoted in a plan by `t:(name arg1...argn)[d]` where `d` is the (non-negative, rational valued) duration in floating point format ($n.n$). Duration constraints can be explicitly temporally annotated to indicate that they should be evaluated in the context of the start or end point of the action, or else they can be left unannotated, in which case the default is that they are evaluated in the context at the start of the action (as indicated in Definition 16).

```
<duration-constraint>           ::= :duration-inequalities
                                    (and <simple-duration-constraint>+)
<duration-constraint>           ::= ()
<duration-constraint>           ::= <simple-duration-constraint>
<simple-duration-constraint>    ::= (<d-op> ?duration <d-value>)
<simple-duration-constraint>    ::= (at <time-specifier>
                                     <simple-duration-constraint>)
<d-op>                          ::=:duration-inequalities <=
<d-op>                          ::=:duration-inequalities >=
<d-op>                          ::= =
<d-value>                       ::= <number>
<d-value>                       ::=:fluents <f-exp>
```

In addition to logical effects, which can occur at the start or end of a durative action, durative actions can have numeric effects that refer to the literal `?duration`. More sophisticated durative actions can also make use of functional expressions describing effects that occur over the duration of the action. This allows functional expressions to be updated by a continuous function of time, rather than only step functions.

```
<da-effect>      ::= ()
<da-effect>      ::= (and <da-effect>*)
<da-effect>      ::= <timed-effect>
<da-effect>      ::=:conditional-effects (forall (<variable>*) <da-effect>)
<da-effect>      ::=:conditional-effects (when <da-GD> <timed-effect>)
<da-effect>      ::=:fluents (<assign-op> <f-head> <f-exp-da>)
<timed-effect>   ::= (at <time-specifier> <a-effect>)
<timed-effect>   ::= (at <time-specifier> <f-assign-da>)
<timed-effect>   ::=:continuous-effects (<assign-op-t> <f-head> <f-exp-t>)
<f-assign-da>    ::= (<assign-op> <f-head> <f-exp-da>)
<f-exp-da>       ::= (<binary-op> <f-exp-da> <f-exp-da>)
<f-exp-da>       ::= (- <f-exp-da>)
<f-exp-da>       ::=:duration-inequalities ?duration
<f-exp-da>       ::= <f-exp>
```

Note that the `?duration` term can only be used to define functional expression updating effects if the duration constraints requirement is set. This is because in other cases the duration value is available as an expression, whereas when duration constraints are provided the duration can, sometimes, be freely selected within constrained boundaries.

持续时间约束是用于规定或限制持续性行动的时序范围。持续时间是持续性行动的一个隐式参数，必须在使用持续性行动的计划中提供。为此，在计划中用 t:(name arg1...arg n)[d] 表示持续性行动，其中 d 是（非负的、有理值）持续时间，以浮点格式表示 ($n.n$)。持续时间约束可以显式地用时间标注来表示它们应该在行动的起点或终点上下文中进行评估，或者可以不标注，在这种情况下，默认是它们在行动开始时上下文中进行评估（如定义16中所示）。

```
<持续时间约束>          ::= :duration-inequalities
                            (and <简单持续时间约束>+)
<持续时间约束>          ::= ()
<持续时间约束>          ::= <简单持续时间约束>
<简单持续时间约束>      ::= (<d-op> ?持续时间 <d-value>)
<简单持续时间约束>      ::= (at <时间规范>
                             <简单持续时间约束>)
<d-op>                  ::=:duration-inequalities <=
<d-op>                  ::=:duration-inequalities >=
<d-op>                  ::= =
<d-value>               ::= <number>
<d-value>               ::=:fluents <f-exp>
```

除了逻辑效果（这些效果可能发生在持续动作的开始或结束处），持续动作还可以具有数值效果，这些效果指的是字面意义上的持续时间。更复杂的持续动作还可以使用描述动作持续期间发生的效果的功能表达式。这使得功能表达式能够通过时间的连续函数进行更新，而不仅仅是阶跃函数。

```
<da-effect>      ::= ()
<da-effect>      ::=(和 <da-effect>*)
<da-effect>      ::= <定时效果>
<da-effect>      ::=:conditional-effects (forall (<变量>*) <da-effect>)
<da-effect>      ::=:conditional-effects (当 <da-GD> <timed-effect>)
<da-effect>      ::=:fluents (<赋值操作> <函数头> <函数表达式>)
<定时效果>       ::= (在 <时间规范> <一个效果>)
<定时效果>       ::= (at <时间限定符> <f-赋值da>)
<定时效果>       ::=:continuous-effects (<赋值操作t> <f-头> <f-表达式t>)
<f-赋值da>       ::= (<赋值操作> <f-头> <f-表达式da>)
<f-exp-da>       ::=(<二进制运算> <浮点数减法> <浮点数减法>)
<f-exp-da>       ::= (- <f-exp-da>)
<f-exp-da>       ::=:duration-inequalities ?持续时间
<f-exp-da>       ::= <f-exp>
```

请注意，?duration 术语只能用于定义更新效果的函数表达式，前提是设置了持续时间约束要求。这是因为在其他情况下，持续时间值作为表达式可用，而当提供持续时间约束时，持续时间有时可以在受限边界内自由选择。

```
<assign-op-t>       ::= increase
<assign-op-t>       ::= decrease
<f-exp-t>           ::= (* <f-exp> #t)
<f-exp-t>           ::= (* #t <f-exp>)
<f-exp-t>           ::= #t
```

The symbol $\#t$ is used to represent the period of time that a given durative action has been active. It is therefore a *local* clock value for each duration, independent of similar clocks for each other duration. There has been discussion with members of the committee about the use of the expression using $\#t$: it was proposed that an expression declaring the rate of change alone could be used. We decided against this on the grounds that the assertion of a rate of change suggests that the rate of change is determined by one process effect alone. In fact, it is intended that if multiple active processes affect the same fluent then these effects are accumulated. Using the expression that directly defines the amount by which each process contributes to the change in a fluent value over time we do not appear to assert (inconsistently) that a fluent has multiple simultaneous rates of change.

A.4 Problems

Planning problems specifications have been modified to exclude several generally unused constructs (named initial situations and expansion information). We have removed the length specification because it is at odds with the intention to supply physics, not advice. Furthermore, the advice this field offers over-emphasises a very coarse plan metric. Instead, we have introduced an optional metric field, which can be used to supply an expression that should be optimized in the construction of a plan. The field states whether the metric is to be minimized or maximized. Of course, a planner is free to ignore this field and make the assumption that plans with fewest steps will be considered good plans. However, we consider this extension to be a crucial one in the development of a more widely applicable planning language. We have provided the variable `total-time` that takes the value of the total execution time for the plan. This allows us to conveniently express the intention to minimize total execution time.

We anticipate that extensions of the plan metric syntax will prove necessary in the longer term, but believe that this version already provides a significant new challenge to the community. Problem specifications are still somewhat impoverished in terms of the ability to easily specify temporal constraints on goals and other non-standard features of initial and goal states. Again, we anticipate the need for extension, but have chosen to leave a clean sheet for future developments.

```
<problem>             ::= (define (problem <name>)
                              (:domain <name>)
                              [<require-def>]
                              [<object declaration> ]
                              <init>
                              <goal>
                              [<metric-spec>]
                              [<length-spec> ])
<object declaration> ::= (:objects <typed list (name)>)
<init>                ::= (:init <init-el>*)
<init-el>             ::= <literal(name)>
```



```
<赋值运算符-t>      ::= 增加
<赋值运算符-t>      ::= 减少
<函数表达式-t>      ::=(* <函数表达式> #t)
<f-exp-t>           ::=(* #t <f-exp>)
<f-exp-t>           ::= #t
```

符号 $\#t$ 用于表示给定持续动作已激活的持续时间。因此，它是每个持续时间的本地时钟值，独立于其他持续时间的类似时钟。与委员会成员讨论过使用 $\#t$ 表达式的用法：有人提议仅使用声明变化速率的表达式。我们基于变化速率的断言表明该速率由单个过程效果单独决定，因此决定反对这一提议。实际上，如果多个活动过程影响同一流，则这些效果是累积的。使用直接定义每个过程对随时间变化的流值变化贡献量的表达式，我们似乎并未断言（不一致地）流具有多个同时变化速率。

A.4 问题

规划问题规范已被修改，以排除几个通常未使用的结构（命名初始情况和扩展信息）。我们已移除长度规范，因为它与提供物理意图而非建议的意图相冲突。此外，该字段提供的建议过分强调一个非常粗糙的计划度量标准。相反，我们引入了一个可选的度量字段，该字段可用于提供在构建计划时应优化的表达式。该字段声明度量标准是要最小化还是要最大化。当然，规划器可以忽略该字段，并假设步骤最少的计划将被视为好的计划。然而，我们认为这是在开发更广泛适用的规划语言中的一个关键扩展。我们提供了变量 total-time，其值为计划的总体执行时间。这使我们能够方便地表达最小化总体执行时间的意图。

我们预计，计划度量语法扩展在长期内将证明是必要的，但相信这个版本已经为社区提供了重大的新挑战。问题规范在轻松指定目标的时间约束以及初始状态和目标状态的其它非标准特征方面仍然有些贫乏。同样，我们预计需要扩展，但已选择为未来的发展留下一张白纸。

```
<问题>                ::= (define (问题 <名称>)
                              (:domain <名称>)
                              [<定义要求>]
                              [<对象声明> ]
                              <初始化>
                              <目标>
                              [<指标规范>]
                              [<长度规范> ])
<对象声明> ::= (:对象 <类型列表 (名称)>)
<初始化>              ::= (:初始化 <初始化元素>*)
<init-el>             ::= <literal(name)>
```

```
<init-el>            ::=:fluents (= <f-head> <number>)
<goal>               ::= (:goal <GD>)
<metric-spec>        ::= (:metric <optimization> <ground-f-exp>)
<optimization>       ::= minimize
<optimization>       ::= maximize
<ground-f-exp>       ::= (<binary-op> <ground-f-exp> <ground-f-exp>)
<ground-f-exp>       ::= (- <ground-f-exp>)
<ground-f-exp>       ::= <number>
<ground-f-exp>       ::= (<function-symbol> <name>*)
<ground-f-exp>       ::= total-time
<ground-f-exp>       ::= <function-symbol>
<length-spec>        ::= (:length [(:serial <integer>)]
                          [(:parallel <integer>)])
                     The length-spec is deprecated.
```

A.5 Requirements

Here is a table of all requirements in PDDL2.1. Some requirements imply others; some are abbreviations for common sets of requirements. If a domain stipulates no requirements, it is assumed to declare a requirement for `:strips`.

Requirement	*Description*
`:strips`	Basic STRIPS-style adds and deletes
`:typing`	Allow type names in declarations of variables
`:negative-preconditions`	Allow `not` in goal descriptions
`:disjunctive-preconditions`	Allow `or` in goal descriptions
`:equality`	Support = as built-in predicate
`:existential-preconditions`	Allow `exists` in goal descriptions
`:universal-preconditions`	Allow `forall` in goal descriptions
`:quantified-preconditions`	= `:existential-preconditions` + `:universal-preconditions`
`:conditional-effects`	Allow `when` in action effects
`:fluents`	Allow function definitions and use of effects using assignment operators and arithmetic preconditions.
`:adl`	= `:strips` + `:typing` + `:negative-preconditions` + `:disjunctive-preconditions` + `:equality` + `:quantified-preconditions` + `:conditional-effects`
`:durative-actions`	Allows durative actions. Note that this does not imply `:fluents`.
`:duration-inequalities`	Allows duration constraints in durative actions using inequalities.
`:continuous-effects`	Allows durative actions to affect fluents continuously over the duration of the actions.

<init-el> ::=:fluents (= <f-head> <number>)<goal> ::=(:goal <GD>)<metric-spec> ::=(:metric <optimization> <ground-f-exp>)<optimization> ::=minimize<optimization> ::=maximize<ground-f-exp> ::= (<binary-op> <ground-f-exp> <ground-f-exp>)<ground-f-exp> ::=(- <ground-f-exp>)<ground-f-exp> ::= <number><ground-f-exp> ::=(<function-symbol> <name>*)<ground-f-exp> ::=total-time<ground-f-exp> ::= <function-symbol><length-spec> ::=(:length [(:serial <integer>)](:parallel <integer>)) 长度规范已弃用。

A.5 要求

这是 PDDL2.1 中所有要求的表格。一些要求隐含其他要求；一些是常见要求集的缩写。如果一个领域没有规定要求，则假定它声明了对 :strips 的要求。

Requirement *Description*:strips 基本STRIPS风格的添加和删除 :typing 允许在变量声明中包含类型名称 :negative-preconditions 允许在目标描述中使用 not :disjunctive-preconditions 允许在目标描述中使用 or :equality 支持 = 作为内置谓词 :existential-preconditions 允许在目标描述中使用 exists :universal-preconditions 允许在目标描述中使用 forall :quantified-preconditions =:existential-preconditions+ :universal-preconditions :conditional-effects 允许在动作效果中使用 when :fluents 允许函数定义和使用赋值运算符和算术前件的效应。 :adl = :strips + :typing+ :negative-preconditions+ :disjunctive-preconditions+ :equality+ :quantified-preconditions+ :conditional-effects :durative-actions 允许持续动作。请注意，这并不隐含 :fluents。 :duration-inequalities 允许在持续动作中使用不等式来表示持续时间约束。 :continuous-effects 允许持续动作在动作持续时间内对 fluents 产生连续影响。

References

Allen, J. (1984). Towards a general theory of action and time. *Artificial Intelligence*, *23*, 123–154.

Allen, J. (1991). Planning as temporal reasoning. In *Proceedings of KR-91*, pp. 3–14.

Bacchus, F., & Ady, M. (2001). Planning with resources and concurrency: A forward chaining approach. In *Proceedings of IJCAI'01*, pp. 417–424.

Bacchus, F., & Kabanza, F. (1998). Planning for temporally extended goals. *Annals of Mathematics and Artificial Intelligence*, *22*, 5–27.

Bacchus, F., & Kabanza, F. (2000). Using temporal logic to express search control knowledge for planning. *Artificial Intelligence*, *116(1-2)*, 123–191.

Bacchus, F., Tenenberg, J., & Koomen, J. (1991). A non-reified temporal logic for AI. *Artificial Intelligence*, *52*, 87–108.

Blum, A., & Furst, M. (1995). Fast Planning through Plan-graph Analysis. In *Proceedings of IJCAI-95*.

Cesta, A., & Oddi, A. (1996). Gaining efficiency and flexibility in the simple temporal problem. In Chittaro, L., Goodwin, S., Hamilton, H., & Montanari, A. (Eds.), *Proceedings of TIME'96*.

Chapman, D. (1987). Planning for conjunctive goals. *Artificial Intelligence*, *29*, 333–377.

Dechter, R., Meiri, I., & Pearl, J. (1991). Temporal constraint networks. *Artificial Intelligence*, *49*.

Do, M. B., & Kambhampati, S. (2001). Sapa: a domain-independent heuristic metric temporal planner. In *Proceedings of ECP-01*.

Drabble, B., & Tate, A. (1994). The use of optimistic and pessimistic resource profiles to inform search in an activity based planner. In *Proceedings of AIPS-94*. AAAI Press.

El-Kholy, A., & Richards, B. (1996). Temporal and resource reasoning in planning: the ParcPlan approach. In *Proceedings of ECAI'96*.

Fikes, R., & Nilsson, N. (1971). STRIPS: A new approach to the application of theorem-proving to problem-solving. *Artificial Intelligence*, *2*(3), 189–208.

Fox, M., & Long, D. (2002). PDDL+ : Planning with time and metric resources. Tech. rep. Department of Computer Science, 21/02, University of Durham, UK. Available at: `http://www.dur.ac.uk/d.p.long/competition.html`.

Galipienso, M., & Sanchis, F. (2002). Representation and reasoning with disjunction temporal constraints. In *Proceedings of TIME'02*.

Garrido, A., Onaindía, E., & Barber, F. (2001). Time-optimal planning in temporal problems. In *Proceedings of ECP'01*.

Gazen, B., & Knoblock, C. (1997). Combining the expressivity of UCPOP with the efficiency of Graphplan. In *Proceedings of ECP-97*, pp. 221–233.

Gelfond, M., Lifschitz, V., & Rabinov, A. (1991). What are the limitations of the situation calculus?. In Boyer, R. (Ed.), *Essays in honor of Woody Bledsoe*, pp. 167–179. Kluwer Academic.

PDDL2.1: Expressing Temporal Planning Domains

参考文献

Allen, J. (1984). Towards a general theory of action and time. *Artificial Intelligence*, *23*, 123–154.

Allen, J. (1991). Planning as temporal reasoning. In *Proceedings of KR-91*, pp. 3–14.

Bacchus, F., & Ady, M. (2001). Planning with resources and concurrency: A forward chaining approach. In *Proceedings of IJCAI'01*, pp. 417–424.

Bacchus, F., & Kabanza, F. (1998). Planning for temporally extended goals. *Annals of Mathematics and Artificial Intelligence*, *22*, 5–27.

Bacchus, F., & Kabanza, F. (2000). Using temporal logic to express search control knowledge for planning. *Artificial Intelligence*, *116(1-2)*, 123–191.

Bacchus, F., Tenenberg, J., & Koomen, J. (1991). A non-reified temporal logic for AI. *Artificial Intelligence*, *52*, 87–108.

Blum, A., & Furst, M. (1995). Fast Planning through Plan-graph Analysis. In *Proceedings of IJCAI-95*.

Cesta, A., & Oddi, A. (1996). Gaining efficiency and flexibility in the simple temporal problem. In Chittaro, L., Goodwin, S., Hamilton, H., & Montanari, A. (Eds.), *Proceedings of TIME'96*.

Chapman, D. (1987). Planning for conjunctive goals. *Artificial Intelligence*, *29*, 333–377.

Dechter, R., Meiri, I., & Pearl, J. (1991). Temporal constraint networks. *Artificial Intelligence*, *49*.

Do, M. B., & Kambhampati, S. (2001). Sapa: a domain-independent heuristic metric temporal planner. In *Proceedings of ECP-01*.

Drabble, B., & Tate, A. (1994). The use of optimistic and pessimistic resource profiles to inform search in an activity based planner. In *Proceedings of AIPS-94*. AAAI Press.

El-Kholy, A., & Richards, B. (1996). Temporal and resource reasoning in planning: the ParcPlan approach. In *Proceedings of ECAI'96*.

Fikes, R., & Nilsson, N. (1971). STRIPS: A new approach to the application of theorem-proving to problem-solving. *Artificial Intelligence*, *2*(3), 189–208.

Fox, M., & Long, D. (2002). PDDL+ : Planning with time and metric resources. Tech. rep. Department of Computer Science, 21/02, University of Durham, UK. Available at: `http://www.dur.ac.uk/d.p.long/competition.html`.

Galipienso, M., & Sanchis, F. (2002). Representation and reasoning with disjunction temporal constraints. In *Proceedings of TIME'02*.

Garrido, A., Onaindía, E., & Barber, F. (2001). Time-optimal planning in temporal problems. In *Proceedings of ECP'01*.

Gazen, B., & Knoblock, C. (1997). Combining the expressivity of UCPOP with the efficiency of Graphplan. In *Proceedings of ECP-97*, pp. 221–233.

Gelfond, M., Lifschitz, V., & Rabinov, A. (1991). What are the limitations of the situation calculus?. In Boyer, R. (Ed.), *Essays in honor of Woody Bledsoe*, pp. 167–179. Kluwer Academic.

Ghallab, M., & Laruelle, H. (1994). Representation and control in IxTeT, a temporal planner. In *Proceedings of AIPS'94*.

Gupta, V., Henziner, T., & Jagadeesan, R. (1997). Robust timed automata. In *HART-97: Hybrid and Real-time Systems, LNCS 1201*, pp. 331–345. Springer-Verlag.

Haslum, P., & Geffner, H. (2001). Heuristic planning with time and resources. In *Proceedings of ECP'01, Toledo*.

Hayes, P., & Allen, J. (1987). Short time periods. In *Proceedings of IJCAI-87*, pp. 981–983.

Helmert, M. (2002). Decidability and undecidability results for planning with numerical state variables. In *Proceedings of AIPS-02*.

Henzinger, T. (1996). The theory of hybrid automata. In *Proceedings of the 11th Annual Symposium on Logic in Computer Science. Invited tutorial.*, pp. 278–292. IEEE Computer Society Press.

Henzinger, T., & Raskin, J.-F. (2000). Robust undecidability of timed and hybrid systems. In *Proceedings of the 3rd International Workshop on Hybrid Systems: Computation and Control. LNCS 1790.*, pp. 145–159. Springer-Verlag.

Howey, R., & Long, D. (2002). Validating plans with continuous effects. Tech. rep., Dept. Computer Science, University of Durham.

Jonsson, A., Morris, P., Muscettola, N., & Rajan, K. (2000). Planning in interplanetary space: theory and practice. In *Proceedings of AIPS-00*.

Kowalski, R., & Sergot, M. (1986). A logic-based calculus of events. *New Generation Computing*, *4*, 67–95.

Laborie, P., & Ghallab, M. (1995). Planning with sharable resource constraints. In *Proceedings of IJCAI-95*. Morgan Kaufmann.

Lifschitz, E. (1986). On the semantics of STRIPS. In *Proceedings of 1986 Workshop: Reasoning about Actions and Plans*.

Long, D., & Fox, M. (2003a). Exploiting a graphplan framework in temporal planning. In *Proceedings of ICAPS'03*.

Long, D., & Fox, M. (2003b). An overview and analysis of the results of the 3rd International Planning Competition. *Journal of Artifical Intelligence Research*, *this issue*.

McAllester, D., & Rosenblitt, D. (1991). Systematic nonlinear planning. In *Proceedings of AAAI'91*, Vol. 2, pp. 634–639, Anaheim, California, USA. AAAI Press/MIT Press.

McCarthy, J. (1980). Circumscription — a form of non-monotonic reasoning. *Artificial Intelligence*, *13*, 27–39.

McCarthy, J., & Hayes, P. (1969). Some philosophical problems from the standpoint of artificial intelligence. In Meltzer, B., & Michie, D. (Eds.), *Machine Intelligence 4*, pp. 463–502. Edinburgh University Press.

McDermott, D. (1982). A temporal logic for reasoning about processes and plans. *Cognitive Science*, *6*, 101–155.

McDermott, D. (2000). The 1998 AI planning systems competition. *AI Magazine*, *21*(2).

Fox & Long

Ghallab, M., & Laruelle, H. (1994). Representation and control in IxTeT, a temporal planner. In *Proceedings of AIPS'94*.

Gupta, V., Henziner, T., & Jagadeesan, R. (1997). Robust timed automata. In *HART-97: Hybrid and Real-time Systems, LNCS 1201*, pp. 331–345. Springer-Verlag.

Haslum, P., & Geffner, H. (2001). Heuristic planning with time and resources. In *Proceedings of ECP'01, Toledo*.

Hayes, P., & Allen, J. (1987). Short time periods. In *Proceedings of IJCAI-87*, pp. 981–983.

Helmert, M. (2002). Decidability and undecidability results for planning with numerical state variables. In *Proceedings of AIPS-02*.

Henzinger, T. (1996). The theory of hybrid automata. In *Proceedings of the 11th Annual Symposium on Logic in Computer Science. Invited tutorial.*, pp. 278–292. IEEE Computer Society Press.

Henzinger, T., & Raskin, J.-F. (2000). Robust undecidability of timed and hybrid systems. In *Proceedings of the 3rd International Workshop on Hybrid Systems: Computation and Control. LNCS 1790.*, pp. 145–159. Springer-Verlag.

Howey, R., & Long, D. (2002). Validating plans with continuous effects. Tech. rep., Dept. Computer Science, University of Durham.

Jonsson, A., Morris, P., Muscettola, N., & Rajan, K. (2000). Planning in interplanetary space: theory and practice. In *Proceedings of AIPS-00*.

Kowalski, R., & Sergot, M. (1986). A logic-based calculus of events. *New Generation Computing*, *4*, 67–95.

Laborie, P., & Ghallab, M. (1995). Planning with sharable resource constraints. In *Proceedings of IJCAI-95*. Morgan Kaufmann.

Lifschitz, E. (1986). On the semantics of STRIPS. In *Proceedings of 1986 Workshop: Reasoning about Actions and Plans*.

Long, D., & Fox, M. (2003a). Exploiting a graphplan framework in temporal planning. In *Proceedings of ICAPS'03*.

Long, D., & Fox, M. (2003b). An overview and analysis of the results of the 3rd International Planning Competition. *Journal of Artifical Intelligence Research*, *this issue*.

McAllester, D., & Rosenblitt, D. (1991). Systematic nonlinear planning. In *Proceedings of AAAI'91*, Vol. 2, pp. 634–639, Anaheim, California, USA. AAAI Press/MIT Press.

McCarthy, J. (1980). Circumscription — a form of non-monotonic reasoning. *Artificial Intelligence*, *13*, 27–39.

McCarthy, J., & Hayes, P. (1969). Some philosophical problems from the standpoint of artificial intelligence. In Meltzer, B., & Michie, D. (Eds.), *Machine Intelligence 4*, pp. 463–502. Edinburgh University Press.

McDermott, D. (1982). A temporal logic for reasoning about processes and plans. *Cognitive Science*, *6*, 101–155.

McDermott, D. (2000). The 1998 AI planning systems competition. *AI Magazine*, *21*(2).

122

McDermott, D. (2003). Reasoning about autonomous processes in an estimated-regression planner. In *Proceedings of ICAPS-03*.

McDermott, D., & the AIPS-98 Planning Competition Committee (1998). PDDL–the planning domain definition language. Tech. rep., Available at: `www.cs.yale.edu/homes/dvm`.

Muscettola, N. (1994). HSTS: Integrating planning and scheduling. In Zweben, M., & Fox, M. (Eds.), *Intelligent Scheduling*, pp. 169–212. Morgan Kaufmann, San Mateo, CA.

Nau, D., Cao, Y., Lotem, A., & Muñoz-Avila, H. (1999). SHOP: Simple hierarchical ordered planner. In *Proceedings of IJCAI'99*.

Pednault, E. (1986). Formulating multiagent, dynamic-world problems in the classical planning framework. In Georgeff, M., & Lansky, A. (Eds.), *Proceedings of the Timberline Oregon Workshop on Reasoning about Actions and Plans*.

Pednault, E. (1989). ADL: Exploring the middle ground between STRIPS and the situation calculus. In *Proceedings of KR-89*, pp. 324–332.

Penberthy, J., & Weld, D. (1994). Temporal planning with continuous change. In *Proceedings of AAAI-94*. AAAI/MIT Press.

Penberthy, J., & Weld, D. (1992). UCPOP: a sound, complete, partial-order planner for ADL. In *Proceedings of KR'92*, pp. 103–114, Los Altos, CA. Kaufmann.

Rabideau, G., Knight, R., Chien, S., Fukunaga, A., & Govindjee, A. (1999). Iterative repair planning for spacecraft operations in the ASPEN system. In *International Symposium on Artificial Intelligence Robotics and Automation in Space (i-SAIRAS)*.

Reichgelt, H. (1989). A comparison of first order and modal theories of time. In Jackson, P., Reichgelt, H., & van Harmelen, F. (Eds.), *Logic-based knowledge representation*, pp. 143–176. MIT Press.

Reiter, R. (1980). A logic for default reasoning. *Artificial Intelligence*, *13*, 81–132.

Russell, S., & Norvig, P. (1995). *Artificial Intelligence: a Modern Approach*. Prentice Hall.

Sandewall, E. (1994). *Features and fluents: the representation of knowledge about dynamical systems, volume I*. Oxford University Press.

Shanahan, M. (1990). Representing continuous change in the event calculus. In *Proceedings of ECAI'90*, pp. 598–603.

Shanahan, M. (1999). The event calculus explained. In Wooldridge, M., & Veloso, M. (Eds.), *Artificial Intelligence Today*, pp. 409–430. Springer Lecture Notes in Artificial Intelligence no. 1600.

Shoham, Y. (1985). Ten requirements for a theory of change. *New Generation Computing*, *3*, 467–477.

Smith, D., & Weld, D. (1999). Temporal planning with mutual exclusion reasoning. In *Proceedings of IJCAI-99, Stockholm*, pp. 326–337.

Tate, A. (1977). Generating project networks. In *Proceedings of IJCAI'77*.

van Bentham, J. (1983). *The logic of time*. Kluwer Academic Press, Dordrecht.

PDDL2.1: Expressing Temporal Planning Domains

McDermott, D. (2003). Reasoning about autonomous processes in an estimated-regression planner. In *Proceedings of ICAPS-03*.

McDermott, D., & the AIPS-98 Planning Competition Committee (1998). PDDL–the planning domain definition language. Tech. rep., Available at: `www.cs.yale.edu/homes/dvm`.

Muscettola, N. (1994). HSTS: Integrating planning and scheduling. In Zweben, M., & Fox, M. (Eds.), *Intelligent Scheduling*, pp. 169–212. Morgan Kaufmann, San Mateo, CA.

Nau, D., Cao, Y., Lotem, A., & Muñoz-Avila, H. (1999). SHOP: Simple hierarchical ordered planner. In *Proceedings of IJCAI'99*.

Pednault, E. (1986). Formulating multiagent, dynamic-world problems in the classical planning framework. In Georgeff, M., & Lansky, A. (Eds.), *Proceedings of the Timberline Oregon Workshop on Reasoning about Actions and Plans*.

Pednault, E. (1989). ADL: Exploring the middle ground between STRIPS and the situation calculus. In *Proceedings of KR-89*, pp. 324–332.

Penberthy, J., & Weld, D. (1994). Temporal planning with continuous change. In *Proceedings of AAAI-94*. AAAI/MIT Press.

Penberthy, J., & Weld, D. (1992). UCPOP: a sound, complete, partial-order planner for ADL. In *Proceedings of KR'92*, pp. 103–114, Los Altos, CA. Kaufmann.

Rabideau, G., Knight, R., Chien, S., Fukunaga, A., & Govindjee, A. (1999). Iterative repair planning for spacecraft operations in the ASPEN system. In *International Symposium on Artificial Intelligence Robotics and Automation in Space (i-SAIRAS)*.

Reichgelt, H. (1989). A comparison of first order and modal theories of time. In Jackson, P., Reichgelt, H., & van Harmelen, F. (Eds.), *Logic-based knowledge representation*, pp. 143–176. MIT Press.

Reiter, R. (1980). A logic for default reasoning. *Artificial Intelligence*, *13*, 81–132.

Russell, S., & Norvig, P. (1995). *Artificial Intelligence: a Modern Approach*. Prentice Hall.

Sandewall, E. (1994). *Features and fluents: the representation of knowledge about dynamical systems, volume I*. Oxford University Press.

Shanahan, M. (1990). Representing continuous change in the event calculus. In *Proceedings of ECAI'90*, pp. 598–603.

Shanahan, M. (1999). The event calculus explained. In Wooldridge, M., & Veloso, M. (Eds.), *Artificial Intelligence Today*, pp. 409–430. Springer Lecture Notes in Artificial Intelligence no. 1600.

Shoham, Y. (1985). Ten requirements for a theory of change. *New Generation Computing*, *3*, 467–477.

Smith, D., & Weld, D. (1999). Temporal planning with mutual exclusion reasoning. In *Proceedings of IJCAI-99, Stockholm*, pp. 326–337.

Tate, A. (1977). Generating project networks. In *Proceedings of IJCAI'77*.

van Bentham, J. (1983). *The logic of time*. Kluwer Academic Press, Dordrecht.

123

Vere, S. (1983). Planning in time: Windows and durations for activities and goals. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *5*.

Vila, L. (1994). A survey on temporal reasoning in artificial intelligence. *AI Communications*, *7*, 4–28.

Wilkins, D. (1988). *Practical Planning: Extending the Classical AI Planning Paradigm*. Morgan Kaufmann Publishers Inc., San Francisco, CA.

Fox & Long

Vere, S. (1983). Planning in time: Windows and durations for activities and goals. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *5*.

Vila, L. (1994). A survey on temporal reasoning in artificial intelligence. *AI Communications*, *7*, 4–28.

Wilkins, D. (1988). *Practical Planning: Extending the Classical AI Planning Paradigm*. Morgan Kaufmann Publishers Inc., San Francisco, CA.

124